

Table of Content

1	INTRODUCTION TO SQL	1
1.1	WHAT IS SQL?	1
1.2	UNDERSTANDING DATA	1
1.3	WHAT IS A DATABASE?	1
1.3.1	FUNCTIONS OF A DATABASE	1
1.3.2	TYPES OF DATABASES	1
1.3.3	RELATIONAL DATABASES	1
1.4	DATABASE MANAGEMENT SYSTEMS (DBMS)	2
1.4.1	RELATIONAL DATABASE MANAGEMENT SYSTEM (RDBMS)	2
1.4.2	EXAMPLES OF RDBMS	2
1.5	BASIC SQL COMMANDS	2
1.6	SUMMARY	2
2	INTRODUCTION TO DATA SCIENCE AND SQL	3
2.1	THE IMPORTANCE OF DATA IN THE MODERN WORLD	3
2.2	GROWTH OF THE DATA SCIENCE INDUSTRY	3
2.3	THE GROWING IMPORTANCE OF DATA SCIENCE	3
2.4	THE ROLE OF SQL IN DATA SCIENCE	3
2.4.1	WHY SQL IS ESSENTIAL	3
2.5	THE ROLE OF PYTHON IN SQL AND DATA SCIENCE	3
2.6	COURSE OVERVIEW	3
2.6.1	MODULE 1: INTRODUCTION TO SQL FOR DATA MANIPULATION	4
2.6.2	MODULE 2: DATA DEFINITION STATEMENTS	4
2.6.3	MODULE 3: ADVANCED SQL OPERATIONS	4
2.6.4	MODULE 4: ACCESSING DATABASES USING PYTHON	4
2.6.5	MODULE 5: REAL-WORLD PROJECT	4
2.6.6	MODULE 6: ADVANCED SQL CONCEPTS	4
2.7	SUMMARY	4
3	RETRIEVING DATA WITH A SELECT STATEMENT	4
3.1	INTRODUCTION	4
3.2	PURPOSE OF A DATABASE MANAGEMENT SYSTEM (DBMS)	5
3.3	UNDERSTANDING THE SELECT STATEMENT	5
3.3.1	BASIC SYNTAX OF SELECT	5
3.3.2	EXAMPLE: RETRIEVING DATA FROM A BOOK TABLE	5
3.4	USING THE WHERE CLAUSE TO FILTER DATA	5
3.4.1	WHAT IS A PREDICATE?	5
3.4.2	EXAMPLE: RETRIEVING A SPECIFIC BOOK	6
3.5	COMPARISON OPERATORS IN SQL	6

3.6	SUMMARY	6
4	<u>SELECT STATEMENT EXAMPLES</u>	6
4.1	INTRODUCTION	6
4.1.1	LEARNING OBJECTIVES	6
4.2	UNDERSTANDING THE SELECT STATEMENT	7
4.2.1	GENERAL SYNTAX	7
4.2.2	RETRIEVE ALL COLUMNS	7
4.2.3	USING THE WHERE CLAUSE	7
4.3	SELECT EXAMPLES	7
4.3.1	EXAMPLE 1: RETRIEVE ALL DATA	8
4.3.2	EXAMPLE 2: RETRIEVE SPECIFIC COLUMNS	8
4.3.3	EXAMPLE 3: RETRIEVE FILTERED ROWS USING WHERE	9
4.3.4	EXAMPLE 4: RETRIEVE SPECIFIC COUNTRY	9
4.4	SUMMARY	9
5	<u>HANDS-ON PRACTICE EXERCISE: SIMPLE SELECT STATEMENTS</u>	10
5.1	OBJECTIVES	10
5.2	SOFTWARE USED IN THIS PRACTICE EXERCISE	10
5.3	WORKING WITH DATASET	10
5.4	DATABASE USED IN THIS PRACTICE EXERCISE	10
5.5	EXPLORING THE DATABASE	10
5.5.1	COLUMN DESCRIPTIONS — FILMLOCATIONS TABLE	10
5.6	USING THE SELECT STATEMENT	11
5.6.1	EXAMPLE 1 — RETRIEVE ALL FILM DETAILS	11
5.6.2	EXAMPLE 2 — RETRIEVE FILM NAMES, DIRECTORS, AND WRITERS	11
5.6.3	EXAMPLE 3 — RETRIEVE FILMS RELEASED IN 2001 AND ONWARDS	11
5.7	PRACTICE EXERCISES ON THE SELECT STATEMENT	11
5.7.1	EXERCISE 1 — RETRIEVE FUN FACTS AND FILMING LOCATIONS	11
5.7.2	EXERCISE 2 — RETRIEVE FILMS RELEASED BEFORE OR IN 2000	11
5.7.3	EXERCISE 3 — RETRIEVE FILMS NOT WRITTEN BY JAMES CAMERON	11
5.8	CONCLUSION	12
6	<u>USEFUL SQL EXPRESSIONS IN SELECT STATEMENTS</u>	12
6.1	COUNT EXPRESSION	12
6.1.1	EXAMPLE 1 — RETRIEVE TOTAL NUMBER OF ROWS IN A TABLE	12
6.1.2	EXAMPLE 2 — RETRIEVE THE NUMBER OF ROWS MATCHING A CONDITION	12
6.2	DISTINCT EXPRESSION	12
6.2.1	EXAMPLE 1 — RETRIEVE UNIQUE VALUES FROM A COLUMN	12
6.2.2	EXAMPLE 2 — RETRIEVE UNIQUE COUNTRIES THAT RECEIVED GOLD MEDALS	12
6.3	LIMIT EXPRESSION	13
6.3.1	EXAMPLE 1 — RETRIEVE THE FIRST 10 ROWS OF A TABLE	13
6.3.2	EXAMPLE 2 — RETRIEVE A FEW ROWS FROM A SPECIFIC YEAR	13

6.4	SUMMARY	13
7	<u>HANDS-ON PRACTICE: COUNT, DISTINCT, LIMIT</u>	<u>13</u>
7.1	OBJECTIVES	13
7.2	EXPLORING THE DATABASE	13
7.2.1	FILMLocations TABLE COLUMNS	14
7.3	USING THE COUNT STATEMENT	14
7.3.1	EXAMPLE 1 — COUNT ALL RECORDS IN A TABLE	14
7.3.2	EXAMPLE 2 — COUNT LOCATIONS FOR A SPECIFIC WRITER	14
7.4	USING THE DISTINCT STATEMENT	14
7.4.1	EXAMPLE 1 — RETRIEVE ALL UNIQUE FILM TITLES	14
7.4.2	EXAMPLE 2 — RETRIEVE DISTINCT RELEASE YEARS FOR A PRODUCTION COMPANY	14
7.5	USING THE LIMIT STATEMENT	14
7.5.1	EXAMPLE 1 — RETRIEVE THE FIRST 25 ROWS FROM THE TABLE	14
7.5.2	EXAMPLE 2 — RETRIEVE 15 ROWS STARTING FROM THE 11TH ROW	14
7.6	PRACTICE EXERCISES	14
7.6.1	COUNT	14
7.6.2	DISTINCT	15
7.6.3	LIMIT	15
7.7	CONCLUSION	15
8	<u>INTRODUCTION TO THE INSERT STATEMENT</u>	<u>16</u>
8.1	OBJECTIVE	16
8.2	UNDERSTANDING THE INSERT STATEMENT	16
8.3	SYNTAX OF THE INSERT STATEMENT	16
8.3.1	EXPLANATION OF SYNTAX	16
8.4	EXAMPLE — INSERTING A SINGLE ROW	16
8.5	EXAMPLE — INSERTING MULTIPLE ROWS	17
8.6	SUMMARY	17
9	<u>INTRODUCTION TO THE UPDATE AND DELETE STATEMENTS</u>	<u>17</u>
9.1	OBJECTIVE	17
9.2	UNDERSTANDING THE UPDATE STATEMENT	17
9.2.1	SYNTAX OF THE UPDATE STATEMENT	18
9.2.2	EXPLANATION OF SYNTAX	18
9.2.3	EXAMPLE — UPDATING DATA	18
9.3	UNDERSTANDING THE DELETE STATEMENT	18
9.3.1	SYNTAX OF THE DELETE STATEMENT	18
9.3.2	EXPLANATION OF SYNTAX	19
9.3.3	EXAMPLE — DELETING DATA	19
9.4	SUMMARY	19
10	<u>HANDS-ON PRACTICE: INSERT, UPDATE, DELETE</u>	<u>19</u>

10.1	OBJECTIVES	19
10.2	UNDERSTANDING DML STATEMENTS	19
10.2.1	INSERT STATEMENT	19
10.2.2	UPDATE STATEMENT	20
10.2.3	DELETE STATEMENT	20
10.3	LAB ENVIRONMENT	20
10.3.1	DATABASE USED	20
10.4	EXERCISE 1: INSERT	20
10.4.1	EXAMPLE: INSERTING A SINGLE ROW	20
10.4.2	EXAMPLE: INSERTING MULTIPLE ROWS	20
10.4.3	PRACTICE TASK	21
10.5	EXERCISE 2: UPDATE	21
10.5.1	EXAMPLE: UPDATING A SINGLE COLUMN	21
10.5.2	EXAMPLE: UPDATING MULTIPLE COLUMNS	21
10.5.3	PRACTICE TASK	21
10.6	EXERCISE 3: DELETE	22
10.6.1	EXAMPLE: DELETING A ROW	22
10.6.2	PRACTICE TASK	22
10.7	SUMMARY	22
11	SUMMARY: BASIC SQL	22
12	DATABASE CONCEPTS	23
12.1	INTRODUCTION	23
12.2	RELATIONAL MODEL OVERVIEW	23
12.3	ENTITY RELATIONSHIP (ER) MODEL	23
12.3.1	EXAMPLE: LIBRARY DATABASE	24
12.3.2	PURPOSE OF ER MODEL	24
12.4	COMPONENTS OF AN ER DIAGRAM	24
12.4.1	ENTITIES	24
12.4.2	ATTRIBUTES	24
12.5	MAPPING ENTITIES TO TABLES	25
12.5.1	DATA TYPES	25
12.6	PRIMARY AND FOREIGN KEYS	25
12.6.1	PRIMARY KEY	26
12.6.2	FOREIGN KEY	26
12.7	SUMMARY	26
13	TYPES OF SQL STATEMENTS	26
13.1	INTRODUCTION	26
13.2	OVERVIEW OF SQL STATEMENTS	27
13.3	DATA DEFINITION LANGUAGE (DDL)	27
13.3.1	COMMON DDL STATEMENTS	27
13.3.2	PURPOSE OF DDL	27

13.4	DATA MANIPULATION LANGUAGE (DML)	27
13.4.1	COMMON DML STATEMENTS	27
13.4.2	PURPOSE OF DML	27
13.5	SUMMARY	28
14	<u>CREATE TABLE STATEMENT</u>	28
14.1	INTRODUCTION	28
14.2	UNDERSTANDING THE CREATE TABLE STATEMENT	28
14.2.1	SYNTAX	28
14.3	EXAMPLE 1: CREATING A SIMPLE TABLE	28
14.3.1	EXPLANATION	29
14.4	EXAMPLE 2: CREATING THE AUTHOR TABLE	29
14.4.1	EXPLANATION	29
14.5	KEY TAKEAWAYS	29
15	<u>ALTER, DROP, AND TRUNCATE TABLES</u>	29
15.1	INTRODUCTION	29
15.2	ALTER TABLE STATEMENT	30
15.2.1	SYNTAX	30
15.2.2	EXAMPLE 1: ADDING A COLUMN	30
15.2.3	EXAMPLE 2: MODIFYING A COLUMN	30
15.2.4	EXAMPLE 3: DROPPING A COLUMN	30
15.3	DROP TABLE STATEMENT	30
15.3.1	SYNTAX	31
15.3.2	EXAMPLE	31
15.4	TRUNCATE TABLE STATEMENT	31
15.4.1	SYNTAX	31
15.4.2	EXAMPLE	31
15.5	KEY TAKEAWAYS	31
16	<u>ALTER AND TRUNCATE TABLES USING MYSQL</u>	31
16.1	INTRODUCTION	31
16.2	ALTER TABLE STATEMENT	32
16.2.1	ADD COLUMN SYNTAX	32
16.2.2	MODIFY COLUMN SYNTAX	32
16.3	TRUNCATE TABLE STATEMENT	33
16.3.1	SYNTAX	33
17	<u>CREATE, ALTER, TRUNCATE, AND DROP IN MYSQL</u>	33
17.1	INTRODUCTION	33
17.2	SOFTWARE USED	33

17.3	OBJECTIVES	33
17.4	TASK 1: CREATE A DATABASE	33
17.5	TASK 2A: CREATE STATEMENT	34
17.6	TASK 2B: INSERT STATEMENT	34
17.7	TASK 3: ALTER STATEMENT	34
17.7.1	1. ADDING A COLUMN	34
17.7.2	2. DELETING A COLUMN	35
17.7.3	3. MODIFYING A COLUMN	35
17.7.4	4. RENAMING A COLUMN	35
17.8	TASK 4: TRUNCATE STATEMENT	35
17.9	TASK 5: DROP STATEMENT	35
17.10	PRACTICE PROBLEMS	35
17.11	CONCLUSION	36
18	<u>UNDERSTANDING RELATIONAL MODEL CONSTRAINTS</u>	<u>36</u>
18.1	OBJECTIVES	36
18.2	OVERVIEW	36
18.3	SAMPLE TABLES	37
18.3.1	BOOKSHOP TABLE	37
18.3.2	BOOKSHOP_AUTHORDETAILS TABLE	37
18.4	TYPES OF RELATIONAL MODEL CONSTRAINTS	37
18.4.1	ENTITY INTEGRITY CONSTRAINT	37
18.4.2	REFERENTIAL INTEGRITY CONSTRAINT	37
18.4.3	DOMAIN INTEGRITY CONSTRAINT	38
18.5	SUMMARY	39
19	<u>SQL SCRIPTS - USES AND APPLICATIONS</u>	<u>39</u>
19.1	SQL SCRIPTS	39
19.2	APPLICATIONS OF SQL SCRIPTS	39
19.2.1	CREATING TABLES	39
19.2.2	DROPPING TABLES	39
19.2.3	INSERTING DATA	39
19.2.4	UPDATING DATA	39
19.2.5	DELETING DATA	40
19.2.6	CREATING VIEWS	40
19.2.7	CREATING STORED PROCEDURES	40
19.2.8	CREATING TRIGGERS	40
19.3	EXAMPLE: CREATING TABLES	40
19.3.1	IMPORTANT NOTES ABOUT SQL SCRIPTS	40
19.3.2	SAMPLE SQL SCRIPT	40
19.3.3	EXPLANATION	41
19.3.4	EXECUTION IN PHPMYADMIN	41

<u>20</u>	<u>HANDS-ON PRACTICE EXERCISE: CREATE AND LOAD TABLES USING SQL SCRIPTS</u>	<u>42</u>
20.1	INTRODUCTION	42
20.2	OBJECTIVES	42
20.3	MYSQL OVERVIEW	42
20.4	DATABASE USED IN THIS EXERCISE	42
20.4.1	TASKS OVERVIEW	42
20.5	TASK 1: CREATE THE DATABASE	43
20.6	TASK 2: CREATE TABLES USING SQL SCRIPT	43
20.6.1	KEY POINTS ABOUT SQL SCRIPTS	43
20.6.2	STEPS TO EXECUTE THE SCRIPT	43
20.7	TASK 3: LOAD DATA INTO TABLES	43
20.7.1	STEPS TO LOAD DATA	43
20.7.2	PRACTICE EXERCISE	44
20.8	CONCLUSION	44
<u>21</u>	<u>SUMMARY: RELATIONAL DATABASE CONCEPTS AND TABLES</u>	<u>44</u>
<u>22</u>	<u>INTRODUCTION TO CLOUD DATABASES</u>	<u>44</u>
22.1	UNDERSTANDING CLOUD DATABASES	44
22.1.1	ADVANTAGES OF USING CLOUD DATABASES	45
22.1.2	EXAMPLES OF CLOUD DATABASES	45
22.2	CLOUD DATABASE SERVICE INSTANCES	45
22.2.1	CHARACTERISTICS OF A DATABASE SERVICE INSTANCE	45
22.3	CREATING AN IBM DB2 ON CLOUD SERVICE INSTANCE	45
22.3.1	STEPS TO CREATE A DB2 SERVICE INSTANCE	45
22.3.2	MANAGING THE SERVICE INSTANCE	46
22.4	SERVICE CREDENTIALS	46
22.4.1	CREATING AND USING SERVICE CREDENTIALS	46
22.5	SUMMARY	46
<u>23</u>	<u>RETRIEVING DATA WITH SELECT STATEMENT STRING PATTERNS</u>	<u>46</u>
23.1	INTRODUCTION	46
23.2	PURPOSE OF SELECT STATEMENT	47
23.3	USING STRING PATTERNS IN THE WHERE CLAUSE	47
23.3.1	THE LIKE PREDICATE	47
23.4	USING RANGES IN QUERIES	47
23.4.1	USING COMPARISON OPERATORS	47
23.4.2	USING THE BETWEEN OPERATOR	48
23.5	USING SETS OF VALUES IN QUERIES	48
23.5.1	USING THE IN OPERATOR	48
23.6	SUMMARY	48

24	<u>SORTING SELECT STATEMENT RESULT SETS</u>	48
24.1	INTRODUCTION	48
24.2	PURPOSE OF THE SELECT STATEMENT	49
24.3	USING THE ORDER BY CLAUSE	49
24.3.1	SORTING IN ASCENDING ORDER	49
24.3.2	SORTING IN DESCENDING ORDER	49
24.3.3	SORTING BY COLUMN SEQUENCE NUMBER	50
24.4	UNDERSTANDING SORTING BEHAVIOR	50
24.5	SUMMARY	50
25	<u>GROUPING SELECT STATEMENT RESULT SETS</u>	50
25.1	INTRODUCTION	50
25.2	ELIMINATING DUPLICATES FROM A RESULT SET	50
25.2.1	EXAMPLE	50
25.3	GROUPING DATA WITH THE GROUP BY CLAUSE	51
25.3.1	EXAMPLE	51
25.3.2	RENAMING COLUMNS WITH AS	51
25.4	FILTERING GROUPED DATA WITH THE HAVING CLAUSE	51
25.4.1	EXAMPLE	51
25.4.2	DIFFERENCE BETWEEN WHERE AND HAVING	51
25.5	SUMMARY	52
26	<u>HANDS-ON LAB: STRING PATTERNS, SORTING AND GROUPING IN MYSQL</u>	52
26.1	INTRODUCTION	52
26.2	OBJECTIVES	52
26.3	DATABASE USED	52
26.4	LOADING THE DATABASE	52
26.5	STRING PATTERNS	53
26.5.1	EXAMPLE 1: FILTER BY ADDRESS	53
26.5.2	EXAMPLE 2: FILTER BY BIRTH YEAR	53
26.5.3	EXAMPLE 3: FILTER BY SALARY AND DEPARTMENT	53
26.6	SORTING	53
26.6.1	EXAMPLE 1: ASCENDING ORDER BY DEPARTMENT	53
26.6.2	EXAMPLE 2: DESCENDING ORDER	53
26.7	GROUPING	53
26.7.1	EXAMPLE 1: COUNT EMPLOYEES PER DEPARTMENT	54
26.7.2	EXAMPLE 2: COUNT AND AVERAGE SALARY	54
26.7.3	EXAMPLE 3: SORTING GROUPED DATA	54
26.7.4	EXAMPLE 4: FILTERING GROUPED DATA WITH HAVING	54
26.8	PRACTICE QUESTIONS	54
26.9	SUMMARY	54
26.10	SUMMARY: REFINING YOUR RESULTS	55

27	SQL BUILT-IN FUNCTIONS	55
27.1	INTRODUCTION	55
27.2	OVERVIEW OF BUILT-IN SQL FUNCTIONS	55
27.3	THE PETRESCUE TABLE	55
27.4	AGGREGATE OR COLUMN FUNCTIONS	56
27.4.1	SUM FUNCTION	56
27.4.2	MIN AND MAX FUNCTIONS	56
27.4.3	AVG FUNCTION	56
27.5	SCALAR AND STRING FUNCTIONS	56
27.5.1	EXAMPLE: ROUND FUNCTION	57
27.5.2	STRING FUNCTIONS	57
27.6	SUMMARY	57
28	DATE AND TIME SQL FUNCTIONS	57
28.1	INTRODUCTION	57
28.2	EXTRACTING COMPONENTS OF DATE AND TIME	58
28.2.1	EXAMPLE: EXTRACTING THE DAY FROM A DATE	58
28.3	USING DATE AND TIME FUNCTIONS IN WHERE CLAUSES	58
28.3.1	EXAMPLE: GET NUMBER OF RESCUES IN MAY	58
28.4	PERFORMING DATE AND TIME ARITHMETIC	59
28.4.1	EXAMPLE: ADD 3 DAYS TO EACH RESCUE DATE	59
28.4.2	EXAMPLE: CALCULATE DAYS PASSED SINCE EACH RESCUE	59
28.5	SUMMARY	59
29	HANDS-ON PRACTICE: BUILT-IN FUNCTIONS	59
29.1	INTRODUCTION	59
29.2	SOFTWARE USED	59
29.3	OBJECTIVES	60
29.4	CREATING THE DATABASE	60
29.5	CREATING THE PETRESCUE TABLE	60
29.6	AGGREGATION FUNCTIONS	60
29.6.1	TOTAL COST OF ALL ANIMAL RESCUES	60
29.6.2	MAXIMUM AND MINIMUM QUANTITY OF ANIMALS RESCUED	60
29.6.3	AVERAGE COST OF ANIMAL RESCUES	60
29.7	SCALAR AND STRING FUNCTIONS	61
29.7.1	ROUNDED COST OF EACH RESCUE	61
29.7.2	LENGTH OF EACH ANIMAL NAME	61
29.7.3	CONVERT ANIMAL NAMES TO UPPER AND LOWER CASE	61
29.8	DATE FUNCTIONS	61
29.8.1	DISPLAY RESCUE DATES BY DAY, MONTH, AND YEAR	61
29.8.2	ADD OR SUBTRACT DAYS FROM RESCUE DATE	61
29.8.3	CALCULATE DAYS SINCE RESCUE	61
29.9	PRACTICE PROBLEMS	61

29.9.1	AVERAGE COST OF RESCUING A DOG	61
29.9.2	DISPLAY ANIMAL NAMES IN UPPERCASE WITHOUT DUPLICATES	61
29.9.3	DISPLAY ALL CAT RESCUES	61
29.9.4	NUMBER OF RESCUES IN THE 5TH MONTH	61
29.9.5	DISPLAY ID AND TARGET DATE (1 YEAR AFTER RESCUE)	61
29.10	CONCLUSION	61

30 SUBQUERIES AND NESTED SELECT STATEMENTS **62**

30.1	INTRODUCTION	62
30.2	WHAT ARE SUBQUERIES?	62
30.2.1	EXAMPLE OF A NESTED QUERY	62
30.3	WHY USE SUBQUERIES?	62
30.4	TYPES OF SUBQUERIES	62
30.4.1	1. SUBQUERIES IN THE WHERE CLAUSE	62
30.4.2	2. SUBQUERIES IN THE SELECT CLAUSE (COLUMN EXPRESSIONS)	62
30.4.3	3. SUBQUERIES IN THE FROM CLAUSE (DERIVED TABLES)	63
30.5	ADVANTAGES OF USING SUBQUERIES	63
30.6	SUMMARY	63

31 HANDS-ON PRACTICE: SUBQUERIES AND NESTED SELECT STATEMENTS **63**

31.1	OBJECTIVES	63
31.2	SOFTWARE USED	64
31.3	DATABASE OVERVIEW	64
31.4	LOADING THE DATABASE	64
31.5	UNDERSTANDING SUBQUERIES AND NESTED SELECTS	64
31.5.1	EXAMPLE 1: RETRIEVE EMPLOYEES WITH SALARY BELOW AVERAGE	64
31.5.2	EXAMPLE 2: DISPLAY EMPLOYEE SALARY WITH MAXIMUM SALARY COLUMN	65
31.5.3	EXAMPLE 3: RETRIEVE THE OLDEST EMPLOYEE	65
31.5.4	EXAMPLE 4: AVERAGE SALARY OF TOP 5 EARNERS	65
31.6	PRACTICE EXERCISES	65
31.6.1	EXERCISE 1: AVERAGE SALARY OF FIVE LOWEST EARNERS	65
31.6.2	EXERCISE 2: EMPLOYEES OLDER THAN AVERAGE AGE	65
31.6.3	EXERCISE 3: EMPLOYEE IDs AND YEARS OF SERVICE, AND AVERAGE YEARS OF SERVICE FROM JOB HISTORY	65
31.7	CONCLUSION	66

32 WORKING WITH MULTIPLE TABLES **66**

32.1	INTRODUCTION	66
32.2	EXAMPLE TABLES: EMPLOYEES AND DEPARTMENTS	66
32.2.1	EMPLOYEES TABLE	66
32.2.2	DEPARTMENTS TABLE	66
32.3	USING SUBQUERIES TO ACCESS MULTIPLE TABLES	67
32.3.1	EXAMPLE 1: RETRIEVE EMPLOYEES WITH MATCHING DEPARTMENT IDS	67

32.3.2	EXAMPLE 2: RETRIEVE EMPLOYEES FROM A SPECIFIC LOCATION	67
32.3.3	EXAMPLE 3: RETRIEVE DEPARTMENT DETAILS FOR HIGH-EARNING EMPLOYEES	67
32.4	ACCESSING MULTIPLE TABLES USING IMPLICIT JOINS	67
32.4.1	EXAMPLE 1: FULL JOIN EXAMPLE	67
32.4.2	EXAMPLE 2: FILTERING MATCHING DEPARTMENT IDS	68
32.4.3	EXAMPLE 3: USING TABLE ALIASES	68
32.4.4	EXAMPLE 4: DISPLAY DEPARTMENT NAME FOR EACH EMPLOYEE	68
32.5	CONCLUSION	68
33	<u>HANDS-ON LAB: WORKING WITH MULTIPLE TABLES</u>	68
33.1	OBJECTIVES	68
33.2	SOFTWARE USED	69
33.3	DATABASE USED	69
33.4	LOADING THE DATABASE	69
33.5	ACCESSING MULTIPLE TABLES WITH SUBQUERIES	69
33.5.1	EXAMPLE 1: RETRIEVE EMPLOYEE RECORDS FOR JOBS IN THE JOBS TABLE	69
33.5.2	EXAMPLE 2: RETRIEVE JOB INFORMATION FOR EMPLOYEES EARNING OVER \$70,000	69
33.6	ACCESSING MULTIPLE TABLES WITH IMPLICIT JOINS	70
33.6.1	EXAMPLE 1: RETRIEVE EMPLOYEE RECORDS FOR JOBS IN THE JOBS TABLE	70
33.6.2	EXAMPLE 2: USING TABLE ALIASES FOR SIMPLICITY	70
33.6.3	EXAMPLE 3: RETRIEVE EMPLOYEE ID, NAME, AND JOB TITLE	70
33.6.4	EXAMPLE 4: USE FULLY QUALIFIED COLUMN NAMES	70
33.7	PRACTICE PROBLEMS	70
33.7.1	PROBLEM 1: RETRIEVE EMPLOYEES WITH JOB_TITLE AS 'JR. DESIGNER'	70
33.7.2	PROBLEM 2: RETRIEVE JOB INFORMATION FOR EMPLOYEES BORN AFTER 1976	71
33.8	CONCLUSION	71
34	<u>SUMMARY: FUNCTIONS, MULTIPLE TABLES, AND SUB-QUERIES</u>	71
34.1	DATABASE MANAGEMENT TOOLS	71
34.2	EFFICIENCY WITH BUILT-IN FUNCTIONS	71
34.3	USING SUB-QUERIES	71
34.4	DERIVED TABLES OR TABLE EXPRESSIONS	72
34.5	KEY TAKEAWAYS	72
35	<u>ACCESSING DATABASES USING PYTHON</u>	72
35.1	INTRODUCTION	72
35.2	BENEFITS OF USING PYTHON FOR DATABASES	72
35.2.1	ADVANTAGES OF PYTHON	72
35.2.2	POPULAR PYTHON PACKAGES FOR DATA SCIENCE	72
35.3	PYTHON DATABASE API (DB API)	73
35.4	JUPYTER NOTEBOOKS	73
35.4.1	EXAMPLES OF NOTEBOOK INTERFACES	73
35.4.2	FEATURES OF JUPYTER NOTEBOOKS	73

35.5	ACCESSING DATABASES FROM JUPYTER NOTEBOOKS	73
35.6	SQL APIs AND PYTHON DB APIs	73
35.6.1	SQL API BASICS	74
35.6.2	PROPRIETARY APIs IN POPULAR SQL-BASED DBMS	74
35.7	SUMMARY	74
36	WRITING CODE USING DB-API	74
36.1	INTRODUCTION	74
36.2	UNDERSTANDING PYTHON DB-API	75
36.2.1	ADVANTAGES OF USING DB-API	75
36.3	CONNECTING PYTHON TO DATABASES	75
36.3.1	DATABASE LIBRARIES AND THEIR DB-APIs	75
36.4	MAIN CONCEPTS IN PYTHON DB-API	75
36.4.1	CONNECTION OBJECTS	75
36.4.2	CURSOR OBJECTS	76
36.4.3	CURSOR BEHAVIOR AND USAGE	76
36.5	WRITING PYTHON CODE WITH DB-API	76
36.6	SUMMARY	76
37	ACCESSING DATABASES USING SQL MAGIC	77
37.1	INTRODUCTION	77
37.2	WHAT ARE MAGIC STATEMENTS IN JUPYTER NOTEBOOKS?	77
37.3	LINE MAGICS	77
37.3.1	COMMONLY USED LINE MAGICS	77
37.3.2	EXAMPLE OF LINE MAGICS	77
37.4	CELL MAGICS	78
37.4.1	EXAMPLES OF CELL MAGICS	78
37.5	INSTALLING AND USING SQL MAGIC	78
37.5.1	STEP 1: INSTALL THE REQUIRED LIBRARY	78
37.5.2	STEP 2: LOAD THE SQL EXTENSION	78
37.5.3	STEP 3: ESTABLISH A DATABASE CONNECTION	78
37.5.4	STEP 4: RUN SQL QUERIES	79
37.6	SUMMARY	79
38	ANALYZING DATA WITH PYTHON	79
38.1	INTRODUCTION	79
38.2	ABOUT McDONALD'S DATASET	79
38.3	CREATING THE DATABASE	79
38.3.1	USING SQLITE3	80
38.3.2	STEPS TO LOAD DATA INTO THE DATABASE	80
38.4	RETRIEVING DATA USING PANDAS	80
38.4.1	EXPLORING THE DATASET	80
38.5	EXPLORING SODIUM CONTENT	81

38.5.1	UNDERSTANDING SODIUM	81
38.5.2	ANALYZING SODIUM IN McDONALD’S MENU	81
38.6	VISUALIZING DATA	81
38.6.1	SCATTER PLOT	81
38.6.2	BOX PLOT	82
38.7	SUMMARY	82
39	SUMMARY: ACCESSING DATABASES USING PYTHON	82
39.1	KEY CONCEPTS	82
39.1.1	MAGIC COMMANDS	82
39.1.2	CELL MAGICS	82
39.1.3	DATABASE API (DB API)	82
39.1.4	DATABASE CURSOR	82
39.2	USING PYTHON LIBRARIES FOR DATABASE OPERATIONS	82
39.2.1	PANDAS	82
39.2.2	SQLITE3	83
39.3	DATA VISUALIZATION	83
39.4	SUMMARY POINTS	83
40	WORKING WITH REAL-WORLD DATASETS	83
40.1	INTRODUCTION	83
40.2	UNDERSTANDING DATA FORMATS	83
40.2.1	EXAMPLE: DOGS.CSV FILE	83
40.3	LOADING CSV DATA INTO DATABASES	84
40.4	WORKING WITH COLUMN NAMES	84
40.5	WRITING READABLE SQL QUERIES	84
40.5.1	IN PYTHON NOTEBOOKS	84
40.6	QUERYING DATABASES USING PYTHON	85
40.6.1	EXAMPLE USAGE	85
40.7	LIMITING QUERY RESULTS	85
40.7.1	EXAMPLE QUERY	85
40.8	SUMMARY	85
41	GETTING INFORMATION ABOUT TABLES AND COLUMNS IN A DATABASE	85
41.1	INTRODUCTION	85
41.2	RETRIEVING A LIST OF TABLES	86
41.2.1	CATALOG TABLES IN DIFFERENT DATABASE SYSTEMS	86
41.2.2	EXAMPLE: SQLITE3	86
41.2.3	EXAMPLE: MYSQL	86
41.3	RETRIEVING COLUMN INFORMATION	86
41.3.1	IN SQLITE3	86
41.3.2	IN MYSQL	86
41.4	SUMMARY	86

42	SUMMARY	87
42.1	UNDERSTANDING REAL-WORLD DATASETS	87
42.2	DATABASE STRUCTURE KNOWLEDGE	87
42.3	QUERY DESIGN SKILLS	87
43	VIEWS	87
43.1	INTRODUCTION	87
43.2	WHAT IS A VIEW?	87
43.3	WHEN TO USE A VIEW	88
43.4	CREATING A VIEW	88
43.4.1	SYNTAX	88
43.4.2	EXAMPLE	88
43.5	CHARACTERISTICS OF VIEWS	88
43.6	REMOVING A VIEW	89
43.7	SUMMARY	89
44	HANDS-ON EXERCISE: USING VIEWS IN MYSQL USING THE COMMANDLINE	89
44.1	INTRODUCTION	89
44.2	OBJECTIVES	89
44.3	SOFTWARE USED	89
44.4	DATABASE USED	89
44.4.1	SETUP INSTRUCTIONS	90
44.5	TASK 1: CREATE A VIEW	90
44.5.1	STEP 1: CREATE THE VIEW	90
44.5.2	STEP 2: QUERY THE VIEW	90
44.6	TASK 2: UPDATE A VIEW	90
44.6.1	STEP 1: REPLACE THE VIEW	90
44.6.2	STEP 2: QUERY THE UPDATED VIEW	90
44.6.3	NOTE	90
44.7	TASK 3: DROP A VIEW	90
44.7.1	STEP 1: DROP THE EMPSALARY VIEW	91
44.7.2	STEP 2: VERIFY DELETION	91
44.8	PRACTICE PROBLEMS	91
44.8.1	PROBLEM 1: CREATE A NEW VIEW	91
44.8.2	PROBLEM 2: MODIFY THE VIEW	91
44.8.3	PROBLEM 3: DROP THE VIEW	91
44.9	CONCLUSION	91
45	INTRODUCTION TO STORED PROCEDURES	91
45.1	WHAT IS A STORED PROCEDURE?	91
45.2	BENEFITS OF STORED PROCEDURES	92
45.2.1	1. REDUCED NETWORK TRAFFIC	92

45.2.2	2. IMPROVED PERFORMANCE	92
45.2.3	3. CODE REUSABILITY	92
45.2.4	4. INCREASED SECURITY	92
45.3	CREATING A STORED PROCEDURE IN SQL	92
45.3.1	EXAMPLE	92
45.3.2	EXPLANATION	93
45.4	CALLING A STORED PROCEDURE	93
45.4.1	EXAMPLE	93
45.5	SUMMARY	93
46	HANDS-ON PRACTICE EXERCISE: STORED PROCEDURES	93
46.1	OVERVIEW	93
46.2	OBJECTIVES	94
46.3	SOFTWARE USED	94
46.4	DATABASE USED	94
46.5	DATA USED	94
46.5.1	SETTING UP THE DATABASE	94
47	STORED PROCEDURE: EXERCISE 1 – RETRIEVING DATA	94
47.1	GOAL	94
47.1.1	STEP 1: CREATE THE STORED PROCEDURE	94
47.1.2	STEP 2: EXECUTE THE STORED PROCEDURE	95
47.1.3	STEP 3: VIEW OR DROP THE PROCEDURE	95
48	STORED PROCEDURE: EXERCISE 2 – UPDATING DATA	95
48.1	GOAL	95
48.1.1	STEP 1: CREATE THE STORED PROCEDURE	95
48.1.2	STEP 2: EXECUTE THE STORED PROCEDURE	95
48.1.3	STEP 3: VIEW OR DROP THE PROCEDURE	96
49	CONCLUSION	96
50	ACID TRANSACTIONS	96
50.1	OVERVIEW	96
50.2	EXAMPLE OF AN ACID TRANSACTION	96
50.3	COMPONENTS OF ACID	97
50.3.1	ATOMIC	97
50.3.2	CONSISTENT	97
50.3.3	ISOLATED	97
50.3.4	DURABLE	97
50.4	MANAGING ACID TRANSACTIONS	97

50.4.1	COMMIT	97
50.4.2	ROLLBACK	97
50.5	USING SQL COMMANDS IN PROGRAMMING LANGUAGES	97
50.6	SUMMARY	98
<u>51</u>	<u>HANDS-ON LAB: COMMITTING AND ROLLING BACK A TRANSACTION</u>	<u>98</u>
51.1	OVERVIEW	98
51.2	OBJECTIVES	98
51.3	SOFTWARE USED	98
51.4	DATABASE USED	98
51.5	DATA USED	99
51.5.1	SETTING UP THE DATABASE	99
51.6	EXAMPLE: COMMITTING AND ROLLING BACK A TRANSACTION	99
51.6.1	SCENARIO	99
51.6.2	STORED PROCEDURE: TRANSACTION_ROSE	99
51.6.3	EXECUTING THE PROCEDURE	100
51.6.4	OBSERVATION	100
51.6.5	EXPECTED OUTCOME	100
51.7	PRACTICE EXERCISE	100
51.7.1	HINT	100
51.7.2	SOLUTION	100
51.8	CONCLUSION	101
51.8.1	KEY TAKEAWAYS	101
<u>52</u>	<u>SUMMARY: VIEWS, STORED PROCEDURES, AND TRANSACTIONS</u>	<u>101</u>
52.1	OVERVIEW	101
52.2	VIEWS	101
52.3	TRANSACTIONS	101
52.4	STORED PROCEDURES	102
52.5	KEY TAKEAWAYS	102
<u>53</u>	<u>INTRODUCTION TO JOIN OVERVIEW</u>	<u>102</u>
53.1	UNDERSTANDING THE JOIN OPERATOR	102
53.2	ROLE OF PRIMARY KEYS AND FOREIGN KEYS	102
53.2.1	EXAMPLE OF RELATIONSHIP	102
53.3	ESTABLISHING RELATIONSHIPS BETWEEN TABLES	103
53.4	COMBINING MULTIPLE TABLES	103
53.5	TYPES OF JOIN OPERATORS	103
53.5.1	INNER JOIN	103
53.5.2	OUTER JOIN	103
53.6	SUMMARY	103
<u>54</u>	<u>INTRODUCTION TO INNER JOIN</u>	<u>104</u>

54.1	UNDERSTANDING INNER JOINS	104
54.2	SYNTAX OF THE INNER JOIN STATEMENT	104
54.2.1	EXAMPLE SCENARIO	104
54.2.2	EXPLANATION OF CLAUSES	104
54.2.3	USE OF ALIASES	104
54.2.4	RESULT SET EXPLANATION	104
54.3	KEY POINTS TO REMEMBER	105
54.4	SUMMARY	105

55 INTRODUCTION TO OUTER JOIN **105**

55.1	UNDERSTANDING OUTER JOINS	105
55.2	LEFT OUTER JOIN	106
55.2.1	SYNTAX OF LEFT OUTER JOIN	106
55.2.2	EXAMPLE SCENARIO	106
55.2.3	EXPLANATION OF THE RESULT SET	106
55.3	RIGHT OUTER JOIN	106
55.3.1	SYNTAX OF RIGHT OUTER JOIN	106
55.3.2	EXAMPLE SCENARIO	106
55.3.3	EXPLANATION OF THE RESULT SET	106
55.4	FULL OUTER JOIN	107
55.4.1	SYNTAX OF FULL OUTER JOIN	107
55.4.2	EXAMPLE SCENARIO	107
55.4.3	EXPLANATION OF THE RESULT SET	107
55.5	KEY POINTS TO REMEMBER	107
55.6	SUMMARY	107

56 HANDS-ON PRACTICE: WORKING WITH JOINS IN MYSQL USING PHPMYADMIN **107**

56.1	INTRODUCTION	107
56.2	OBJECTIVES	108
56.3	SOFTWARE USED	108
56.4	DATABASE OVERVIEW	108
56.5	LOADING THE DATABASE	108
56.5.1	SETTING UP THE DATABASE	108
56.6	WORKING WITH JOINS	109
56.6.1	EXAMPLE 1: RETRIEVE NAMES AND JOB START DATES OF EMPLOYEES IN DEPARTMENT 5	109
56.6.2	EXAMPLE 2: RETRIEVE EMPLOYEE ID, LAST NAME, DEPARTMENT ID, AND DEPARTMENT NAME	109
56.6.3	EXAMPLE 3: RETRIEVE FIRST NAME, LAST NAME, AND DEPARTMENT NAME OF ALL EMPLOYEES	109
56.7	PRACTICE PROBLEMS	109
56.7.1	PROBLEM 1	109
56.7.2	PROBLEM 2	110
56.7.3	PROBLEM 3	110

56.8	CONCLUSION	110
<u>57</u>	<u>SUMMARY: JOIN STATEMENTS</u>	<u>110</u>

Module 1

1 Introduction to SQL

1.1 What is SQL?

SQL (Structured Query Language) is a programming language used to manage and manipulate data in **relational databases**. It allows users to **query, retrieve, and modify data** efficiently. SQL is one of the foundational tools in data management and data science.

1.2 Understanding Data

Data is a **collection of facts**—it can exist in the form of words, numbers, or even images. In modern times, **data is one of the most valuable assets** for any organization. For example:

- Banks store customer data such as names, addresses, phone numbers, and account details.
- Credit card companies and payment platforms like PayPal store transaction and identity information.

Because data is so valuable, it must be **stored securely** and **accessed efficiently**. This is where databases come into play.

1.3 What is a Database?

A **database** is a structured system designed to store, organize, and retrieve data efficiently. Databases are used everywhere—from banking systems to healthcare to online applications.

1.3.1 Functions of a Database

A database provides functionality to:

- **Add new data**
- **Modify existing data**
- **Query or retrieve data**

1.3.2 Types of Databases

Different types of databases exist for different requirements. One of the most common types is the **relational database**, where data is stored in **tables**.

1.3.3 Relational Databases

In a **relational database**, data is stored in tabular form—**rows and columns**, much like a spreadsheet.

- **Columns** represent attributes such as *FirstName*, *LastName*, *Email*, and *City*.
- **Rows** represent individual records.

A **table** is a collection of related data—for example, a table of employees or authors. In relational databases, you can establish **relationships** between tables, allowing complex data analysis and retrieval.

1.4 Database Management Systems (DBMS)

A **Database Management System (DBMS)** is software that manages databases. It allows users to **store, modify, and query** data efficiently.

1.4.1 Relational Database Management System (RDBMS)

An **RDBMS** is a specific type of DBMS designed for relational databases. It manages how data is stored, accessed, and organized. RDBMS technology forms the **backbone of many industries**, including banking, healthcare, and transportation.

1.4.2 Examples of RDBMS

Some popular relational database systems include:

- **MySQL**
- **Oracle Database**
- **DB2 Warehouse**
- **DB2 on Cloud**

1.5 Basic SQL Commands

SQL provides a set of fundamental commands that allow users to interact with databases. The five most common commands are:

1. **CREATE** – to create a new table
2. **INSERT** – to add data to a table
3. **SELECT** – to retrieve data from a table
4. **UPDATE** – to modify existing data
5. **DELETE** – to remove data from a table

1.6 Summary

- SQL is a language used for managing and querying data in relational databases.
- Data is one of the most critical assets of any business.
- Databases store and organize data for quick and secure access.
- Relational databases use tables, rows, and columns to store structured data.
- RDBMS software manages relational databases efficiently.
- The five essential SQL commands are **CREATE, INSERT, SELECT, UPDATE, and DELETE**.

By understanding SQL and databases, you gain the foundation needed to work with data effectively across multiple industries.

2 Introduction to Data Science and SQL

2.1 The Importance of Data in the Modern World

In this data-centric era, the role of data and the structures used to support its use have become increasingly essential to humanity and the world at large. Data science has grown into a **multi-billion-dollar industry** and continues to expand as we discover new applications across various domains, including science, politics, business, and everyday life.

2.2 Growth of the Data Science Industry

Analysts estimate that the **data science industry will surpass \$500 billion** by the end of this decade. As a data scientist, you can expect an **average annual salary of \$124,000**—an exciting and rewarding career path. Every day, we generate and store **trillions of gigabytes of data**, making the ability to analyze, manipulate, and manage this information increasingly critical.

2.3 The Growing Importance of Data Science

As the world moves toward a more data-driven future, the importance of data science continues to rise. In today's rapidly evolving digital landscape, **databases** serve as vast repositories that store and organize this data efficiently.

2.4 The Role of SQL in Data Science

SQL (Structured Query Language) is the key that unlocks these data repositories, allowing users to **query, update, and manipulate data** effectively. It plays a vital role in the **data science workflow**, empowering professionals to extract meaningful insights from large and complex datasets.

2.4.1 Why SQL is Essential

- SQL is **powerful, reliable, and globally relevant**, used widely in both industry and academia.
- It enables **efficient data access** and **data manipulation** within relational databases.
- It is a foundational tool for **data scientists, analysts, and engineers** worldwide.

2.5 The Role of Python in SQL and Data Science

A versatile language, **Python** is often used in conjunction with SQL to connect to databases, execute queries, and retrieve results in a format suitable for further data analysis. Python offers numerous **libraries** that simplify the process of connecting and interacting with databases.

2.6 Course Overview

This course aims to equip you with the **knowledge and skills** to harness the power of **databases and SQL** in your data science journey.

2.6.1 Module 1: Introduction to SQL for Data Manipulation

Learn the basics of SQL for performing **data manipulation** operations within a database table.

2.6.2 Module 2: Data Definition Statements

Understand how to use **data definition statements** and learn to **create and modify database tables**.

2.6.3 Module 3: Advanced SQL Operations

Explore **built-in functions**, **sub-queries**, and **working with multiple tables**.

2.6.4 Module 4: Accessing Databases Using Python

Learn how to use **Python and Jupyter Notebooks** to connect to and interact with databases.

2.6.5 Module 5: Real-World Project

Work on a **real-world project** designed to test and reinforce your SQL and Python skills.

2.6.6 Module 6: Advanced SQL Concepts

Delve into advanced database topics, including:

- **Stored Procedures**
- **Views**
- **ACID Transactions**
- **Inner and Outer JOINS**

2.7 Summary

This course offers an engaging, interactive, and effective learning experience designed to help you master SQL for Data Science. With a focus on real-world application, you will gain practical expertise in databases, SQL, and Python integration.

3 Retrieving Data with a SELECT Statement

3.1 Introduction

In this lesson, we explore how to **retrieve data from a relational database table** using the **SELECT statement**. We'll study.

- Retrieve data from a relational database table.
- Understand the use of a predicate.
- Identify the syntax of the SELECT statement using the WHERE clause.
- List the comparison operators supported by a relational database management system (RDBMS).

3.2 Purpose of a Database Management System (DBMS)

The primary goal of a **Database Management System** is not only to store data but also to **facilitate its retrieval**. Once we create a table and insert data, we often want to view that data. To achieve this, we use the **SELECT statement**, one of the most important Data Manipulation Language (DML) statements.

3.3 Understanding the SELECT Statement

Data Manipulation Language (DML) statements are used to read and modify data. The **SELECT statement** is referred to as a *query*, and its output is known as a **result set** or **result table**.

3.3.1 Basic Syntax of SELECT

The simplest form of a SELECT statement is:

```
SELECT * FROM table_name;
```

This command retrieves **all columns and rows** from the specified table.

3.3.2 Example: Retrieving Data from a Book Table

Consider a table named **book** created using attributes from a book entity. After inserting data, the query:

```
SELECT * FROM book;
```

would display all columns and rows from the book table — for example, four rows of data.

Alternatively, we can specify the columns individually:

```
SELECT book_id, title FROM book;
```

This retrieves only the **book_id** and **title** columns for each row. The order of the columns in the output matches the order in the SELECT statement.

3.4 Using the WHERE Clause to Filter Data

Sometimes, we need to retrieve specific data instead of all records. The **WHERE clause** allows us to apply **conditions** to restrict the result set.

3.4.1 What is a Predicate?

A **predicate** is a condition that evaluates to **TRUE**, **FALSE**, or **UNKNOWN**. It is used in the **WHERE** clause to define search criteria.

3.4.2 Example: Retrieving a Specific Book

If we want to know the title of the book whose `book_id` is **B1**, we can write:

```
SELECT book_id, title FROM book WHERE book_id = 'B1';
```

This query restricts the result set to only one row — the one that satisfies the condition.

3.5 Comparison Operators in SQL

SQL supports several comparison operators that can be used in predicates within the **WHERE** clause:

- `=` (Equal to)
- `>` (Greater than)
- `<` (Less than)
- `>=` (Greater than or equal to)
- `<=` (Less than or equal to)
- `<>` or `!=` (Not equal to)

These operators help refine searches and control the data retrieved from a table.

3.6 Summary

- The **SELECT statement** is used to retrieve data from database tables.
- A **predicate** defines the condition used to filter data with the **WHERE** clause.
- The **SELECT** statement can retrieve all or specific columns from a table.
- **Comparison operators** help filter data based on specific conditions.

By mastering the **SELECT** statement and its clauses, you can effectively query and retrieve precise data from relational databases.

4 SELECT Statement Examples

4.1 Introduction

In this section, you will learn how to use the **SELECT statement** to retrieve data from a database table. The **SELECT** command is one of the most commonly used **Database Query commands** in SQL.

4.1.1 Learning Objectives

At the end of this reading, you will be able to:

- Use various **SELECT** queries to retrieve data from a database.
- Apply the **WHERE** clause to filter data.

4.2 Understanding the SELECT Statement

The **SELECT** statement retrieves information from one or more database tables. It can be written in several forms depending on the requirement.

4.2.1 General Syntax

The general syntax of a **SELECT** statement is:

```
SELECT COLUMN1, COLUMN2, ... FROM TABLE_1;
```

This query retrieves data from the specified columns of **TABLE_1**.

4.2.2 Retrieve All Columns

To retrieve all columns from a table, use the ***** wildcard instead of listing each column name:

```
SELECT * FROM TABLE_1;
```

This retrieves the entire table.

4.2.3 Using the WHERE Clause

To filter data based on specific conditions, use the **WHERE** clause:

```
SELECT <COLUMNS> FROM TABLE_1 WHERE <predicate>;
```

The **predicate** defines the condition to filter the results.

4.3 SELECT Examples

Below is an example database table named **COUNTRY**, which contains the columns **ID**, **Name**, and **CCode**. Here, **CCode** represents a two-letter country code.

ID	Name	CCode
1	United States of America	US
2	China	CH
3	Japan	JA
4	Germany	GE
5	India	IN

6	United Kingdom	UK
7	France	FR
8	Italy	IT
9	Canada	CA
10	Brazil	BR

4.3.1 Example 1: Retrieve All Data

SELECT * FROM COUNTRY;

Explanation:

- SELECT * instructs the database to retrieve all columns.
- FROM COUNTRY specifies the table to retrieve data from.

Response:

All rows and columns from the COUNTRY table are displayed.

4.3.2 Example 2: Retrieve Specific Columns

SELECT ID, Name FROM COUNTRY;

Explanation:

- SELECT ID, Name retrieves only the specified columns.
- FROM COUNTRY defines the table to query.

Response:

ID	Name
1	United States of America
2	China
3	Japan
4	Germany
5	India
6	United Kingdom
7	France
8	Italy

9	Canada
10	Brazil

4.3.3 Example 3: Retrieve Filtered Rows Using WHERE

SELECT * FROM COUNTRY WHERE ID <= 5;

Explanation:

- SELECT * retrieves all columns.
- WHERE ID <= 5 filters rows where the ID value is less than or equal to 5.

Response:

ID	Name	CCode
1	United States of America	US
2	China	CH
3	Japan	JA
4	Germany	GE
5	India	IN

4.3.4 Example 4: Retrieve Specific Country

SELECT * FROM COUNTRY WHERE CCode = 'CA';

Explanation:

- SELECT * retrieves all columns.
- WHERE CCode = 'CA' filters rows where the CCode value equals 'CA'.

Response:

ID Name CCode

9 Canada CA

4.4 Summary

- The **SELECT** statement retrieves data from database tables.
- Use * to retrieve all columns.
- Use the **WHERE** clause to filter results based on conditions.
- You can specify individual columns to limit retrieved data.

5 Hands-on Practice Exercise: Simple SELECT Statements

In this practice exercise, we will learn one of the most commonly used statements in SQL (Structured Query Language) — the **SELECT** statement. The **SELECT** statement is used to select data from a database.

5.1 Objectives

After completing this exercise, we will be able to:

- Query a database to obtain a response as a result set.
- Retrieve all or selected columns of a dataset.
- Apply criteria commands to filter the result set.

5.2 Software Used in This Practice Exercise

We will use **Datasette**, an open-source tool for exploring and publishing data. The Datasette GitHub repository can be visited for reference and additional details.

5.3 Working with Dataset

The Datasette tool provides a platform to input and execute SQL queries. By clicking the **Submit Query** button, we can execute the SQL query and view the result set.

5.4 Database Used in This Practice Exercise

The database used in this practice exercise comes from the dataset source **Film Locations in San Francisco** under a **PDDL: Public Domain Dedication and License**.

5.5 Exploring the Database

Let's first explore the **SanFranciscoFilmLocations** database using the Datasette tool.

If the first statement below is not already in the Datasette textbox, copy and paste the following code into the textbox and click **Submit Query**:

```
SELECT * FROM FilmLocations;
```

After running this query, we can scroll through the table and explore all the columns and rows of the **FilmLocations** table to get an overall idea of its contents.

5.5.1 Column Descriptions — FilmLocations Table

- **Title:** Titles of the films
- **ReleaseYear:** Year of public release of the films
- **Locations:** Locations in San Francisco where the films were shot

- **FunFacts:** Fun facts about the filming locations
- **ProductionCompany:** Companies that produced the films
- **Distributor:** Companies that distributed the films
- **Director:** Directors of the films
- **Writer:** Writers of the films
- **Actor1:** First actor who acted in the films
- **Actor2:** Second actor who acted in the films
- **Actor3:** Third actor who acted in the films

5.6 Using the SELECT Statement

Now, let's go through some examples of **SELECT** queries.

5.6.1 Example 1 — Retrieve All Film Details

To retrieve details of all films from the **FilmLocations** table (including all columns):

```
SELECT * FROM FilmLocations;
```

After running this query, we can view all the records with complete details.

5.6.2 Example 2 — Retrieve Film Names, Directors, and Writers

To retrieve only the film titles, along with director and writer names:

```
SELECT Title, Director, Writer FROM FilmLocations;
```

5.6.3 Example 3 — Retrieve Films Released in 2001 and Onwards

To retrieve film names, filming locations, and release years for films released in 2001 or later:

```
SELECT Title, ReleaseYear, Locations FROM FilmLocations WHERE ReleaseYear >= 2001;
```

5.7 Practice Exercises on the SELECT Statement

5.7.1 Exercise 1 — Retrieve Fun Facts and Filming Locations

```
SELECT Locations, FunFacts FROM FilmLocations;
```

5.7.2 Exercise 2 — Retrieve Films Released Before or in 2000

Retrieve names, filming locations, and release years of all films released in or before the year 2000:

```
SELECT Title, ReleaseYear, Locations FROM FilmLocations WHERE ReleaseYear <= 2000;
```

5.7.3 Exercise 3 — Retrieve Films Not Written by James Cameron

Retrieve names, production companies, filming locations, and release years of the films not written by James Cameron:

```
SELECT Title, ProductionCompany, Locations, ReleaseYear FROM FilmLocations WHERE Writer <> "James Cameron";
```

5.8 Conclusion

We are now able to:

- Query a database using **SELECT** statements.
- Retrieve all or selected columns of data.
- Filter query responses to meet defined criteria.

6 Useful SQL Expressions in SELECT Statements

In this practice exercise, we will explore a few useful SQL expressions that can be used with **SELECT** statements. These expressions include **COUNT**, **DISTINCT**, and **LIMIT**.

6.1 COUNT Expression

The **COUNT** function is a built-in database function that retrieves the number of rows that match the query criteria.

6.1.1 Example 1 — Retrieve Total Number of Rows in a Table

To get the total number of rows in a given table:

```
SELECT COUNT(*) FROM tablename;
```

6.1.2 Example 2 — Retrieve the Number of Rows Matching a Condition

Suppose we have a table named **MEDALS** that includes a column **COUNTRY**. To retrieve the number of rows where the medal recipient is from Canada:

```
SELECT COUNT(COUNTRY) FROM MEDALS WHERE COUNTRY = 'CANADA';
```

6.2 DISTINCT Expression

The **DISTINCT** keyword is used to remove duplicate values from a result set.

6.2.1 Example 1 — Retrieve Unique Values from a Column

To retrieve unique values from a specific column:

```
SELECT DISTINCT columnname FROM tablename;
```

6.2.2 Example 2 — Retrieve Unique Countries that Received Gold Medals

Using the **MEDALS** table, we can get the list of unique countries that received gold medals:

```
SELECT DISTINCT COUNTRY FROM MEDALS WHERE MEDALTYPE = 'GOLD';
```

This removes all duplicate entries for countries that have won gold multiple times.

6.3 LIMIT Expression

The **LIMIT** expression is used to restrict the number of rows retrieved from the database.

6.3.1 Example 1 — Retrieve the First 10 Rows of a Table

To retrieve just the first 10 rows in a table:

```
SELECT * FROM tablename LIMIT 10;
```

6.3.2 Example 2 — Retrieve a Few Rows from a Specific Year

To retrieve a few rows from the **MEDALS** table for a specific year, such as 2018:

```
SELECT * FROM MEDALS WHERE YEAR = 2018 LIMIT 5;
```

This is useful for examining a small portion of a large dataset instead of retrieving the entire result set.

6.4 Summary

In this exercise, we explored three useful SQL expressions that enhance the functionality of **SELECT** statements:

- **COUNT** — retrieves the number of rows matching a condition.
- **DISTINCT** — removes duplicate values from query results.
- **LIMIT** — restricts the number of rows retrieved.

These expressions help make data queries more efficient and insightful.

7 Hands-on Practice: COUNT, DISTINCT, LIMIT

In this practice exercise, we will learn how to use three useful SQL expressions — **COUNT**, **DISTINCT**, and **LIMIT** — that are often applied with **SELECT** statements.

7.1 Objectives

After completing this exercise, we will be able to:

- Retrieve the number of rows that match a query condition using **COUNT**.
- Remove duplicate values from a result set using **DISTINCT**.
- Restrict the number of rows retrieved from a table using **LIMIT**.

7.2 Exploring the Database

Let's begin by exploring the **SanFranciscoFilmLocations** database using the **Datasette** tool.

To display all records from the **FilmLocations** table, we can execute the following query:

```
SELECT * FROM FilmLocations;
```

Once we run the query, we can scroll through the table to explore all columns and rows to understand its structure.

7.2.1 FilmLocations Table Columns

- **Title** — Titles of the films.
- **ReleaseYear** — Year of public release.
- **Locations** — Filming locations in San Francisco.
- **FunFacts** — Interesting facts about the filming locations.
- **ProductionCompany** — Companies that produced the films.
- **Distributor** — Companies that distributed the films.
- **Director** — Directors of the films.
- **Writer** — Writers of the films.
- **Actor1, Actor2, Actor3** — Main actors of the films.

7.3 Using the COUNT Statement

The **COUNT** function is used to determine the number of rows that satisfy a given condition.

7.3.1 Example 1 — Count All Records in a Table

```
SELECT COUNT(*) FROM FilmLocations;
```

7.3.2 Example 2 — Count Locations for a Specific Writer

```
SELECT COUNT(Locations) FROM FilmLocations WHERE Writer = "James Cameron";
```

7.4 Using the DISTINCT Statement

The **DISTINCT** keyword is used to return only unique values from a column, eliminating duplicates.

7.4.1 Example 1 — Retrieve All Unique Film Titles

```
SELECT DISTINCT Title FROM FilmLocations;
```

7.4.2 Example 2 — Retrieve Distinct Release Years for a Production Company

```
SELECT COUNT(DISTINCT ReleaseYear) FROM FilmLocations WHERE ProductionCompany = "Warner Bros. Pictures";
```

7.5 Using the LIMIT Statement

The **LIMIT** keyword restricts the number of rows returned by a query.

7.5.1 Example 1 — Retrieve the First 25 Rows from the Table

```
SELECT * FROM FilmLocations LIMIT 25;
```

7.5.2 Example 2 — Retrieve 15 Rows Starting from the 11th Row

```
SELECT * FROM FilmLocations LIMIT 15 OFFSET 10;
```

7.6 Practice Exercises

7.6.1 COUNT

1. Retrieve the number of film locations directed by Woody Allen.


```
SELECT COUNT(Locations) FROM FilmLocations WHERE Director = "Woody Allen";
```

2. Retrieve the number of films shot at Russian Hill.

```
SELECT COUNT(Title) FROM FilmLocations WHERE Locations = "Russian Hill";
```

3. Retrieve the number of rows with a release year before 1950.

```
SELECT COUNT(*) FROM FilmLocations WHERE ReleaseYear < 1950;
```

7.6.2 DISTINCT

1. Retrieve unique films released in or after 2001 along with their release years.

```
SELECT DISTINCT Title, ReleaseYear FROM FilmLocations WHERE ReleaseYear >= 2001;
```

2. Retrieve distinct films and their directors shot at City Hall.

```
SELECT DISTINCT Title, Director FROM FilmLocations WHERE Locations = "City Hall";
```

3. Retrieve the number of unique distributors for films featuring Clint Eastwood as the first actor.

```
SELECT COUNT(DISTINCT Distributor) FROM FilmLocations WHERE Actor1 = "Clint Eastwood";
```

7.6.3 LIMIT

1. Retrieve the names of the first 50 films.

```
SELECT DISTINCT Title FROM FilmLocations LIMIT 50;
```

2. Retrieve the first 10 film names released in 2015.

```
SELECT DISTINCT Title FROM FilmLocations WHERE ReleaseYear = 2015 LIMIT 10;
```

3. Retrieve the next 3 film names after the first 5 released in 2015.

```
SELECT DISTINCT Title FROM FilmLocations WHERE ReleaseYear = 2015 LIMIT 3 OFFSET 5;
```

7.7 Conclusion

In this exercise, we learned how to use **COUNT**, **DISTINCT**, and **LIMIT** statements in SQL to refine our data retrieval process. We can now:

- Use **COUNT** to determine the number of entries that meet specific conditions.
- Use **DISTINCT** to find unique entries in a dataset.
- Use **LIMIT** to restrict the number of rows returned.
- Combine these statements to perform more complex and efficient database queries.

8 Introduction to the INSERT Statement

In this practice exercise, we will learn how to populate a relational database table using the **INSERT** statement.

8.1 Objective

At the end of this exercise, we will be able to:

- Identify the syntax of the **INSERT** statement.
- Explain two methods to add rows to a table.

8.2 Understanding the INSERT Statement

After creating a table, it needs to be populated with data. To insert data into a table, we use the **INSERT** statement. The **INSERT** statement is used to add new rows to a table.

The **INSERT** statement** is one of the Data Manipulation Language (DML)** statements. DML statements are used to read and modify data.

8.3 Syntax of the INSERT Statement

The basic syntax of the **INSERT** statement is:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

8.3.1 Explanation of Syntax

- **table_name** identifies the table where data will be inserted.
- The **column list** specifies the columns that will receive the values.
- The **VALUES** clause provides the actual data values for each column.

The number of values provided must match the number of columns listed to ensure that each column receives a value.

8.4 Example — Inserting a Single Row

Suppose we have an **Author** table with six columns:

- Author_ID
- LastName
- FirstName
- Email
- City
- Country

We can insert data for one author (Raul Chong) using the following SQL statement:

```
INSERT INTO Author (Author_ID, LastName, FirstName, Email, City, Country)
VALUES ('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA');
```

This statement adds one row to the **Author** table.

8.5 Example — Inserting Multiple Rows

Tables do not need to be populated one row at a time. We can insert multiple rows at once by specifying each set of values in the **VALUES** clause, separated by commas.

For example:

```
INSERT INTO Author (Author_ID, LastName, FirstName, Email, City, Country)
VALUES
('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA'),
('A2', 'Ahuja', 'Rav', 'RA@IBM.com', 'Toronto', 'CA');
```

This statement inserts two rows — one for **Raul Chong** and one for **Rav Ahuja**.

8.6 Summary

- The **INSERT** statement adds new rows to a table.
- It is part of the **Data Manipulation Language (DML)** category.
- The number of columns and values must be equal.
- Data can be inserted **one row at a time** or **multiple rows simultaneously**.

9 Introduction to the UPDATE and DELETE Statements

In this practice exercise, we will learn how to alter and delete data in a relational database table using the **UPDATE** and **DELETE** statements.

9.1 Objective

At the end of this exercise, we will be able to:

- Identify the syntax of the **UPDATE** statement and the **DELETE** statement.
- Explain the importance of the **WHERE** clause in both statements.

9.2 Understanding the UPDATE Statement

After a table is created and populated with data, we can alter the data in the table using the **UPDATE** statement.

The **UPDATE** statement is one of the **Data Manipulation Language (DML)** statements. DML statements are used to read and modify data in a database.

9.2.1 Syntax of the UPDATE Statement

The basic syntax of the **UPDATE** statement is:

```
UPDATE table_name  
SET column_name = value  
WHERE condition;
```

9.2.2 Explanation of Syntax

- **table_name** identifies the table containing the data to be updated.
- **column_name** identifies the column whose value will be changed.
- The **WHERE** clause specifies the condition that determines which rows will be updated.

9.2.3 Example — Updating Data

Consider the **Author** table we created earlier. Suppose we want to update the **FIRSTNAME** and **LASTNAME** of the author with **AUTHOR_ID** = 'A2' from *Rav Ahuja* to *Lakshmi Katta*.

Before the update, we can view all rows in the **Author** table using:

```
SELECT * FROM Author;
```

To update the author's name, we use the following statement:

```
UPDATE Author  
SET LastName = 'Katta', FirstName = 'Lakshmi'  
WHERE Author_ID = 'A2';
```

After executing the update, we can confirm the change by running:

```
SELECT * FROM Author;
```

Important: If the **WHERE** clause is omitted, *all rows in the table will be updated*. For example, if no condition is specified, all authors would have their names changed to *Lakshmi Katta*.

9.3 Understanding the DELETE Statement

At times, we may need to remove one or more rows from a table. This can be done using the **DELETE** statement.

Like the **UPDATE** statement, the **DELETE** statement is also a **DML** statement used to modify data.

9.3.1 Syntax of the DELETE Statement

The basic syntax of the **DELETE** statement is:

```
DELETE FROM table_name
```

WHERE condition;

9.3.2 Explanation of Syntax

- **table_name** identifies the table from which data will be deleted.
- The **WHERE** clause specifies the condition that determines which rows will be removed.

9.3.3 Example — Deleting Data

Continuing with the **Author** table example, suppose we want to delete the rows for authors with **AUTHOR_ID** values of 'A2' and 'A3'. We can use the following statement:

```
DELETE FROM Author
WHERE Author_ID IN ('A2', 'A3');
```

Important: If the **WHERE** clause is omitted, *all rows in the table will be deleted*.

9.4 Summary

- The **UPDATE** statement is used to alter existing data in a table.
- The **DELETE** statement is used to remove rows from a table.
- Both statements rely heavily on the **WHERE** clause to ensure that only the intended rows are affected.
- Omitting the **WHERE** clause will modify or delete *all rows* in the table.

10 Hands-on Practice: INSERT, UPDATE, DELETE

In this practice exercise, we will learn some commonly used **Data Manipulation Language (DML)** statements of SQL other than **SELECT**. We will explore how to insert new rows, update existing data, and remove rows from a table.

10.1 Objectives

At the end of this exercise, we will be able to:

- Insert new rows into a table.
- Update data in existing rows.
- Remove rows from a table.

10.2 Understanding DML Statements

10.2.1 INSERT Statement

The **INSERT** statement is used to add new rows to a table.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

10.2.2 UPDATE Statement

The **UPDATE** statement modifies existing data in one or more rows of a table.

Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

10.2.3 DELETE Statement

The **DELETE** statement removes one or more rows from a table.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

10.3 Lab Environment

10.3.1 Database Used

We will work with the **Instructor** database. The table has the following columns:

- **ins_id**: Unique identification number of instructors.
- **lastname**: Last name of instructors.
- **firstname**: First name of instructors.
- **city**: City where instructors are located.
- **country**: Two-letter country code.

To explore the table, execute the following command:

```
SELECT * FROM Instructor;
```

10.4 Exercise 1: INSERT

In this exercise, we will practice inserting new rows into the Instructor table.

10.4.1 Example: Inserting a Single Row

Insert a new instructor record with id 4 for *Sandip Saha* who lives in *Edmonton, CA*.

```
INSERT INTO Instructor(ins_id, lastname, firstname, city, country)  
VALUES(4, 'Saha', 'Sandip', 'Edmonton', 'CA');
```

Then verify:

```
SELECT * FROM Instructor;
```

10.4.2 Example: Inserting Multiple Rows

Insert two new instructor records — *John Doe* (Sydney, AU) and *Jane Doe* (Dhaka, BD).

```
INSERT INTO Instructor(ins_id, lastname, firstname, city, country)
VALUES(5, 'Doe', 'John', 'Sydney', 'AU'), (6, 'Doe', 'Jane', 'Dhaka', 'BD');
```

Then verify:

```
SELECT * FROM Instructor;
```

10.4.3 Practice Task

Insert a new record with id 7 for *Antonio Cangiano* who lives in *Vancouver, CA*.

```
INSERT INTO Instructor(ins_id, lastname, firstname, city, country)
VALUES(7, 'Cangiano', 'Antonio', 'Vancouver', 'CA');
```

Insert two more records — *Steve Ryan* (Barlby, GB) and *Ramesh Sannareddy* (Hyderabad, IN).

```
INSERT INTO Instructor(ins_id, lastname, firstname, city, country)
VALUES(8, 'Ryan', 'Steve', 'Barlby', 'GB'), (9, 'Sannareddy', 'Ramesh', 'Hyderabad', 'IN');
```

10.5 Exercise 2: UPDATE

In this exercise, we will update data in the Instructor table.

10.5.1 Example: Updating a Single Column

Update the city for *Sandip* to *Toronto*.

```
UPDATE Instructor
SET city='Toronto'
WHERE firstname='Sandip';
```

Then verify:

```
SELECT * FROM Instructor;
```

10.5.2 Example: Updating Multiple Columns

Update the city and country for *Doe* with id 5 to *Dubai* and *AE*.

```
UPDATE Instructor
SET city='Dubai', country='AE'
WHERE ins_id=5;
```

10.5.3 Practice Task

Update the city of the instructor with id 1 to *Markham*.

```
UPDATE Instructor
SET city='Markham'
WHERE ins_id=1;
```

Update the city and country for *Sandip* (id 4) to *Dhaka* and *BD*.

```
UPDATE Instructor
SET city='Dhaka', country='BD'
```

```
WHERE ins_id=4;
```

10.6 Exercise 3: DELETE

In this exercise, we will remove data from the Instructor table.

10.6.1 Example: Deleting a Row

Remove the instructor record of *Doe* whose id is 6.

```
DELETE FROM Instructor  
WHERE ins_id=6;
```

Then verify:

```
SELECT * FROM Instructor;
```

10.6.2 Practice Task

Remove the instructor record of *Hima*.

```
DELETE FROM Instructor  
WHERE firstname='Hima';
```

10.7 Summary

- **INSERT** adds new rows to a table.
- **UPDATE** modifies data in existing rows.
- **DELETE** removes rows from a table.
- Always use a **WHERE** clause in **UPDATE** and **DELETE** statements to avoid altering or deleting all rows.

11 Summary: Basic SQL

At this point in the course, we know:

- **Data Manipulation Language (DML)** statements are used to read and modify data.
- The **WHERE** clause uses a predicate to refine searches within SQL queries.
- **SQL** retrieves specific data from databases efficiently.
- **COUNT**, **DISTINCT**, and **LIMIT** are expressions commonly used with **SELECT** statements.
- The **SELECT** statement has many real-world applications for retrieving and analyzing data.
- **INSERT**, **UPDATE**, and **DELETE** are DML statements used to populate, modify, and remove data from tables.

Module 2

12 Database Concepts

12.1 Introduction

Welcome to **Database Concepts**. In this practice exercise, we will learn about different types of models, how we use models to map data to tables, and how to define relationships between tables.

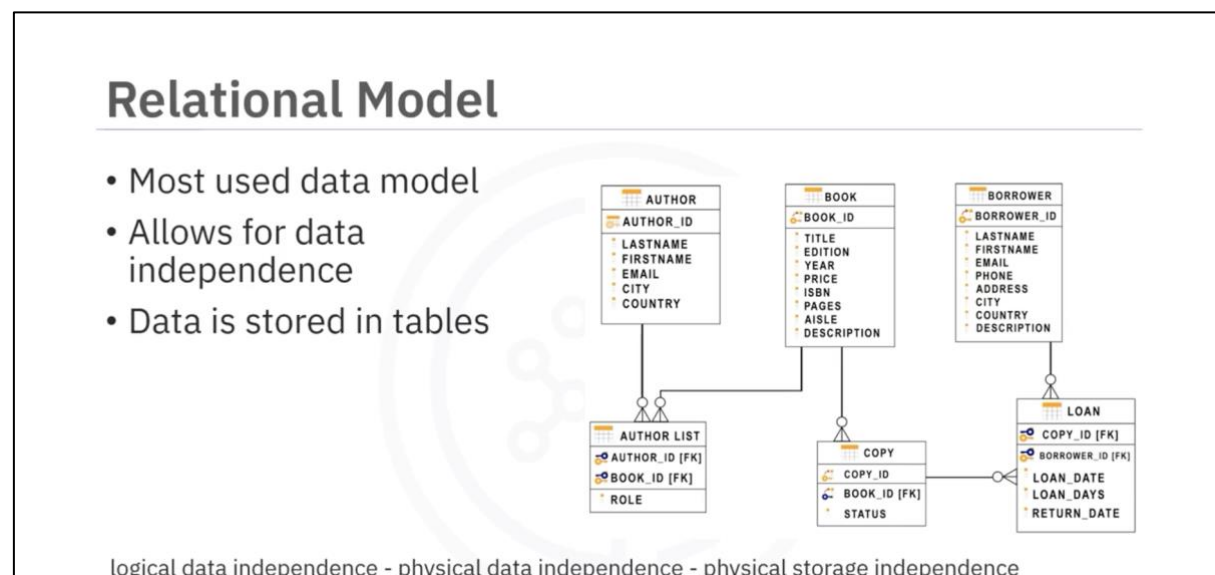
At the end of this exercise, we will be able to:

- Explain the advantages of the relational model.
- Describe how entity names and attributes map to relational database tables.
- Differentiate between an entity and an attribute.
- Identify commonly used data types.
- Describe the function of primary keys.

12.2 Relational Model Overview

The **relational model** is the most widely used data model for databases because it allows for **data independence**. Data is stored in a simple data structure known as **tables**, which provide:

- Logical data independence
- Physical data independence
- Physical storage independence



12.3 Entity Relationship (ER) Model

An **Entity Relationship (ER) data model** is an alternative to the relational model and is often used as a **design tool** for relational databases.

12.3.1 Example: Library Database

Using a simplified library database as an example, an **Entity Relationship Diagram (ERD)** represents entities (tables) and their relationships:

- A **book** can be written by one or many authors.
- The library can have one or many copies of a book.
- Each copy can be borrowed by only one borrower at a time.

12.3.2 Purpose of ER Model

The **ER model** encourages us to think of a database as a collection of entities rather than isolated data points. It focuses on designing logical relationships before implementation.

12.4 Components of an ER Diagram

The main building blocks of an **ER diagram** are **entities** and **attributes**.

12.4.1 Entities

- Represent objects or concepts that exist independently, such as a **person, place, or thing**.
- In an ER diagram, entities are represented by **rectangles**.

12.4.2 Attributes

- Describe characteristics or properties of an entity.
- In an ER diagram, attributes are represented by **ovals**.

12.4.2.1 Example

Using the library example:

- The **Book** is an entity.
- Attributes of the Book entity include:
 - Book Title
 - Edition
 - Year Published
 - ISBN

Entity-Relationship Model

- Used as a tool to design relational databases



Attributes are connected to one entity and describe its properties.

12.5 Mapping Entities to Tables

When mapping to a relational database:

- Each **entity** becomes a **table**.
- Each **attribute** becomes a **column**.

A table is composed of **rows and columns**. Initially, the entity structure defines columns, and later data values are inserted into rows to complete the table.

12.5.1 Data Types

Each attribute stores data values of specific formats, such as:

- **Character types:** CHAR, VARCHAR
- **Numeric types:** INTEGER, DECIMAL
- **Date/Time types:** DATE, TIMESTAMP

12.5.1.1 Example: Book Table

- Title → VARCHAR (variable length)
- Edition → INTEGER
- Year → INTEGER
- ISBN → CHAR (includes numbers and dashes)

12.6 Primary and Foreign Keys

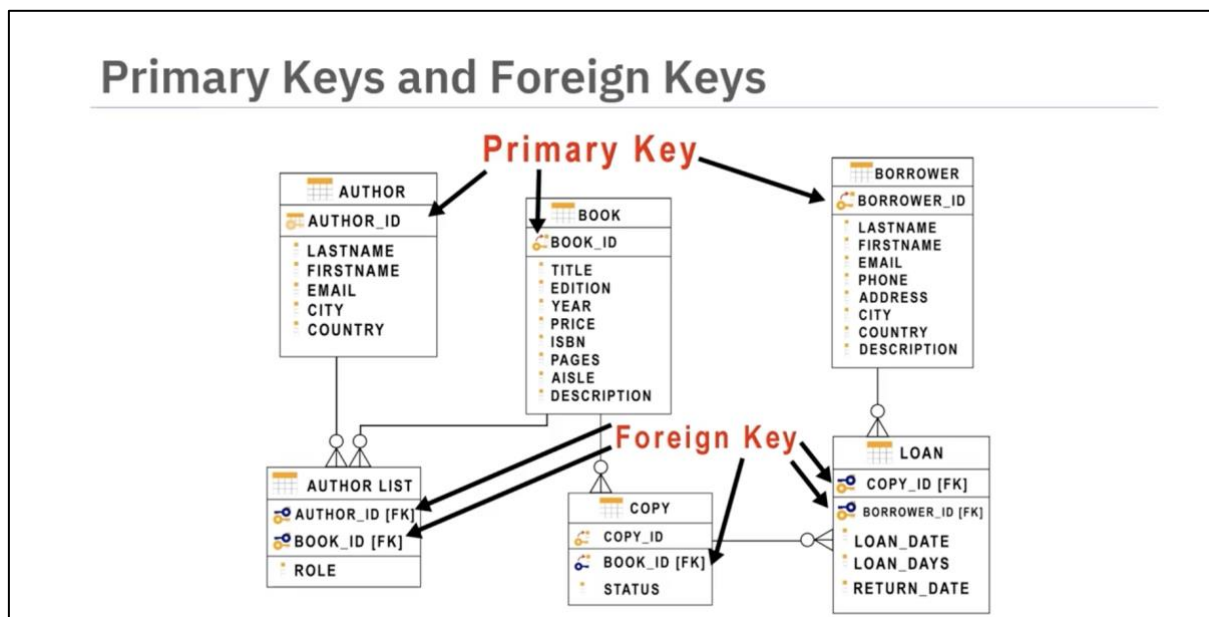
Each table in a relational database is assigned a **primary key**.

12.6.1 Primary Key

- Uniquely identifies each row (tuple) in a table.
- Prevents data duplication.
- Enables defining relationships between tables.

12.6.2 Foreign Key

- A **foreign key** is a primary key from another table.
- Creates a **relationship** between two tables.



12.7 Summary

- The **relational model** offers logical and physical data independence.
- **Entities** are independent objects with multiple **attributes**.
- When mapping to a database, entities become **tables** and attributes become **columns**.
- Common data types include CHAR, VARCHAR, INTEGER, and DATE.
- A **primary key** uniquely identifies rows and prevents duplication, while **foreign keys** link related tables.

13 Types of SQL Statements

13.1 Introduction

Welcome to **Types of SQL Statements**. In this practice exercise, we will learn to distinguish between **Data Definition Language (DDL)** statements and **Data Manipulation Language (DML)** statements.

At the end of this exercise, we will be able to:

- Differentiate between DDL and DML statements.

- Identify the purpose of each type of SQL statement.

13.2 Overview of SQL Statements

SQL statements are used to interact with database components such as **entities (tables)**, **attributes (columns)**, and their **tuples (rows with data values)** in relational databases.

SQL statements are classified into two main categories:

- **Data Definition Language (DDL)**
- **Data Manipulation Language (DML)**

13.3 Data Definition Language (DDL)

DDL statements are used to define, modify, or remove database objects such as tables.

13.3.1 Common DDL Statements

- **CREATE** – Used to create new tables and define their columns.
- **ALTER** – Used to modify existing tables, including adding or removing columns and changing data types.
- **TRUNCATE** – Deletes all data from a table without removing the table itself.
- **DROP** – Deletes the entire table from the database.

13.3.2 Purpose of DDL

DDL statements deal with the **structure** of the database, helping us define and maintain tables and their schemas.

13.4 Data Manipulation Language (DML)

DML statements are used to read and modify data stored in tables. These are commonly referred to as **CRUD operations** — **Create, Read, Update, and Delete**.

13.4.1 Common DML Statements

- **INSERT** – Adds new rows of data to a table.
- **SELECT** – Retrieves rows from one or more tables.
- **UPDATE** – Edits existing data in a table.
- **DELETE** – Removes one or more rows from a table.

13.4.2 Purpose of DML

DML statements focus on **data manipulation**, allowing us to work directly with the records inside database tables.

13.5 Summary

- **DDL statements** define or change database structures such as tables.
- **DML statements** manipulate and manage the actual data stored within those tables.
- Together, DDL and DML form the core of how SQL interacts with relational databases.

14 CREATE TABLE Statement

14.1 Introduction

Welcome to **CREATE TABLE Statement**. In this practice exercise, we will learn how the **entity name** and **attributes** are used to create a relational database table.

At the end of this exercise, we will be able to:

- Explain how entity names and attributes are used to define tables.
- Understand the syntax and components of the **CREATE TABLE** statement.

14.2 Understanding the CREATE TABLE Statement

The **CREATE TABLE** statement is one of the most common **Data Definition Language (DDL)** statements in SQL. It allows us to create new tables in a relational database.

14.2.1 Syntax

The basic syntax for creating a table is as follows:

```
CREATE TABLE table_name (  
    column_name data_type [optional constraints],  
    column_name data_type [optional constraints],  
    ...  
);
```

- The statement begins with **CREATE TABLE** followed by the **name of the table** we want to create.
- The list of columns is enclosed in parentheses ().
- Each column definition includes the **column name**, its **data type**, and any optional **constraints** (such as PRIMARY KEY, NOT NULL, etc.).
- Each column definition is separated by a comma.

14.3 Example 1: Creating a Simple Table

Let's look at an example of creating a table for provinces in Canada:

```
CREATE TABLE provinces (  
    id CHAR(2) PRIMARY KEY NOT NULL,  
    name VARCHAR(24)  
);
```

14.3.1 Explanation

- **CHAR(2)** defines a fixed-length character field with 2 characters (e.g., AB, BC).
- **VARCHAR(24)** defines a variable-length character field with up to 24 characters (e.g., ALBERTA, BRITISH COLUMBIA).

Issuing this statement creates a table named **provinces** with two columns:

- **id** for storing 2-letter province codes.
- **name** for storing the full province name.

14.4 Example 2: Creating the AUTHOR Table

Let's look at a more detailed example from a **Library database** that includes several entities such as **AUTHOR**, **BOOK**, and **BORROWER**.

To create the **AUTHOR** table, we can use the following command:

```
CREATE TABLE author (  
  author_id CHAR(2) PRIMARY KEY NOT NULL,  
  lastname VARCHAR(15) NOT NULL,  
  firstname VARCHAR(15) NOT NULL,  
  email VARCHAR(40),  
  city VARCHAR(15),  
  country CHAR(2)  
);
```

14.4.1 Explanation

- The **AUTHOR_ID** column is the **Primary Key**, ensuring each author has a unique identifier.
- **NOT NULL** constraints on **lastname** and **firstname** ensure that every author must have a name.
- Optional fields like **email**, **city**, and **country** can accept NULL values.

14.5 Key Takeaways

- **CREATE** is a **DDL** statement used to define and create database tables.
- The **CREATE TABLE** statement includes definitions of attributes such as:
 - **Column names**
 - **Data types**
 - **Optional constraints** (e.g., PRIMARY KEY, NOT NULL)
- Primary Keys uniquely identify each record in a table and prevent duplicate values.
- Constraints like **NOT NULL** enforce data integrity by preventing missing values.

15 ALTER, DROP, and TRUNCATE Tables

15.1 Introduction

Welcome to **ALTER, DROP, and TRUNCATE Tables**. In this practice exercise, we will learn about the **ALTER TABLE**, **DROP TABLE**, and **TRUNCATE TABLE** statements.

At the end of this exercise, we will be able to:

- Describe the **ALTER TABLE**, **DROP TABLE**, and **TRUNCATE TABLE** statements.
- Explain their syntax.
- Use these statements in SQL queries.

15.2 ALTER TABLE Statement

The **ALTER TABLE** statement allows us to modify the structure of an existing table. It can be used to:

- Add or remove columns.
- Modify the data type of existing columns.
- Add or remove keys and constraints.

15.2.1 Syntax

```
ALTER TABLE table_name  
  ADD column_name data_type;  
ALTER TABLE table_name  
  MODIFY column_name new_data_type;  
ALTER TABLE table_name  
  DROP COLUMN column_name;
```

15.2.2 Example 1: Adding a Column

To add a new column `telephone_number` to the **author** table:

```
ALTER TABLE author ADD COLUMN telephone_number BIGINT;
```

Here, `BIGINT` can store a number up to 19 digits long.

15.2.3 Example 2: Modifying a Column

To change the data type of the `telephone_number` column from **BIGINT** to **CHAR(20)**:

```
ALTER TABLE author MODIFY telephone_number CHAR(20);
```

Note: Changing a column's data type can cause issues if the existing data is not compatible with the new type.

15.2.4 Example 3: Dropping a Column

If the `telephone_number` column is no longer required, we can remove it using:

```
ALTER TABLE author DROP COLUMN telephone_number;
```

15.3 DROP TABLE Statement

The **DROP TABLE** statement is used to delete an existing table from a database. When executed, both the table and its data are removed permanently.

15.3.1 Syntax

DROP TABLE table_name;

15.3.2 Example

To delete the **author** table:

```
DROP TABLE author;
```

Caution: Dropping a table permanently deletes all data and structure related to it.

15.4 TRUNCATE TABLE Statement

The **TRUNCATE TABLE** statement removes all rows from a table without deleting the table structure itself. It is faster and more efficient than the **DELETE** statement without a **WHERE** clause.

15.4.1 Syntax

```
TRUNCATE TABLE table_name IMMEDIATE;
```

The **IMMEDIATE** keyword specifies that the operation should be processed right away and **cannot be undone**.

15.4.2 Example

To remove all rows from the **author** table:

```
TRUNCATE TABLE author IMMEDIATE;
```

15.5 Key Takeaways

- The **ALTER TABLE** statement modifies an existing table's structure (add, modify, or drop columns).
- The **DROP TABLE** statement permanently deletes a table and its data.
- The **TRUNCATE TABLE** statement removes all rows from a table while retaining its structure.

16 ALTER and TRUNCATE Tables Using MySQL

16.1 Introduction

In this practice exercise, we will explore examples of **ALTER** and **TRUNCATE** statements using **MySQL**. These statements allow us to modify the structure of tables and manage data efficiently.

At the end of this exercise, we will be able to:

- Use the **ALTER TABLE** statement with correct syntax.
- Use the **TRUNCATE TABLE** statement.
- Execute examples of **ALTER** and **TRUNCATE** statements in **MySQL**.

16.2 ALTER TABLE Statement

The **ALTER TABLE** statement is used to modify the structure of an existing table. It can be used to:

- Add or remove columns.
- Modify the data type of existing columns.
- Add or remove keys and constraints.

16.2.1 ADD COLUMN Syntax

```
ALTER TABLE table_name  
ADD column_name data_type;
```

A variation of the syntax is:

```
ALTER TABLE table_name  
ADD COLUMN column_name data_type;
```

By default, all the entries in the new column are initially assigned the value **NULL**. You can later use **UPDATE** statements to populate the column.

16.2.1.1 Example: Adding a Column

To add a new column `telephone_number` to the **author** table:

```
ALTER TABLE author  
ADD telephone_number BIGINT;
```

Here, **BIGINT** is a data type used for storing large integer values.

16.2.2 MODIFY COLUMN Syntax

```
ALTER TABLE table_name  
MODIFY column_name data_type;
```

Sometimes, the data may need to be changed to a different format. For example, using a numeric type for `telephone_number` does not allow parentheses, plus signs, or dashes. In such cases, **CHAR** is a more suitable data type.

16.2.2.1 Example: Modifying a Column Data Type

To modify the data type of the `telephone_number` column to **CHAR(20)**:

```
ALTER TABLE author  
MODIFY telephone_number CHAR(20);
```

After modification, you can use **UPDATE** statements to adjust the entries.

16.3 TRUNCATE TABLE Statement

The **TRUNCATE TABLE** statement is used to delete all rows from a table while preserving its structure. It is faster and more efficient than **DELETE** without a **WHERE** clause.

16.3.1 Syntax

```
TRUNCATE TABLE table_name;
```

16.3.1.1 Example: Truncating a Table

To delete all rows from the **author** table:

```
TRUNCATE TABLE author;
```

Note: The **TRUNCATE** statement removes all rows but does not delete the table structure itself.

17 CREATE, ALTER, TRUNCATE, and DROP in MySQL

17.1 Introduction

In this practice exercise, we will learn how to create tables and load data using the **phpMyAdmin** graphical user interface (GUI) tool in the **MySQL** database service.

We will also explore how to use SQL statements such as **CREATE**, **ALTER**, **TRUNCATE**, and **DROP** to manage databases and tables.

17.2 Software Used

We will use **MySQL**, a **Relational Database Management System (RDBMS)** designed to efficiently store, manipulate, and retrieve data.

17.3 Objectives

By the end of this exercise, we will be able to:

- Create a database.
- Create new tables in a database.
- Add, delete, or modify columns in an existing table.
- Remove all rows from a table without deleting it.
- Delete an existing table from a database.

17.4 Task 1: Create a Database

To create a new database using the **phpMyAdmin** GUI of **MySQL**:

1. Open **Skills Network Toolbox** and click **MySQL** under the Database section.
2. Click **Create** to start **MySQL**.
3. Click the **phpMyAdmin** button to open it in the same or a new browser tab.

4. In the phpMyAdmin tree view, click **New** to create a new database.
5. Enter **Mysql_Learners** as the database name, keep the default **utf8** encoding, and click **Create**.

Note: UTF-8 is the most commonly used character encoding for data.

17.5 Task 2a: CREATE Statement

We will now use the **CREATE** statement to create two tables — **PETSALE** and **PET**.

```
CREATE TABLE PETSALE (  
    ID INTEGER NOT NULL,  
    PET CHAR(20),  
    SALEPRICE DECIMAL(6,2),  
    PROFIT DECIMAL(6,2),  
    SALEDATE DATE  
);
```

```
CREATE TABLE PET (  
    ID INTEGER NOT NULL,  
    ANIMAL VARCHAR(20),  
    QUANTITY INTEGER  
);
```

17.6 Task 2b: INSERT Statement

Insert records into the newly created tables:

```
INSERT INTO PETSALE VALUES  
(1,'Cat',450.09,100.47,'2018-05-29'),  
(2,'Dog',666.66,150.76,'2018-06-01'),  
(3,'Parrot',50.00,8.9,'2018-06-04'),  
(4,'Hamster',60.60,12,'2018-06-11'),  
(5,'Goldfish',48.48,3.5,'2018-06-14');
```

```
INSERT INTO PET VALUES  
(1,'Cat',3),  
(2,'Dog',4),  
(3,'Hamster',2);
```

```
SELECT * FROM PETSALE;  
SELECT * FROM PET;
```

17.7 Task 3: ALTER Statement

The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table.

17.7.1 1. Adding a Column

```
ALTER TABLE PETSALE ADD COLUMN QUANTITY INTEGER;  
SELECT * FROM PETSALE;
```

Then update the new column with values:

```
UPDATE PETSALE SET QUANTITY = 9 WHERE ID = 1;  
UPDATE PETSALE SET QUANTITY = 3 WHERE ID = 2;  
UPDATE PETSALE SET QUANTITY = 2 WHERE ID = 3;
```

```
UPDATE PETSale SET QUANTITY = 6 WHERE ID = 4;
UPDATE PETSale SET QUANTITY = 24 WHERE ID = 5;
SELECT * FROM PETSale;
```

17.7.2 2. Deleting a Column

```
ALTER TABLE PETSale DROP COLUMN PROFIT;
SELECT * FROM PETSale;
```

17.7.3 3. Modifying a Column

```
ALTER TABLE PETSale MODIFY PET VARCHAR(20);
SELECT * FROM PETSale;
```

17.7.4 4. Renaming a Column

```
ALTER TABLE PETSale CHANGE 'PET' 'ANIMAL' VARCHAR(20);
SELECT * FROM PETSale;
```

17.8 Task 4: TRUNCATE Statement

The **TRUNCATE** statement removes all rows from a table but keeps the table structure intact.

```
TRUNCATE TABLE PET;
SELECT * FROM PET;
```

17.9 Task 5: DROP Statement

The **DROP** statement deletes an entire table from the database.

```
DROP TABLE PET;
SELECT * FROM PET;
```

Note: The second statement will show an error because the table no longer exists.

17.10 Practice Problems

1. Create a new table named **Toys** with the following structure:

```
CREATE TABLE Toys (
  ID INTEGER NOT NULL,
  Variety VARCHAR(20),
  Quantity INTEGER
);
```

2. Insert data into the **Toys** table:

```
INSERT INTO Toys VALUES
(1, 'Chew toy', 20),
(2, 'Balls', 50),
(3, 'Bowls', 30),
(4, 'Foldable bed', 40);
```

3. Modify the length of Variety to 30 characters:

```
ALTER TABLE Toys MODIFY Variety VARCHAR(30);
```

4. Truncate the **Toys** table:

```
TRUNCATE TABLE Toys;
```

5. Drop the **Toys** table:

DROP TABLE Toys;

17.11 Conclusion

By completing this exercise, we learned how to:

- Create a database using phpMyAdmin in MySQL.
- Use the **CREATE** statement to create tables.
- Use the **INSERT** statement to add records.
- Use the **ALTER** statement to modify table structures.
- Use the **TRUNCATE** statement to clear table contents.
- Use the **DROP** statement to permanently delete tables.

18 Understanding Relational Model Constraints

18.1 Objectives

After completing this exercise, we will be able to:

- Define and identify **entity integrity**, **referential integrity**, and **domain integrity** constraints.
- Explain how each constraint maintains data integrity.
- Recognize examples of how these constraints are implemented in SQL.

18.2 Overview

In any well-designed database, maintaining **data integrity** is essential to ensure the **accuracy**, **consistency**, and **reliability** of stored information. In this exercise, we will explore three key relational model constraints:

- **Entity Integrity**
- **Referential Integrity**
- **Domain Integrity**

These constraints enforce rules on how data is stored and related within tables.

Imagine we are building a database **BookShopDB**. We must ensure that:

- Each book and author has a unique identifier.
- Every book must be linked to a valid author.
- Attributes like price, title, and date must have values within acceptable ranges and formats.

This exercise will walk us through these constraints with examples to strengthen our understanding.

18.3 Sample Tables

18.3.1 BookShop Table

This is a sample table structure and data view for the **BookShop** table used in this exercise.

18.3.2 BookShop_AuthorDetails Table

This is a sample table structure and data view for the **BookShop_AuthorDetails** table used in this exercise.

18.4 Types of Relational Model Constraints

18.4.1 Entity Integrity Constraint

This constraint ensures that every table in a relational database has a **primary key**, which uniquely identifies each row in the table. A primary key column:

- Must **not contain NULL values**.
- Must be **unique** across all rows.

This guarantees that each record (or entity) in a table is distinct and identifiable, preventing duplication and missing identifiers.

18.4.1.1 Example

```
CREATE TABLE BookShop (  
  BOOK_ID INT PRIMARY KEY,  
  TITLE VARCHAR(100),  
  AUTHOR_ID INT  
);
```

18.4.1.2 Explanation

Here, **BOOK_ID** is the primary key. It must be:

- **Unique** — no two books can have the same ID.
- **Not NULL** — every book must have an ID.

Every table in a relational database should have a primary key to satisfy entity integrity.

18.4.2 Referential Integrity Constraint

This constraint ensures that a **foreign key** in one table always refers to a valid **primary key** in another table. It maintains consistent and meaningful relationships between tables, preventing invalid or orphaned references.

18.4.2.1 Example

```
CREATE TABLE BookShop_AuthorDetails (  
  AUTHOR_ID INT PRIMARY KEY,  
  AUTHOR_NAME VARCHAR(100)  
);
```

```
CREATE TABLE BookShop (
  BOOK_ID INT PRIMARY KEY,
  TITLE VARCHAR(100),
  AUTHOR_ID INT,
  FOREIGN KEY (AUTHOR_ID) REFERENCES BookShop_AuthorDetails(AUTHOR_ID)
);
```

18.4.2.2 Explanation

- **AUTHOR_ID** in BookShop references **AUTHOR_ID** in BookShop_AuthorDetails.
- Every **AUTHOR_ID** in BookShop must exist in BookShop_AuthorDetails.
- Attempting to insert a book with a non-existent **AUTHOR_ID** will fail.

18.4.3 Domain Integrity Constraint

This constraint ensures that all values stored in a column fall within a defined domain. It includes rules about:

- Data type
- Format
- Allowed values
- Nullability

This helps ensure that the data in a column is valid, logical, and consistent with its intended use.

18.4.3.1 Example

```
CREATE TABLE BookShop (
  BOOK_ID INT PRIMARY KEY,
  TITLE VARCHAR(100) NOT NULL,
  PRICE DECIMAL(5, 2) CHECK (PRICE >= 0),
  PUBLISHED_DATE DATE
);
```

18.4.3.2 Explanation

Two standard mechanisms used to enforce domain integrity are **CHECK** and **NOT NULL** constraints:

18.4.3.2.1 CHECK Constraint

The **CHECK** constraint enforces rules about the range or pattern of acceptable values within a column. It ensures that the data entered meets logical conditions.

- In the above example, the **PRICE** column uses a **CHECK** constraint to ensure that the price is not negative.

18.4.3.2.2 NOT NULL Constraint

The **NOT NULL** constraint ensures that a column must contain a value and cannot be left empty.

- The **TITLE** column is NOT NULL, meaning every book must have a title.

18.5 Summary

In this exercise, we explored three key types of constraints used in relational databases:

- **Entity Integrity** ensures each record is unique and identifiable.
- **Referential Integrity** maintains valid relationships between tables.
- **Domain Integrity** enforces valid, logical, and consistent data values.

By applying these constraints, we can create databases that are accurate, consistent, and easy to manage.

19 SQL Scripts - Uses and Applications

19.1 SQL Scripts

SQL scripts are a series of commands or programs that are executed on an SQL server. These scripts are essential for performing complex database operations efficiently. They can be used to create, modify, or delete database objects such as **tables**, **views**, **stored procedures**, and **functions**.

19.2 Applications of SQL Scripts

SQL scripts are versatile and can perform a wide variety of database management tasks. Some of the key applications include:

19.2.1 Creating Tables

SQL scripts can be used to create new tables within a database. This is particularly useful when adding new functionality to an application or introducing new types of data to store.

19.2.2 Dropping Tables

SQL scripts often include commands to **drop tables** before creating new ones to ensure that a table with the same name does not already exist in the database.

19.2.3 Inserting Data

SQL scripts can populate tables with data. This is useful for loading test data, initializing databases, or importing data from external sources.

19.2.4 Updating Data

SQL scripts can update existing data in tables. This is beneficial for correcting errors or reflecting changes in business requirements.

19.2.5 Deleting Data

SQL scripts can remove old or obsolete records from tables, maintaining a clean and up-to-date database.

19.2.6 Creating Views

Views are virtual tables that allow you to query data from multiple tables as if it were a single table. SQL scripts can create views to simplify complex queries and improve data accessibility.

19.2.7 Creating Stored Procedures

Stored procedures are precompiled SQL statements that can be executed on demand. SQL scripts can create stored procedures to encapsulate complex business logic and improve database management.

19.2.8 Creating Triggers

Triggers are special types of stored procedures automatically executed in response to specific events (such as INSERT, UPDATE, or DELETE). SQL scripts can create triggers that enforce business rules and maintain data integrity.

19.3 Example: Creating Tables

Instead of manually typing DDL commands in the SQL editor, we can execute a script containing multiple **CREATE TABLE** statements for all required tables in a dataset.

19.3.1 Important Notes About SQL Scripts

- SQL scripts are a collection of SQL commands compiled in a single file.
- Each command should end with a delimiter, typically a **semicolon (;)**.
- It is advisable to save SQL scripts with the **.sql** extension.
- When imported into **phpMyAdmin**, the commands are executed sequentially.

19.3.2 Sample SQL Script

```
DROP TABLE IF EXISTS PATIENTS;  
DROP TABLE IF EXISTS MEDICAL_HISTORY;  
DROP TABLE IF EXISTS MEDICAL_PROCEDURES;  
DROP TABLE IF EXISTS MEDICAL_DEPARTMENTS;  
DROP TABLE IF EXISTS MEDICAL_LOCATIONS;
```

```
CREATE TABLE PATIENTS (  
  PATIENT_ID CHAR(9) NOT NULL,  
  FIRST_NAME VARCHAR(15) NOT NULL,  
  LAST_NAME VARCHAR(15) NOT NULL,  
  SSN CHAR(9),  
  BIRTH_DATE DATE,  
  SEX CHAR,  
  ADDRESS VARCHAR(30),  
  DEPT_ID CHAR(9) NOT NULL,  
  PRIMARY KEY (PATIENT_ID)
```

```
);

CREATE TABLE MEDICAL_HISTORY (
    MEDICAL_HISTORY_ID CHAR(9) NOT NULL,
    PATIENT_ID CHAR(9) NOT NULL,
    DIAGNOSIS_DATE DATE,
    DIAGNOSIS_CODE VARCHAR(10),
    MEDICAL_CONDITION VARCHAR(100),
    DEPT_ID CHAR(9),
    PRIMARY KEY (MEDICAL_HISTORY_ID)
);

CREATE TABLE MEDICAL_PROCEDURES (
    PROCEDURE_ID CHAR(9) NOT NULL,
    PROCEDURE_NAME VARCHAR(30),
    PROCEDURE_DATE DATE,
    PATIENT_ID CHAR(9) NOT NULL,
    DEPT_ID CHAR(9),
    PRIMARY KEY (PROCEDURE_ID)
);

CREATE TABLE MEDICAL_DEPARTMENTS (
    DEPT_ID CHAR(9) NOT NULL,
    DEPT_NAME VARCHAR(15),
    MANAGER_ID CHAR(9),
    LOCATION_ID CHAR(9),
    PRIMARY KEY (DEPT_ID)
);

CREATE TABLE MEDICAL_LOCATIONS (
    LOCATION_ID CHAR(9) NOT NULL,
    DEPT_ID CHAR(9) NOT NULL,
    LOCATION_NAME VARCHAR(50),
    PRIMARY KEY (LOCATION_ID, DEPT_ID)
);
```

19.3.3 Explanation

This SQL script:

1. Drops existing tables with the same names (if they exist).
2. Creates five new tables sequentially: **PATIENTS**, **MEDICAL_HISTORY**, **MEDICAL_PROCEDURES**, **MEDICAL_DEPARTMENTS**, and **MEDICAL_LOCATIONS**.

19.3.4 Execution in phpMyAdmin

- Save the script as a **.sql** file.
- Open **phpMyAdmin**, select the target database, and upload the file.
- Click **Go** to execute the script.
- Upon successful execution, phpMyAdmin displays confirmation messages, and the new tables appear in the database structure tree.

You can then click on any table (e.g., **PATIENTS**) to view its **table definition**, including column names, data types, and constraints.

20 Hands-on Practice Exercise: Create and Load Tables using SQL Scripts

20.1 Introduction

In this practice exercise, we will learn how to create tables and load data using the **phpMyAdmin** graphical user interface (GUI) tool in the **MySQL** database service.

20.2 Objectives

After completing this exercise, we will be able to use phpMyAdmin with MySQL to:

- Create a database in MySQL
- Create tables using SQL scripts
- Load data into tables directly from CSV files

20.3 MySQL Overview

MySQL is a **Relational Database Management System (RDBMS)** designed to efficiently store, manipulate, and retrieve data.

To complete this exercise, we will use the **MySQL relational database service** available as part of the **IBM Skills Network Labs (SN Labs)** Cloud IDE, the virtual lab environment used in this course.

20.4 Database Used in This Exercise

We will work with a sample **Cardio-Vascular Diseases (CVD)** database. This database schema consists of five tables:

- **PATIENTS**
- **MEDICAL_HISTORY**
- **MEDICAL_PROCEDURES**
- **MEDICAL_DEPARTMENTS**
- **MEDICAL_LOCATIONS**

Each table includes a few rows of sample data. The goal is to create this database in MySQL by completing the following three tasks:

20.4.1 Tasks Overview

1. **Task 1:** Create the database using phpMyAdmin GUI.
2. **Task 2:** Create all tables using an SQL script.
3. **Task 3:** Populate each table with data from CSV files.

20.5 Task 1: Create the Database

Follow the steps below to create the database **CVD** in MySQL:

1. Launch **phpMyAdmin**.
2. Click on **Skills Network Toolbox**. In the Database section, select **MySQL**.
3. To start MySQL, click **Create**.
4. Once MySQL has started, click the **phpMyAdmin** button to open it in the same or a new browser tab.
5. In phpMyAdmin, click **New** in the tree view to create a new empty database.
6. Enter **CVD** as the database name and click **Create**.
7. Leave the default **utf8 encoding**, as it is the most commonly used character encoding.

20.6 Task 2: Create Tables Using SQL Script

In this task, we will execute a script containing **CREATE TABLE** commands for all the tables instead of manually typing each DDL command.

20.6.1 Key Points About SQL Scripts

- SQL scripts are sets of SQL commands compiled in a single file.
- Each command must end with a semicolon (;).
- The file extension should be **.sql**.
- When imported into phpMyAdmin, the commands run sequentially.

20.6.2 Steps to Execute the Script

1. Download the file **CVD_Database_Create_Tables_Script.sql**.
2. In phpMyAdmin, select the **CVD** database.
3. Click the **Import** tab.
4. Click **Choose File** and upload the script file.
5. Scroll down and click **Go**.
6. The script will execute successfully, creating all five tables.

You can then click any table (e.g., **PATIENTS**) to view its structure, including columns, data types, and constraints.

20.7 Task 3: Load Data into Tables

After creating the tables, we will load data from **CSV files** directly into them.

20.7.1 Steps to Load Data

1. Download the following five CSV files:
 - Patients.csv
 - MedicalHistory.csv
 - MedicalProcedures.csv
 - MedicalDepartments.csv

- MedicalLocations.csv
- 2. In phpMyAdmin, select the desired table (e.g., **PATIENTS**).
- 3. Click the **Import** tab.
- 4. Choose the corresponding CSV file and click **Go**.
- 5. A success message will confirm that the records have been inserted successfully.
- 6. Click **Browse** to view the loaded data in the table.

20.7.2 Practice Exercise

Repeat the same process for all other tables to fully populate the CVD database.

20.8 Conclusion

In this exercise, we learned how to:

- Use phpMyAdmin GUI to operate on MySQL servers.
- Create a new database in phpMyAdmin.
- Create tables using SQL scripts.
- Load data from CSV files directly into MySQL tables.

By completing these steps, we now understand how to efficiently create and manage databases using SQL scripts and graphical interfaces like phpMyAdmin.

21 Summary: Relational Database Concepts and Tables

At this point in the course, we know:

- A **database** is a repository of data that provides functionality for adding, modifying, and querying the data.
- **SQL** is a language used to query or retrieve data from a relational database.
- The **Relational Model** is the most used data model for databases because it allows for data independence.
- The **primary key** of a relational table uniquely identifies each tuple or row, preventing duplication of data and providing a way of defining relationships between tables.
- **SQL statements** fall into two different categories:
 - **Data Definition Language (DDL)** statements
 - **Data Manipulation Language (DML)** statements

22 Introduction to Cloud Databases

22.1 Understanding Cloud Databases

A **Cloud database** is a database service built and accessed through a Cloud platform. It serves the same functions as traditional databases but with the added flexibility and scalability of Cloud computing. Cloud databases enable users to create, manage, and query databases without managing physical infrastructure.

22.1.1 Advantages of Using Cloud Databases

Cloud databases provide several benefits that make them popular among organizations:

- **Ease of Use:** Users can access Cloud databases from virtually anywhere using a vendor's API, web interface, or their own applications.
- **Scalability:** Cloud databases can automatically expand or shrink their storage and computing capacity during runtime to meet changing demands. Organizations only pay for the resources they use.
- **Disaster Recovery:** Cloud databases maintain backups across geographically distributed regions, ensuring data safety during equipment failures, power outages, or natural disasters.

22.1.2 Examples of Cloud Databases

Several relational databases are available on Cloud platforms, including:

- IBM Db2 on Cloud
- Databases for PostgreSQL on IBM Cloud
- Oracle Database Cloud Service
- Microsoft Azure SQL Database
- Amazon Relational Database Service (RDS)

These databases can run either as a **Virtual Machine (VM)** that users manage or as a **Managed Service** offered by the vendor.

22.2 Cloud Database Service Instances

To use a database on the Cloud, users must **provision an instance** of a Database-as-a-Service (DBaaS). This instance provides access to database resources without requiring hardware setup, software installation, or database administration.

22.2.1 Characteristics of a Database Service Instance

- The instance holds data in related tables.
- Once data is loaded, users can connect to the database via a web interface or APIs in their applications.
- Applications send **SQL queries** to the instance, which processes the statements and returns results.

22.3 Creating an IBM Db2 on Cloud Service Instance

IBM Db2 on Cloud is a fully managed SQL database that allows users to work with databases in the Cloud without worrying about underlying hardware or software maintenance.

22.3.1 Steps to Create a Db2 Service Instance

1. **Navigate to IBM Cloud Catalog** and select the **Db2 service**.

2. Choose the **Lite plan** (a free option).
3. Modify settings if needed (service instance name, region, organization, and space).
4. Click **Create** to set up the instance.

22.3.2 Managing the Service Instance

- Once created, the Db2 service instance appears under **Services** in the IBM Cloud dashboard.
- From the dashboard, click **Open Console** to launch the web console, where you can:
 - Create tables
 - Load and explore data
 - Execute SQL queries

22.4 Service Credentials

To connect your applications to the database instance, you need **service credentials**.

22.4.1 Creating and Using Service Credentials

- Create new credentials for the first time or multiple sets for different users/applications.
- Each credential set contains:
 - Database name and port number
 - Host name (server on which the database resides)
 - Username and password

Note: The username also serves as the **schema name**, meaning that tables you create will be stored under this schema by default.

22.5 Summary

We have learned how Cloud databases work, their advantages, and how to create and manage a **Db2 on Cloud** service instance. With this understanding, we are ready to create our own Cloud database and begin executing SQL queries.

Note: There are three labs given in the “Using_Cloud_Database_as_IBM_DB2” folder, if you want to use or practice the IBM DB2 cloud database.

Module 3

23 Retrieving Data with SELECT Statement String Patterns

23.1 Introduction

In this practice exercise, we will explore advanced techniques for retrieving data from relational database tables using the **SELECT** statement. We will learn how to simplify SELECT statements by applying **string patterns**, **ranges**, and **sets of values**.

23.2 Purpose of SELECT Statement

The main purpose of a Database Management System (DBMS) is not only to store data but also to facilitate its retrieval. The simplest form of a SELECT statement is:

```
SELECT * FROM table_name;
```

For example, based on a library database model with a table named **Book**, the query:

```
SELECT * FROM Book;
```

returns all rows and columns from the table. You can also retrieve a subset of columns such as:

```
SELECT Book_ID, Title FROM Book;
```

Additionally, you can restrict the result set using the **WHERE** clause. For example:

```
SELECT Title FROM Book WHERE Book_ID = 'B1';
```

23.3 Using String Patterns in the WHERE Clause

Sometimes, we may not know the exact value to specify in the WHERE clause. For instance, if we can't remember the author's name but recall that it starts with **R**, we can use **string patterns** to search for matching rows.

23.3.1 The LIKE Predicate

The **LIKE** predicate allows searching for a pattern within a column. It uses the **percent sign (%)** as a wildcard to represent missing characters.

Examples:

- % before a pattern matches any characters before it.
- % after a pattern matches any characters after it.
- % before and after matches characters on both sides.

For example:

```
SELECT FirstName FROM Author WHERE FirstName LIKE 'R%';
```

This query returns all authors whose first name starts with the letter **R**, such as **Raul** and **Rav**.

23.4 Using Ranges in Queries

If we want to retrieve books with a number of pages between **290** and **300**, we can use comparison operators or a range expression.

23.4.1 Using Comparison Operators

```
SELECT Title FROM Book WHERE Pages >= 290 AND Pages <= 300;
```

23.4.2 Using the BETWEEN Operator

We can simplify the same query using the **BETWEEN** operator:

```
SELECT Title FROM Book WHERE Pages BETWEEN 290 AND 300;
```

The **BETWEEN** operator is inclusive of both values, making the query easier and quicker to write.

23.5 Using Sets of Values in Queries

Sometimes, data values cannot be grouped under ranges. For example, to find authors from specific countries, we can use the **IN** operator instead of repeating multiple conditions.

23.5.1 Using the IN Operator

If we want to retrieve authors from Australia or Brazil:

```
SELECT * FROM Author WHERE Country IN ('Australia', 'Brazil');
```

For more countries, such as Canada, India, and China:

```
SELECT * FROM Author WHERE Country IN ('Canada', 'India', 'China');
```

The **IN** operator simplifies the WHERE clause by listing all possible matching values within parentheses.

23.6 Summary

We have learned how to simplify SQL SELECT statements using:

- **String patterns** with the LIKE predicate.
- **Ranges** using the BETWEEN operator.
- **Sets of values** using the IN operator.

These techniques make SQL queries more efficient, readable, and easier to manage.

24 Sorting SELECT Statement Result Sets

24.1 Introduction

In this practice exercise, we will explore advanced techniques for retrieving and sorting data from a relational database table using the **SELECT** statement. We will learn how to sort result sets in both ascending and descending order and specify which column to use for sorting.

24.2 Purpose of the SELECT Statement

The main purpose of a **Database Management System (DBMS)** is not only to store data but also to facilitate data retrieval. In its simplest form, a **SELECT** statement retrieves all data from a table:

```
SELECT * FROM table_name;
```

For example, in a library database model with a table named **Book**, the query:

```
SELECT * FROM Book;
```

returns all rows and columns from the **Book** table.

We can choose to list only specific columns. For instance:

```
SELECT Title FROM Book;
```

However, the result set may not appear in any specific order. To make data easier to read, we can sort it using the **ORDER BY** clause.

24.3 Using the ORDER BY Clause

The **ORDER BY** clause is used to sort a result set based on one or more columns.

24.3.1 Sorting in Ascending Order

By default, the **ORDER BY** clause sorts data in ascending order.

Example:

```
SELECT Title FROM Book ORDER BY Title;
```

This query displays book titles in alphabetical (A–Z) order.

24.3.2 Sorting in Descending Order

To sort in descending order, use the keyword **DESC** after the column name.

Example:

```
SELECT Title FROM Book ORDER BY Title DESC;
```

This query displays book titles in reverse alphabetical (Z–A) order.

24.3.3 Sorting by Column Sequence Number

Instead of specifying the column name, you can use the **column sequence number** in the **ORDER BY** clause. The sequence number corresponds to the column's position in the **SELECT** statement.

Example:

```
SELECT Title, Pages FROM Book ORDER BY 2;
```

In this query, the number **2** refers to the second column (**Pages**) in the **SELECT** list. Therefore, the result set is sorted by the number of pages in ascending order.

24.4 Understanding Sorting Behavior

When multiple rows have similar values in the sorted column, sorting continues from the point where the data differs. This ensures consistent and predictable ordering in result sets.

24.5 Summary

We have learned how to:

- Use the **ORDER BY** clause to sort query results.
- Sort data in **ascending (ASC)** or **descending (DESC)** order.
- Use **column sequence numbers** for sorting.

These techniques make SQL queries more readable and allow for better data organization and analysis.

25 Grouping SELECT Statement Result Sets

25.1 Introduction

In this practice exercise, we will explore advanced techniques for retrieving, sorting, and grouping data from a relational database table using the **SELECT** statement. We will learn how to eliminate duplicate values, group related data, and apply conditions to grouped results.

25.2 Eliminating Duplicates from a Result Set

At times, a **SELECT** statement result set can contain duplicate values. To remove these duplicates, we use the **DISTINCT** keyword.

25.2.1 Example

Suppose we have a table named **Author**, and the **Country** column lists the two-letter country code of each author's country. The following query lists all countries of authors:

```
SELECT Country FROM Author ORDER BY 1;
```

This query returns all author countries, sorted alphabetically. However, if multiple authors come from the same country, the result set will contain duplicates. To eliminate these duplicates, we use the **DISTINCT** keyword:

```
SELECT DISTINCT Country FROM Author ORDER BY 1;
```

This reduces the result set to unique country entries only.

25.3 Grouping Data with the GROUP BY Clause

After removing duplicates, we might want to count how many authors come from each country. This can be achieved with the **GROUP BY** clause.

25.3.1 Example

```
SELECT Country, COUNT(Country) FROM Author GROUP BY Country;
```

The **GROUP BY** clause groups rows that have the same value in the **Country** column and applies the **COUNT()** function to calculate the number of authors in each country.

25.3.2 Renaming Columns with AS

The column generated by the **COUNT()** function does not have a default name. We can assign a descriptive name using the **AS** keyword:

```
SELECT Country, COUNT(Country) AS Count FROM Author GROUP BY Country;
```

This renames the derived column to **Count**, making the result set easier to understand.

25.4 Filtering Grouped Data with the HAVING Clause

Once the data is grouped, we can restrict the result set further by applying a condition to the groups using the **HAVING** clause.

25.4.1 Example

To display only countries with more than four authors:

```
SELECT Country, COUNT(Country) AS Count FROM Author GROUP BY Country HAVING COUNT(Country) > 4;
```

This query filters out countries with four or fewer authors and returns only those with five or more. For example, if **China** and **India** each have six authors, they will be displayed in the result set.

25.4.2 Difference Between WHERE and HAVING

- The **WHERE** clause filters rows **before** grouping.
- The **HAVING** clause filters groups **after** grouping.

25.5 Summary

We have learned how to:

- Eliminate duplicates from a result set using **DISTINCT**.
- Group data using the **GROUP BY** clause.
- Rename derived columns using **AS**.
- Apply conditions to grouped data using **HAVING**.

These techniques enhance our ability to retrieve meaningful insights from relational databases.

26 Hands-on Lab: String Patterns, Sorting and Grouping in MySQL

26.1 Introduction

In this practice exercise, we will learn how to create tables and load data in the MySQL database service using the phpMyAdmin graphical user interface (GUI). We will also explore filtering, sorting, and grouping query results.

26.2 Objectives

After completing this exercise, we will be able to:

- Use the **WHERE** clause to refine query results.
- Apply predicates to filter searches.
- Use wildcard characters (%) for pattern matching.
- Specify ranges using **BETWEEN ... AND ...**.
- Sort query results in ascending or descending order using **ORDER BY**.
- Group query results using **GROUP BY**.

26.3 Database Used

We will work with a sample **HR database**, which contains the following tables:

- EMPLOYEES
- JOB_HISTORY
- JOBS
- DEPARTMENTS
- LOCATIONS

Each table contains sample data.

26.4 Loading the Database

1. Open phpMyAdmin from the Skills Network Toolbox in Cloud IDE.
2. Create a blank database named HR.
3. Use the provided Script_Create_Tables.sql to create tables.

4. Load CSV files into the respective tables:
 - Departments.csv
 - Jobs.csv
 - JobsHistory.csv
 - Locations.csv
 - Employees.csv

26.5 String Patterns

We can filter query results using string patterns with the **LIKE** operator.

26.5.1 Example 1: Filter by Address

Retrieve first and last names of employees living in Elgin, IL:

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE ADDRESS LIKE '%Elgin,IL%';
```

26.5.2 Example 2: Filter by Birth Year

Retrieve employees born in the 1970s:

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE B_DATE LIKE '197%';
```

26.5.3 Example 3: Filter by Salary and Department

Retrieve employees in department 5 with salary between 60000 and 70000:

```
SELECT *
FROM EMPLOYEES
WHERE SALARY BETWEEN 60000 AND 70000 AND DEP_ID = 5;
```

26.6 Sorting

Use **ORDER BY** to sort query results.

26.6.1 Example 1: Ascending Order by Department

```
SELECT F_NAME, L_NAME, DEP_ID
FROM EMPLOYEES
ORDER BY DEP_ID;
```

26.6.2 Example 2: Descending Order

Sort by department descending, then last name descending:

```
SELECT F_NAME, L_NAME, DEP_ID
FROM EMPLOYEES
ORDER BY DEP_ID DESC, L_NAME DESC;
```

26.7 Grouping

Use **GROUP BY** to group related data.

26.7.1 Example 1: Count Employees per Department

```
SELECT DEP_ID, COUNT(*)  
FROM EMPLOYEES  
GROUP BY DEP_ID;
```

26.7.2 Example 2: Count and Average Salary

```
SELECT DEP_ID, COUNT(*) AS NUM_EMPLOYEES, AVG(SALARY) AS AVG_SALARY  
FROM EMPLOYEES  
GROUP BY DEP_ID;
```

26.7.3 Example 3: Sorting Grouped Data

Sort by average salary:

```
SELECT DEP_ID, COUNT(*) AS NUM_EMPLOYEES, AVG(SALARY) AS AVG_SALARY  
FROM EMPLOYEES  
GROUP BY DEP_ID  
ORDER BY AVG_SALARY;
```

26.7.4 Example 4: Filtering Grouped Data with HAVING

```
SELECT DEP_ID, COUNT(*) AS NUM_EMPLOYEES, AVG(SALARY) AS AVG_SALARY  
FROM EMPLOYEES  
GROUP BY DEP_ID  
HAVING COUNT(*) < 4  
ORDER BY AVG_SALARY;
```

26.8 Practice Questions

1. Retrieve employees whose first names start with 'S':

```
SELECT F_NAME, L_NAME  
FROM EMPLOYEES  
WHERE F_NAME LIKE 'S%';
```

2. Arrange all EMPLOYEES records by date of birth:

```
SELECT *  
FROM EMPLOYEES  
ORDER BY B_DATE;
```

3. Group by department and filter for average salary ≥ 60000 :

```
SELECT DEP_ID, AVG(SALARY)  
FROM EMPLOYEES  
GROUP BY DEP_ID  
HAVING AVG(SALARY)  $\geq$  60000;
```

4. Sort the previous result in descending order of average salary:

```
SELECT DEP_ID, AVG(SALARY)  
FROM EMPLOYEES  
GROUP BY DEP_ID  
HAVING AVG(SALARY)  $\geq$  60000  
ORDER BY AVG(SALARY) DESC;
```

26.9 Summary

At this point, we know:

- How to use the **WHERE** clause to refine query results.
- The search condition of **WHERE** uses a predicate to filter results.
- How to use the wildcard character (%) as a substitute for unknown characters.
- How to specify numeric ranges with **BETWEEN ... AND ...**.
- How to sort results in ascending or descending order using **ORDER BY**.
- How to group query results using **GROUP BY**.

26.10 Summary: Refining Your Results

At this point in the course, we have learned the following key concepts:

- We can use the **WHERE** clause to refine our query results.
- The search condition of the **WHERE** clause uses a predicate to refine the search.
- We can use the wildcard character (%) as a substitute for unknown characters in a pattern.
- We can use **BETWEEN ... AND ...** to specify a range of numbers.
- We can sort query results in ascending or descending order using the **ORDER BY** clause.
- We can group query results using the **GROUP BY** clause.

27 SQL Built-in Functions

27.1 Introduction

In this lesson, we will explore the **SQL functions built into the database**. While it is possible to first fetch data from a database and then perform operations from applications or notebooks, most databases include built-in functions. These functions can be directly included in SQL statements, allowing operations to be performed on data within the database itself.

Using database functions can reduce the amount of data retrieved from the database, lowering network traffic and bandwidth usage. When working with large datasets, built-in functions are often faster than performing calculations in an application.

27.2 Overview of Built-in SQL Functions

There are two main categories of SQL functions:

- **Aggregate or Column Functions**
- **Scalar and String Functions**

It is also possible to create **user-defined functions**, but that is an advanced topic beyond the scope of this lesson.

27.3 The PETRESCUE Table

For our examples, we will use a database table named **PETRESCUE** that records pet rescue transaction details. The table includes the following columns:

- **ID**

- ANIMAL
- QUANTITY
- COST
- RESCUEDATE

This table is populated with multiple rows of data used for demonstration.

27.4 Aggregate or Column Functions

An **aggregate function** takes a collection of values (for example, all values in a column) and returns a single value or NULL. Examples include **SUM**, **MIN**, **MAX**, and **AVG**.

27.4.1 SUM Function

Used to add up all the values in a column.

```
SELECT SUM(COST) FROM PETRESCUE;
```

To rename the result column:

```
SELECT SUM(COST) AS SUM_OF_COST FROM PETRESCUE;
```

27.4.2 MIN and MAX Functions

- **MIN** returns the lowest value.
- **MAX** returns the highest value.

Example:

```
SELECT MAX(QUANTITY) FROM PETRESCUE;
```

To get the minimum ID for dogs:

```
SELECT MIN(ID) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

27.4.3 AVG Function

Used to calculate the average or mean value.

```
SELECT AVG(COST) FROM PETRESCUE;
```

You can also perform mathematical operations between columns:

```
SELECT AVG(COST / QUANTITY) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

In this case, the cost is divided by quantity to calculate the average cost per dog.

27.5 Scalar and String Functions

Scalar functions operate on individual values rather than groups of values.

27.5.1 Example: ROUND Function

To round up or down every value in the COST column to the nearest integer:

```
SELECT ROUND(COST) FROM PETRESCUE;
```

27.5.2 String Functions

String functions operate on character (CHAR) or variable character (VARCHAR) values.

27.5.2.1 LENGTH Function

To find the length of each value in the ANIMAL column:

```
SELECT LENGTH(ANIMAL) FROM PETRESCUE;
```

27.5.2.2 UPPER and LOWER Functions

To convert strings to uppercase or lowercase:

```
SELECT UPPER(ANIMAL) FROM PETRESCUE;  
SELECT LOWER(ANIMAL) FROM PETRESCUE;
```

These functions are particularly useful in WHERE clauses when you are unsure of the case format of stored values.

Example:

```
SELECT * FROM PETRESCUE WHERE LOWER(ANIMAL) = 'cat';
```

27.5.2.3 Nested Functions

You can nest functions to perform combined operations. For instance, to retrieve distinct uppercase values from the ANIMAL column:

```
SELECT DISTINCT(UPPER(ANIMAL)) FROM PETRESCUE;
```

27.6 Summary

In this lesson, we learned about:

- **Aggregate functions** such as SUM, MIN, MAX, and AVG.
- **Scalar and string functions** like ROUND, LOWER, and UPPER.
- How these functions can be applied directly in SQL queries to improve performance and efficiency.

28 Date and Time SQL Functions

28.1 Introduction

In this lesson, we will learn about **date and time functions in SQL**. By the end of this lesson, we will be able to:

- Define date and time functions.

- Discuss date and time arithmetic.

Most databases contain special data types for **dates** and **times**. SQL supports the following types:

- **DATE** – 8 digits in the format YYYYMMDD
- **TIME** – 6 digits in the format HHMMSS
- **TIMESTAMP** – 20 digits in the format YYYYMMDDHHMMSSZZZZZZ, where ZZZZZZ represents microseconds.

These data types are used to record and manipulate temporal information efficiently.

28.2 Extracting Components of Date and Time

SQL provides functions to extract specific parts of a date or time value. These include:

- **DAY**
- **MONTH**
- **DAYOFMONTH**
- **DAYOFWEEK**
- **DAYOFYEAR**
- **WEEK**
- **HOURL**
- **MINUTE**
- **SECOND**

28.2.1 Example: Extracting the Day from a Date

The DAY() function is used to extract the day portion from a date.

```
SELECT DAY(RESCUEDATE)
FROM PETRESCUE
WHERE ANIMAL = 'Cat';
```

This query returns the day portion of each rescue date where the animal is a cat.

28.3 Using Date and Time Functions in WHERE Clauses

Date and time functions can be used to filter data based on specific conditions.

28.3.1 Example: Get Number of Rescues in May

The following query retrieves the number of rescues that occurred in May (Month 5):

```
SELECT COUNT(*)
FROM PETRESCUE
WHERE MONTH(RESCUEDATE) = 5;
```

28.4 Performing Date and Time Arithmetic

SQL allows performing arithmetic operations with dates and times.

28.4.1 Example: Add 3 Days to Each Rescue Date

This query calculates the date three days after each rescue date:

```
SELECT DATE_ADD(RESCUEDATE, INTERVAL 3 DAY)
FROM PETRESCUE;
```

This is useful, for example, if an order must be processed within three days after rescue.

28.4.2 Example: Calculate Days Passed Since Each Rescue

Special registers such as **CURRENT DATE** and **CURRENT TIME** can be used to find the current system date and time.

```
SELECT CURRENT_DATE - RESCUEDATE
FROM PETRESCUE;
```

This query returns the number of days that have passed since each rescue date.

28.5 Summary

- SQL includes **DATE**, **TIME**, and **TIMESTAMP** data types.
- Functions exist to extract date and time components such as day, month, week, hour, and second.
- Date and time functions can be used in **WHERE** clauses.
- Date arithmetic enables adding or subtracting time intervals.
- Special registers like **CURRENT_DATE** and **CURRENT_TIME** provide real-time values for calculations.

29 Hands-on Practice: Built-in Functions

29.1 Introduction

In this practice exercise, we will explore **built-in functions in SQL** that enhance our data analysis capabilities. These include aggregation, scalar, string, and date functions. We will learn how to use each type effectively in MySQL.

29.2 Software Used

We will use **MySQL**, a Relational Database Management System (RDBMS) that stores, manipulates, and retrieves data efficiently.

29.3 Objectives

After completing this exercise, we will be able to compose queries using:

- Aggregation Functions
- Scalar Functions
- String Functions
- Date Functions

29.4 Creating the Database

1. Open **Skills Network Toolbox** and navigate to the **Database** section.
2. Click **MySQL** and then **Create** to start the MySQL service.
3. Once started, click the **phpMyAdmin** button to open the interface.
4. In phpMyAdmin, click **New** and create a database named **Mysql_Learners**.
5. Keep the default **utf8 encoding**.

29.5 Creating the PETRESCUE Table

1. Download the provided script file PETRESCUE-CREATE.sql.
2. Import it into the **Mysql_Learners** database using the **Import** option.

Once executed, the **PETRESCUE** table will be created with the following structure:

Column Name	Data Type	Description
ID	INTEGER	ID of the entry
ANIMAL	VARCHAR(20)	Type of animal
QUANTITY	INTEGER	Number of animals
COST	DECIMAL(6,2)	Cost incurred
RESCUEDATE	DATE	Date of rescue

29.6 Aggregation Functions

29.6.1 Total Cost of All Animal Rescues

```
SELECT SUM(COST) AS SUM_OF_COST FROM PETRESCUE;
```

29.6.2 Maximum and Minimum Quantity of Animals Rescued

```
SELECT MAX(QUANTITY) FROM PETRESCUE;  
SELECT MIN(QUANTITY) FROM PETRESCUE;
```

29.6.3 Average Cost of Animal Rescues

```
SELECT AVG(COST) FROM PETRESCUE;
```

29.7 Scalar and String Functions

29.7.1 Rounded Cost of Each Rescue

```
SELECT ROUND(COST) FROM PETRESCUE;  
SELECT ROUND(COST, 2) FROM PETRESCUE;
```

29.7.2 Length of Each Animal Name

```
SELECT LENGTH(ANIMAL) FROM PETRESCUE;
```

29.7.3 Convert Animal Names to Upper and Lower Case

```
SELECT UCASE(ANIMAL) FROM PETRESCUE;  
SELECT LCASE(ANIMAL) FROM PETRESCUE;
```

29.8 Date Functions

29.8.1 Display Rescue Dates by Day, Month, and Year

```
SELECT DAY(RESCUEDATE) FROM PETRESCUE;  
SELECT MONTH(RESCUEDATE) FROM PETRESCUE;  
SELECT YEAR(RESCUEDATE) FROM PETRESCUE;
```

29.8.2 Add or Subtract Days from Rescue Date

```
SELECT DATE_ADD(RESCUEDATE, INTERVAL 3 DAY) FROM PETRESCUE;  
SELECT DATE_SUB(RESCUEDATE, INTERVAL 3 DAY) FROM PETRESCUE;
```

29.8.3 Calculate Days Since Rescue

```
SELECT DATEDIFF(CURRENT_DATE, RESCUEDATE) FROM PETRESCUE;  
SELECT FROM_DAYS(DATEDIFF(CURRENT_DATE, RESCUEDATE)) FROM PETRESCUE;
```

29.9 Practice Problems

29.9.1 Average Cost of Rescuing a Dog

```
SELECT AVG(COST/QUANTITY) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

29.9.2 Display Animal Names in Uppercase Without Duplicates

```
SELECT DISTINCT UCASE(ANIMAL) FROM PETRESCUE;
```

29.9.3 Display All Cat Rescues

```
SELECT * FROM PETRESCUE WHERE LCASE(ANIMAL) = 'cat';
```

29.9.4 Number of Rescues in the 5th Month

```
SELECT SUM(QUANTITY) FROM PETRESCUE WHERE MONTH(RESCUEDATE) = '05';
```

29.9.5 Display ID and Target Date (1 Year After Rescue)

```
SELECT ID, DATE_ADD(RESCUEDATE, INTERVAL 1 YEAR) FROM PETRESCUE;
```

29.10 Conclusion

By completing this exercise, we have learned to:

- Use **aggregation functions** to calculate totals, maximums, minimums, and averages.
- Apply **scalar functions** for rounding numerical values.
- Utilize **string functions** for formatting text.
- Perform **date operations** to manipulate and calculate date-related data.

30 Subqueries and Nested Select Statements

30.1 Introduction

In this practice exercise, we will learn how to write **subqueries** or **nested select statements** in SQL. Subqueries allow us to create more powerful and flexible queries by nesting one query inside another.

30.2 What Are Subqueries?

A **subquery** (also known as a **sub-select**) is a query placed within parentheses and embedded inside another query. This approach helps us perform operations that might not be possible with a single query.

30.2.1 Example of a Nested Query

Consider the `employees` table, which contains columns such as **employee_id**, **first_name**, **last_name**, and **salary**. A nested query can be used within the **WHERE** clause of another query, such as:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

This example retrieves all employees earning more than the **average salary**.

30.3 Why Use Subqueries?

Built-in aggregate functions like `AVG()` cannot always be evaluated directly in the **WHERE** clause. Using a subquery helps us evaluate such functions and use their results in conditions.

30.4 Types of Subqueries

Subqueries can appear in various parts of a query. The most common types are:

30.4.1 1. Subqueries in the WHERE Clause

These are used to filter results based on the outcome of another query.

Example:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

30.4.2 2. Subqueries in the SELECT Clause (Column Expressions)

Subqueries can also appear in the list of columns to calculate or compare values dynamically.

Example:


```
SELECT employee_id, salary,  
       (SELECT AVG(salary) FROM employees) AS average_salary  
FROM employees;
```

This query compares each employee's salary with the overall average salary.

30.4.3 3. Subqueries in the FROM Clause (Derived Tables)

Subqueries can also be used as temporary tables within the **FROM** clause. These are often called **derived tables** or **table expressions**.

Example:

```
SELECT *  
FROM (SELECT employee_id, first_name, last_name, department_id  
      FROM employees) AS employee_info;
```

This derived table excludes sensitive fields like **date_of_birth** and **salary**, providing a simplified dataset for further processing.

30.5 Advantages of Using Subqueries

- Allow breaking down complex queries into simpler components.
- Enable calculations within specific clauses (**WHERE**, **SELECT**, **FROM**).
- Useful for filtering, comparison, and temporary table creation.

30.6 Summary

In this exercise, we learned:

- The concept and structure of **subqueries**.
- How to use subqueries in **WHERE**, **SELECT**, and **FROM** clauses.
- How subqueries help overcome limitations of aggregate functions.

By understanding subqueries, we can create richer, more dynamic SQL queries that make our data analysis more efficient and powerful.

31 Hands-on Practice: Subqueries and Nested Select Statements

31.1 Objectives

After completing this exercise, we will be able to:

- Write SQL queries that demonstrate the necessity of using subqueries.
- Compose subqueries in the **WHERE** clause.
- Build column expressions using subqueries.
- Write table expressions using subqueries.

31.2 Software Used

In this practice exercise, we will use **MySQL**, a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.

31.3 Database Overview

The database used in this exercise is a sample **HR database**, consisting of five tables:

- **EMPLOYEES**
- **JOB_HISTORY**
- **JOBS**
- **DEPARTMENTS**
- **LOCATIONS**

Each table contains a few rows of sample data representing employees, departments, job roles, and related information.

31.4 Loading the Database

Follow the steps below to set up the HR database in MySQL:

1. Open the **phpMyAdmin** interface from the Skills Network Toolbox in Cloud IDE.
2. Create a blank database named **HR**.
3. Use the provided SQL script `Script_Create_Tables.sql` to create the necessary tables.
4. Download the following CSV files and import them into their respective tables:
 - Departments.csv
 - Jobs.csv
 - JobsHistory.csv
 - Locations.csv
 - Employees.csv

31.5 Understanding Subqueries and Nested SELECTs

A **subquery** is a query nested inside another SQL query. Subqueries help retrieve intermediate results that can be used by the main query.

31.5.1 Example 1: Retrieve Employees with Salary Below Average

The following query returns all employees whose salaries are lower than the average salary:

```
SELECT *  
FROM EMPLOYEES  
WHERE SALARY < (SELECT AVG(SALARY) FROM EMPLOYEES);
```

The subquery (`SELECT AVG(SALARY) FROM EMPLOYEES`) calculates the average salary, which is then compared with each employee's salary.

31.5.2 Example 2: Display Employee Salary with Maximum Salary Column

The query below retrieves each employee's ID, salary, and the maximum salary value:

```
SELECT EMP_ID, SALARY, (SELECT MAX(SALARY) FROM EMPLOYEES) AS MAX_SALARY
FROM EMPLOYEES;
```

This uses a subquery within the **SELECT** clause to add a computed column.

31.5.3 Example 3: Retrieve the Oldest Employee

To find the first and last names of the oldest employee (the one with the smallest birth date):

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE B_DATE = (SELECT MIN(B_DATE) FROM EMPLOYEES);
```

31.5.4 Example 4: Average Salary of Top 5 Earners

Subqueries can also be used as **derived tables**. The following example finds the average salary of the top 5 highest-paid employees:

```
SELECT AVG(SALARY)
FROM (SELECT SALARY
      FROM EMPLOYEES
      ORDER BY SALARY DESC
      LIMIT 5) AS SALARY_TABLE;
```

It's important to assign an alias (in this case, `SALARY_TABLE`) to every derived table.

31.6 Practice Exercises

31.6.1 Exercise 1: Average Salary of Five Lowest Earners

```
SELECT AVG(SALARY)
FROM (SELECT SALARY
      FROM EMPLOYEES
      ORDER BY SALARY
      LIMIT 5) AS SALARY_TABLE;
```

31.6.2 Exercise 2: Employees Older Than Average Age

```
SELECT *
FROM EMPLOYEES
WHERE YEAR(FROM_DAYS(DATEDIFF(CURRENT_DATE, B_DATE))) >
      (SELECT AVG(YEAR(FROM_DAYS(DATEDIFF(CURRENT_DATE, B_DATE))))
      FROM EMPLOYEES);
```

31.6.3 Exercise 3: Employee IDs and Years of Service, and Average Years of Service from Job History

```
SELECT EMPL_ID,
      YEAR(FROM_DAYS(DATEDIFF(CURRENT_DATE, START_DATE))),
      (SELECT AVG(YEAR(FROM_DAYS(DATEDIFF(CURRENT_DATE, START_DATE))))
      FROM JOB_HISTORY)
FROM JOB_HISTORY;
```

31.7 Conclusion

We have completed this practice exercise on **Subqueries and Nested SELECT Statements**. We can now:

- Write SQL queries that demonstrate the use of subqueries.
- Use subqueries in **WHERE**, **SELECT**, and **FROM** clauses.
- Build column and table expressions using subqueries to enhance query performance and flexibility.

32 Working with Multiple Tables

32.1 Introduction

In this lesson, we will learn how to write SQL queries that access more than one table. There are several ways to work with multiple tables in a single query, such as:

- Using **subqueries**
- Using **implicit joins**
- Using **join operators** such as **INNER JOIN** and **OUTER JOIN**

In this exercise, we will focus on **subqueries** and **implicit joins**.

32.2 Example Tables: Employees and Departments

We will use two tables, **Employees** and **Departments**, for our examples.

32.2.1 Employees Table

The **Employees** table contains columns such as:

- Employee ID
- First Name
- Last Name
- Salary
- Department ID

32.2.2 Departments Table

The **Departments** table contains columns such as:

- Department ID
- Department Name
- Manager ID
- Location ID

These tables will be used to demonstrate how to retrieve and combine data.

32.3 Using Subqueries to Access Multiple Tables

Subqueries allow us to retrieve data from one table based on conditions derived from another table.

32.3.1 Example 1: Retrieve Employees with Matching Department IDs

To get employee records where the **Department ID** exists in the **Departments** table:

```
SELECT *  
FROM EMPLOYEES  
WHERE DEPT_ID IN (SELECT DEPT_ID FROM DEPARTMENTS);
```

Here, the outer query accesses the **Employees** table, while the subquery filters results using data from the **Departments** table.

32.3.2 Example 2: Retrieve Employees from a Specific Location

Suppose we want to list employees working in a specific location, but location information is available only in the **Departments** table:

```
SELECT *  
FROM EMPLOYEES  
WHERE DEPT_ID IN (SELECT DEPT_ID FROM DEPARTMENTS WHERE LOC_ID = 'L0002');
```

The subquery retrieves the department IDs for the given location, which are then used to filter employees.

32.3.3 Example 3: Retrieve Department Details for High-Earning Employees

To find the **Department ID** and **Department Name** for employees earning more than \$70,000:

```
SELECT DEPT_ID, DEPT_NAME  
FROM DEPARTMENTS  
WHERE DEPT_ID IN (SELECT DEPT_ID FROM EMPLOYEES WHERE SALARY > 70000);
```

This subquery selects departments with employees whose salaries exceed the specified amount.

32.4 Accessing Multiple Tables Using Implicit Joins

Another way to work with multiple tables is by specifying them together in the **FROM** clause.

32.4.1 Example 1: Full Join Example

```
SELECT *  
FROM EMPLOYEES, DEPARTMENTS;
```

This creates a **Cartesian product** (full join), where every row in one table is combined with every row in the other.

32.4.2 Example 2: Filtering Matching Department IDs

To limit results to rows with matching department IDs:

```
SELECT *  
FROM EMPLOYEES, DEPARTMENTS  
WHERE EMPLOYEES.DEPT_ID = DEPARTMENTS.DEPT_ID;
```

Here, we use fully qualified column names to avoid ambiguity when both tables have similarly named columns.

32.4.3 Example 3: Using Table Aliases

To make queries cleaner, we can use table aliases:

```
SELECT *  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPT_ID = D.DEPT_ID;
```

Aliases **E** and **D** represent **Employees** and **Departments**, respectively.

32.4.4 Example 4: Display Department Name for Each Employee

```
SELECT E.EMP_ID, D.DEPT_NAME  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPT_ID = D.DEPT_ID;
```

This query displays the employee ID along with their department name.

32.5 Conclusion

In this lesson, we have learned how to:

- Retrieve data from multiple tables using **subqueries**
- Access multiple tables with **implicit joins**
- Use **table aliases** for cleaner and more readable queries

These techniques are foundational for mastering multi-table SQL operations.

33 Hands-on Lab: Working with Multiple Tables

33.1 Objectives

After completing this exercise, we will be able to:

- Write SQL queries that access more than one table.
- Compose queries that access multiple tables using nested statements in the **WHERE** clause.
- Build queries with multiple tables in the **FROM** clause.
- Write implicit join queries with join criteria specified in the **WHERE** clause.
- Specify aliases for table names and qualify column names with table aliases.

In this practice exercise, we will complete SQL problems that provide hands-on experience with queries accessing multiple tables.

33.2 Software Used

We will use **MySQL**, a Relational Database Management System (RDBMS) designed to store, manipulate, and retrieve data efficiently.

33.3 Database Used

We will work with a sample **HR database** consisting of five tables:

- **EMPLOYEES**
- **JOB_HISTORY**
- **JOBS**
- **DEPARTMENTS**
- **LOCATIONS**

Each table contains a few rows of sample data representing a basic HR schema.

33.4 Loading the Database

Follow these steps to load and prepare the database:

1. Open **phpMyAdmin** from the Skills Network Toolbox in Cloud IDE.
2. Create a blank database named **HR**.
3. Use the provided script **Script_Create_Tables.sql** to create the required tables.
4. Download and import the following CSV files as data for the respective tables:
 - Departments.csv
 - Jobs.csv
 - JobsHistory.csv
 - Locations.csv
 - Employees.csv

33.5 Accessing Multiple Tables with Subqueries

Let's explore examples that demonstrate queries accessing multiple tables using **subqueries**.

33.5.1 Example 1: Retrieve Employee Records for Jobs in the JOBS Table

```
SELECT *  
FROM EMPLOYEES  
WHERE JOB_ID IN (SELECT JOB_IDENT FROM JOBS);
```

This query retrieves employee records that correspond to existing job identifiers in the **JOBS** table.

33.5.2 Example 2: Retrieve JOB Information for Employees Earning Over \$70,000

```
SELECT JOB_TITLE, MIN_SALARY, MAX_SALARY, JOB_IDENT  
FROM JOBS
```

```
WHERE JOB_IDENT IN (SELECT JOB_ID FROM EMPLOYEES WHERE SALARY > 70000);
```

This retrieves job details for employees whose salary exceeds \$70,000.

33.6 Accessing Multiple Tables with Implicit Joins

Now let's look at how to retrieve data using **implicit joins**.

33.6.1 Example 1: Retrieve Employee Records for Jobs in the JOBS Table

```
SELECT *  
FROM EMPLOYEES, JOBS  
WHERE EMPLOYEES.JOB_ID = JOBS.JOB_IDENT;
```

This query retrieves all employee data along with matching job details based on job identifiers.

33.6.2 Example 2: Using Table Aliases for Simplicity

```
SELECT *  
FROM EMPLOYEES E, JOBS J  
WHERE E.JOB_ID = J.JOB_IDENT;
```

Here, **E** and **J** are shorter aliases for **EMPLOYEES** and **JOBS**, making the query easier to read.

33.6.3 Example 3: Retrieve Employee ID, Name, and Job Title

```
SELECT EMP_ID, F_NAME, L_NAME, JOB_TITLE  
FROM EMPLOYEES E, JOBS J  
WHERE E.JOB_ID = J.JOB_IDENT;
```

This query returns only the Employee ID, first name, last name, and job title for each record.

33.6.4 Example 4: Use Fully Qualified Column Names

```
SELECT E.EMP_ID, E.F_NAME, E.L_NAME, J.JOB_TITLE  
FROM EMPLOYEES E, JOBS J  
WHERE E.JOB_ID = J.JOB_IDENT;
```

Fully qualifying columns clarifies which table each column originates from.

33.7 Practice Problems

33.7.1 Problem 1: Retrieve Employees with JOB_TITLE as 'Jr. Designer'

a. Using Subqueries:

```
SELECT *  
FROM EMPLOYEES  
WHERE JOB_ID IN (SELECT JOB_IDENT FROM JOBS WHERE JOB_TITLE = 'Jr. Designer');
```

b. Using Implicit Joins:

```
SELECT *  
FROM EMPLOYEES E, JOBS J  
WHERE E.JOB_ID = J.JOB_IDENT AND J.JOB_TITLE = 'Jr. Designer';
```


33.7.2 Problem 2: Retrieve JOB Information for Employees Born After 1976

a. Using Subqueries:

```
SELECT JOB_TITLE, MIN_SALARY, MAX_SALARY, JOB_ID
FROM JOBS
WHERE JOB_ID IN (SELECT JOB_ID FROM EMPLOYEES WHERE YEAR(B_DATE) > 1976);
```

b. Using Implicit Joins:

```
SELECT J.JOB_TITLE, J.MIN_SALARY, J.MAX_SALARY, J.JOB_ID
FROM JOBS J, EMPLOYEES E
WHERE J.JOB_ID = E.JOB_ID AND YEAR(E.B_DATE) > 1976;
```

33.8 Conclusion

At the end of this exercise, we are now able to:

- Write SQL queries that access multiple tables.
- Use nested subqueries in the **WHERE** clause.
- Build queries with multiple tables in the **FROM** clause.
- Implement implicit joins with proper join criteria.
- Apply table aliases and qualify column names effectively.

34 Summary: Functions, Multiple Tables, and Sub-queries

At this stage of the course, we know the following:

34.1 Database Management Tools

- Database management systems provide **built-in functions** that allow operations to be performed directly within the database.
- These tools enable efficient data processing and manipulation without transferring data externally.

34.2 Efficiency with Built-in Functions

- When working with large datasets, using built-in database functions can significantly **save time**.
- Instead of retrieving data into an application and then executing operations, it is often faster and more efficient to perform operations directly within the database.

34.3 Using Sub-queries

- Sub-queries help create **more powerful and flexible queries**.
- A **sub-select expression** can evaluate built-in aggregate functions such as the **AVERAGE()** function.

34.4 Derived Tables or Table Expressions

- **Derived tables** (also called table expressions) are sub-queries where the **outer query** uses the **result of the sub-query** as a data source.

34.5 Key Takeaways

- Built-in functions enhance performance and simplify data operations.
- Sub-queries and derived tables increase the flexibility and power of SQL queries.
- Understanding these concepts allows us to write more efficient and advanced SQL queries.

Module 4

35 Accessing Databases Using Python

35.1 Introduction

Databases are powerful tools for data scientists. In this module, we will learn how to access databases using Python. By the end of this module, we will be able to:

- Explain basic concepts related to connecting Python to databases.
- Create tables, load data, and query data using SQL from Jupyter Notebooks.
- Analyze data retrieved from databases.

In lab assignments, we will also learn how to:

- Create an instance in the cloud.
- Connect to a database.
- Query data using SQL.
- Analyze data using Python.

35.2 Benefits of Using Python for Databases

Python is a popular scripting language with a rich ecosystem, making it ideal for connecting to databases.

35.2.1 Advantages of Python

- **Easy to Learn:** Python has simple, readable syntax.
- **Open Source:** Available freely and supported on multiple platforms.
- **Cross-Platform Compatibility:** Programs can run on different systems without modification (if system-dependent features are avoided).

35.2.2 Popular Python Packages for Data Science

Python provides several libraries for data analysis and visualization:

- NumPy
- Pandas
- Matplotlib
- SciPy

35.3 Python Database API (DB API)

Python supports relational database systems through the **Database API (DB API)**, which simplifies writing Python code to access databases. Detailed documentation is readily available for reference.

35.4 Jupyter Notebooks

Jupyter Notebooks are popular in the field of data science. They provide an environment for creating and sharing documents containing live code, equations, visualizations, and explanatory text.

35.4.1 Examples of Notebook Interfaces

- Mathematica Notebook
- Maple Worksheet
- Matlab Notebook
- IPython Jupyter
- R Markdown
- Apache Zeppelin
- Apache Spark Notebook
- Databricks Cloud

35.4.2 Features of Jupyter Notebooks

- Supports over **40 programming languages**, including Python, R, Julia, and Scala.
- Allows sharing via **email, Dropbox, GitHub**, and the Jupyter Notebook Viewer.
- Produces **rich interactive outputs** such as HTML, images, videos, LaTeX, and custom types.
- Can integrate with big data tools like **Apache Spark** and libraries such as **pandas, scikit-learn, ggplot2**, and **TensorFlow**.

35.5 Accessing Databases from Jupyter Notebooks

A typical user accesses databases using Python code written in a Jupyter Notebook, which serves as a web-based editor. The Python program communicates with the **Database Management System (DBMS)** via API calls.

35.6 SQL APIs and Python DB APIs

An **Application Programming Interface (API)** is a set of functions used to access a service.

35.6.1 SQL API Basics

- The SQL API provides library function calls for interacting with the DBMS.
- To execute SQL statements, the program:
 1. Connects to the DBMS using an API call.
 2. Builds SQL statements as text strings.
 3. Passes these statements to the DBMS through API calls.
 4. Retrieves query results and status information.
 5. Ends database access by disconnecting via an API call.

35.6.2 Proprietary APIs in Popular SQL-Based DBMS

Each database system has its own API library for connection and communication.

Database System	API Used
MySQL	C API
PostgreSQL	psycopg2 API
IBM DB2	IBM_DB API
SQL Server	dblib API
Microsoft Windows OS	ODBC
Oracle	OCI
Java Applications	JDBC

35.7 Summary

- Python provides powerful tools for connecting to and analyzing data from databases.
- Jupyter Notebooks offer an interactive platform for coding, data analysis, and visualization.
- APIs like DB API simplify database connectivity.
- SQL APIs facilitate the exchange of SQL commands and data between applications and databases.
- Proprietary APIs support various database systems for seamless integration with Python applications.

36 Writing Code Using DB-API

36.1 Introduction

Welcome to **Writing Code Using DB-API**. In this section, we will learn about Python's Database API (DB-API), its components, and how to use it to connect and interact with databases. By the end, we will be able to:

- Explain the basic concepts of the Python DB-API and database cursors.
- Write Python code to connect to and interact with databases using DB-API.

36.2 Understanding Python DB-API

The **Python DB-API** is a standard interface for connecting to relational databases. It allows developers to write one program that can work with multiple kinds of databases, without needing to write separate programs for each.

36.2.1 Advantages of Using DB-API

- **Easy to Implement and Understand:** The DB-API is straightforward, making it beginner-friendly.
- **Encourages Consistency:** It promotes similarity among Python database modules.
- **Portability:** The same code can work with multiple databases.
- **Broader Connectivity:** Enables access to a wide range of relational databases from Python.

36.3 Connecting Python to Databases

Python connects to a database using **DB-API calls**. The user writes Python programs, often in environments like **Jupyter Notebooks**, that communicate with the **Database Management System (DBMS)** via these API calls.

36.3.1 Database Libraries and Their DB-APIs

Each database system provides its own Python library for DB-API connectivity.

Database	Python Library (DB-API)
IBM DB2	ibm_db
MySQL	mysql.connector
PostgreSQL	psycopg2
MongoDB	pymongo

36.4 Main Concepts in Python DB-API

The two core components in DB-API are **Connection Objects** and **Cursor Objects**.

36.4.1 Connection Objects

Connection objects manage the connection between Python and the database. They provide several methods for handling transactions:

- **cursor():** Creates and returns a new cursor object.

- **commit():** Commits any pending transaction to the database.
- **rollback():** Rolls back to the start of any pending transaction.
- **close():** Closes the database connection.

36.4.2 Cursor Objects

Cursors are used to execute SQL queries and retrieve results. They function like cursors in text editors—allowing you to move through records.

- Used to run and fetch query results.
- Keep track of the program's current position within the result set.
- Cursors from the same connection are **not isolated**—changes made by one cursor are visible to others.
- Cursors from different connections **may or may not** be isolated, depending on transaction support.

36.4.3 Cursor Behavior and Usage

A database cursor acts like a **file handle**:

- Just as a file is opened to access its contents, a cursor is opened to access query results.
- After finishing, the cursor must be closed to free resources.
- Cursors track the current position within query results.

36.5 Writing Python Code with DB-API

Let's walk through a basic example of how Python interacts with a database using the DB-API:

1. **Import the Database Module:** Import the Python library corresponding to your database (e.g., `ibm_db`, `psycopg2`).
2. **Connect to the Database:** Use the `connect()` function and pass parameters such as database name, username, and password. This returns a **connection object**.
3. **Create a Cursor Object:** Use `connection.cursor()` to create a cursor.
4. **Execute Queries:** Run SQL queries using cursor methods like `execute()`.
5. **Fetch Results:** Retrieve query results using methods like `fetchone()` or `fetchall()`.
6. **Close the Connection:** Once done, close the connection to avoid unused connections consuming resources.

36.6 Summary

- DB-API provides a unified interface to connect Python with different databases.
- **Connection** and **Cursor** objects are key components of database interaction.
- Each database uses its own Python library that implements the DB-API standard.
- Always close connections after database operations to free system resources.

37 Accessing Databases Using SQL Magic

37.1 Introduction

Welcome to this section on **Accessing Databases Using SQL Magic**. In this section, we will learn:

- What magic statements are.
- The difference between **line magics** and **cell magics**.
- Some popularly used magic statements.
- How to use **SQL Magic** in Jupyter Notebooks to access databases.

37.2 What Are Magic Statements in Jupyter Notebooks?

Magic commands, also known as **magic functions**, are special commands in Jupyter Notebooks that provide additional functionality. They are not part of standard Python syntax but affect the notebook's behavior. These commands are designed to make common data analysis and coding tasks easier and more efficient.

There are two types of magic statements in Jupyter Notebooks:

1. **Line Magics**
2. **Cell Magics**

37.3 Line Magics

Line magics are prefixed with a single % character and operate on a **single line** of input. They are similar to command-line calls in a terminal shell.

37.3.1 Commonly Used Line Magics

- `%pwd` – Prints the current working directory.
- `%ls` – Lists all files in the current directory.
- `%history` – Displays the command history.
- `%reset` – Clears all variables from the namespace.
- `%who` – Lists all variables in the namespace.
- `%whos` – Provides detailed information about variables in the namespace.
- `%matplotlib inline` – Displays matplotlib plots within the notebook.
- `%timeit` – Measures the execution time of a single statement.

37.3.2 Example of Line Magics

You can use multiple line magics in one Jupyter Notebook cell, each on a separate line:

```
%pwd  
%ls
```

This will print the current working directory and list all files in it.

37.4 Cell Magics

Cell magics are prefixed with two `%%` characters and operate on **multiple lines** of input. They can transform or execute the entire cell, sometimes even in a different programming language.

37.4.1 Examples of Cell Magics

- `%%timeit` – Times the execution of the entire cell.
- `%%writefile myfile.txt` – Writes the contents of the cell to `myfile.txt`.
- `%%html` – Renders HTML code within the cell.
- `%%javascript` or `%%js` – Executes JavaScript code.
- `%%bash` – Executes bash commands in the cell.

37.4.1.1 Example:

```
%%html
<h1>Hello World</h1>
```

This displays **Hello World** as a heading.

```
%%bash
echo "Hello World"
```

This prints **Hello World** in the terminal output.

37.5 Installing and Using SQL Magic

Before using **SQL Magic**, some dependencies must be installed.

37.5.1 Step 1: Install the Required Library

Run the following command in a notebook cell:

```
!pip install ipython-sql
```

This installs the `ipython-sql` package required for SQL Magic.

37.5.2 Step 2: Load the SQL Extension

Load the SQL extension into your Jupyter Notebook using:

```
%load_ext sql
```

37.5.3 Step 3: Establish a Database Connection

To connect to a database, use the `%sql` magic command with the connection string. For example, to connect to a SQLite database named **HR.db**:

```
%sql sqlite:///HR.db
```


37.5.4 Step 4: Run SQL Queries

Once connected, you can use `%sql` (line magic) or `%%sql` (cell magic) to execute SQL queries directly from the notebook.

37.5.4.1 Example:

```
%%sql  
SELECT * FROM Employee;
```

This command displays the contents of the **Employee** table from the connected database.

37.6 Summary

- Magic commands extend Jupyter Notebook's functionality beyond Python.
- **Line Magics** operate on a single line, while **Cell Magics** can execute or transform entire cells.
- Common magic commands include `%pwd`, `%ls`, `%who`, and `%timeit`.
- **SQL Magic** allows us to connect to databases and execute SQL queries directly within a Jupyter Notebook.
- We can connect to databases like SQLite using the `%sql` command after loading the SQL extension.

38 Analyzing Data with Python

38.1 Introduction

Welcome to this section on **Analyzing Data with Python**. In this exercise, we will:

- Perform **Exploratory Data Analysis (EDA)** using Pandas.
- Utilize **SQLite3** with Python for data storage and retrieval.
- Visualize data using **Seaborn** for insights and pattern detection.

We will be using the **McDonald's Menu Nutritional Facts** dataset for popular menu items at McDonald's to perform basic exploratory analysis.

38.2 About McDonald's Dataset

McDonald's is an American fast-food company and the world's largest restaurant chain by revenue. The menu includes hamburgers, fries, soft drinks, milkshakes, desserts, salads, fish, smoothies, and fruits. McDonald's provides nutritional information for its menu items to help customers make informed food choices. The dataset used in this exercise is obtained from **Kaggle**.

38.3 Creating the Database

We will create a database table on an SQL server to store the McDonald's menu nutrition facts dataset.

38.3.1 Using SQLite3

SQLite3 is an in-process Python library that provides a self-contained, serverless, zero-configuration transactional SQL database engine.

38.3.2 Steps to Load Data into the Database

1. **Read the CSV File** – Use the `pandas.read_csv()` function to load the dataset into a variable, e.g., `data`.
2. **Create a Database Connection** – Use `sqlite3.connect('mcdonalds.db')` and save the connection object in a variable, e.g., `conn`.
3. **Save Data to SQL Table** – Use the `to_sql()` method of the Pandas DataFrame to load data into the database.

Example:

```
data.to_sql('mcdonalds_nutrition', conn)
```

Now, the CSV data is available as a table `mcdonalds_nutrition` that can be queried.

38.4 Retrieving Data Using Pandas

To retrieve data from the database table:

1. Load the data into a DataFrame using the `read_sql()` method.
2. Pass the SQL SELECT query and connection object as parameters.
3. View the first few rows of the DataFrame using the `head()` method.

38.4.1 Exploring the Dataset

Use Pandas methods for mathematical and statistical exploration.

```
data.describe()
```

This displays summary statistics such as count, mean, median, standard deviation, and maximum values.

From the dataset, we observe:

- 260 food items are listed.
- 9 unique categories of food items.
- Maximum total fat value is **118 grams**.

38.5 Exploring Sodium Content

38.5.1 Understanding Sodium

Sodium, primarily found in table salt, is essential for fluid balance and maintaining blood pressure. However, excessive sodium intake can cause health issues such as high blood pressure and swelling.

The daily recommended sodium limit is **less than 2,000 mg**.

38.5.2 Analyzing Sodium in McDonald's Menu

We can analyze which food item has the **maximum sodium content** using visualization.

38.5.2.1 Creating a Swarm Plot

Use the **Seaborn** package to create a categorical scatter plot:

```
sns.swarmplot(x='Category', y='Sodium', data=df)
```

This plot shows sodium levels across different food categories. A noticeable high value around **3600 mg** indicates extreme sodium content in certain items.

38.5.2.2 Identifying the Highest Sodium Item

To find the food item with the highest sodium:

1. Use the `describe()` method to view sodium summary statistics.
2. Identify the maximum sodium value (3600 mg).
3. Use `idxmax()` to get the index of the highest sodium item.
4. Retrieve the corresponding food item name using the `.at[]` accessor.

Example:

```
index = df['Sodium'].idxmax()
item = df.at[index, 'Item']
```

Result: The item with the highest sodium content is *Chicken McNuggets (40 pieces)*.

38.6 Visualizing Data

38.6.1 Scatter Plot

Scatter plots show relationships between two variables. Use the Seaborn `jointplot()` function to visualize **Protein** vs. **Total Fat**:

```
sns.jointplot(x='Protein', y='Total Fat', data=df)
```

This plot shows a **positive correlation ($r = 0.81$)** between protein and total fat. Histograms on the top and side show the distribution of each variable. Any point far from the pattern may indicate an **outlier**.

38.6.2 Box Plot

Box plots represent the distribution of a variable and help identify outliers. Use the Seaborn `boxplot()` function for **Sugar**:

```
sns.boxplot(x='Sugars', data=df)
```

The average sugar content is around **30 grams**, with outliers near **128 grams**, likely representing high-sugar items such as candies.

38.7 Summary

- SQLite3 provides a simple way to store and query data in Python.
- Pandas is used for data exploration and summary statistics.
- Seaborn helps visualize patterns and relationships through plots.
- We identified *Chicken McNuggets (40 pieces)* as the food item with the highest sodium content.
- Visualizations like scatter and box plots help detect correlations and outliers effectively.

39 Summary: Accessing Databases Using Python

At this point in the course, we know:

39.1 Key Concepts

39.1.1 Magic Commands

- Special commands that provide enhanced functionalities in Jupyter notebooks.

39.1.2 Cell Magics

- Commands prefixed with `%%` that operate on multiple lines of input.

39.1.3 Database API (DB API)

- A standardized interface for connecting Python programs to databases.
- Two main components:
 - **Connection Objects**: Used to connect and manage the database.
 - **Query (Cursor) Objects**: Used to execute SQL queries and fetch data.

39.1.4 Database Cursor

- A control structure that enables traversal over records in a database.

39.2 Using Python Libraries for Database Operations

39.2.1 Pandas

- Equipped with mathematical and statistical methods for data analysis.

- The `pandas.read_csv()` function reads data from CSV files.
- The `read_sql()` method retrieves data from database tables using SQL SELECT queries.

39.2.2 SQLite3

- The `sqlite3.connect()` function establishes a connection to a database.
- Allows creating, reading, updating, and deleting data through SQL commands.

39.3 Data Visualization

- A categorical scatterplot can be created using the **Seaborn** package with the `swarmplot()` method.
- Visualization helps in understanding data distribution and identifying patterns.

39.4 Summary Points

- Python provides built-in and library-supported methods for database access and manipulation.
- Pandas and SQLite3 offer a seamless integration for working with structured data.
- Seaborn enhances analysis by allowing effective visualization of retrieved data.

Module 5

40 Working with Real-World Datasets

40.1 Introduction

In this Section, we explore hints and tips for working with real-world datasets. By the end of this session, we will be able to:

- Discuss the format of data in databases.
- Explain how to load data into databases.

40.2 Understanding Data Formats

Many real-world datasets are provided as **CSV (Comma-Separated Values)** files. These are text files where data values are typically separated by commas, although other separators like semicolons may sometimes be used.

40.2.1 Example: DOGS.csv File

To understand how data is structured, consider a fictional dataset named **DOGS.csv**, which contains the names and breeds of dogs.

40.2.1.1 Sample Structure

ID Name of Dog Breed

1	Wolfie	German Shepherd
2	Fluffy	Pomeranian
3	Huggy	Labrador

The first row usually contains **attribute labels**, which map to column names in a database table.

40.3 Loading CSV Data into Databases

When importing a CSV file into a database using **phpMyAdmin**:

1. Browse and select the CSV file to upload.
2. Choose **CSV** from the dropdown under the *Format* section.
3. Enable the checkbox "**The first line of the file contains the table column names.**"
4. Click **Go** to upload.

MySQL automatically:

- Detects header names as column names.
- Creates and populates the table with appropriate CREATE and INSERT statements.
- Uses the file name (in lowercase) as the default table name.

40.4 Working with Column Names

Column names in CSV files may include spaces or special characters. When querying such columns:

- Use **backticks** (``) around column names.
- Avoid using single (') or double (" ") quotes, as they won't work for column names.

40.5 Writing Readable SQL Queries

For long or complex queries (joins or nested queries), it's recommended to split them across multiple lines for better readability.

40.5.1 In Python Notebooks

- Use a **backslash** (\) at the end of a line to continue the SQL query onto the next line.
- When using **SQL cell magic** (%%sql), you don't need to add a backslash, as the cell is already interpreted as SQL code.

40.6 Querying Databases Using Python

Python's **Pandas** library allows querying tables directly from a database using the `read_sql()` method.

40.6.1 Example Usage

```
query_statement = 'SELECT * FROM dogs LIMIT 3'
df = pandas.read_sql(query_statement, connection)
```

40.6.1.1 Handling Quotes in Queries

- Use single quotes to enclose the query string in Python.
- Use double quotes inside the SQL query to specify column names with spaces or special characters.
- Use a **backslash** (\) as an escape character when including single quotes inside SQL queries (e.g., in WHERE clauses).

40.7 Limiting Query Results

Databases often contain large datasets. To view only a few records, use the **LIMIT** clause.

40.7.1 Example Query

```
SELECT * FROM census_data LIMIT 3;
```

This retrieves only the first three rows of the table.

40.8 Summary

- Real-world datasets are commonly stored as CSV files.
- Use **phpMyAdmin** to import CSV data into databases efficiently.
- Enclose column names with spaces in **backticks** when querying.
- Use **LIMIT** to restrict the number of returned rows.
- In Python, use **Pandas' read_sql()** to query databases.
- Split long queries across multiple lines for better readability.

41 Getting Information About Tables and Columns in a Database

41.1 Introduction

In this section, we explore how to get information about tables and their columns in a database. By the end of this session, we will be able to:

- Discuss how to get a list of tables in a database.
- Discuss how to get a list of tables in an SQLite3 database.

41.2 Retrieving a List of Tables

Sometimes a database may contain several tables, and it can be difficult to remember the exact names. For example, you may wonder whether the table is named **dog**, **dogs**, or **four_legged_mammals**.

Database systems usually have **system or catalog tables** that store metadata about tables and their properties. You can query these catalog tables to retrieve information.

41.2.1 Catalog Tables in Different Database Systems

- **DB2** – SYSCAT.TABLES
- **SQL Server** – INFORMATION_SCHEMA.TABLES
- **SQLite3** – sqlite_master
- **MySQL** – SHOW TABLES

41.2.2 Example: SQLite3

To get a list of tables in an SQLite3 database, use the following query:

```
SELECT name FROM sqlite_master WHERE type='table';
```

41.2.3 Example: MySQL

To get a list of tables in a MySQL database, run the command:

```
SHOW TABLES;
```

41.3 Retrieving Column Information

To extract column names (attributes) or column headers of tables, you can use different commands depending on the database system.

41.3.1 In SQLite3

Use the following command to get details about a table's columns:

```
PRAGMA_table_info(table_name);
```

41.3.2 In MySQL

Use the following command to describe the structure of a table:

```
DESCRIBE table_name;
```

41.4 Summary

- Database systems maintain catalog tables that can be queried to list tables and their properties.
- In **SQLite3**, use:
 - `SELECT name FROM sqlite_master WHERE type='table';`
 - To extract column details in SQLite3, use:

- `PRAGMA table_info(table_name);`
- In **MySQL**, use `SHOW TABLES` to list tables and `DESCRIBE table_name` to view column information.

42 Summary

This reading highlights your progress in working with real-world datasets and understanding database structures.

42.1 Understanding Real-World Datasets

- You have gained practical experience in handling and analyzing data.
- This experience is crucial for data-driven decision-making.

42.2 Database Structure Knowledge

- You learned how to obtain table and column details from databases.
- Understanding these details is essential for designing effective queries.

42.3 Query Design Skills

- The knowledge gained will help you create more efficient SQL queries.
- This skill is vital for data analysis and manipulation tasks.

Module 6

43 Views

43.1 Introduction

In this section, we explore the concept of **Views** in SQL. By the end of this session, we will be able to:

- Define a view.
- Describe when to use a view.
- Explain the syntax for creating a view.

43.2 What Is a View?

A **view** is an alternative way of representing data that exists in one or more tables or other views. A view can include all or some of the columns from one or more base tables or existing views.

Creating a view defines a named specification of a results table that can be queried in the same way as a table. Data in the base table can also be modified through a view using **INSERT**, **UPDATE**, and **DELETE** statements.

When a view is defined, its **definition** is stored, but the **data** remains in the base tables.

43.3 When to Use a View

Views can be used to:

- Show selected data from a table to hide sensitive information such as salaries, tax details, or birth dates.
- Combine data from two or more tables in meaningful ways.
- Simplify access to data by granting access to the view instead of the underlying tables.
- Display only the portions of data relevant to a specific process or user.

Example:

A view can be created on the *Employees* table to display only non-sensitive information such as Employee ID, name, address, job ID, manager ID, and department ID, while excluding salary and birthdate.

43.4 Creating a View

To define a view, use the **CREATE VIEW** statement and assign a name (up to 128 characters). List the columns to include in the view and optionally use aliases for column names.

43.4.1 Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
[WHERE condition];
```

43.4.2 Example

The following statement creates a view called **EMPINFO** based on the *Employees* table:

```
CREATE VIEW EMPINFO AS
SELECT EMPLOYEE_ID, NAME, JOB_ID, MANAGER_ID, DEPARTMENT_ID
FROM EMPLOYEES
WHERE MANAGER_ID = 30002;
```

This view returns only the rows where the **MANAGER_ID** is 30002.

You can query the view as follows:

```
SELECT * FROM EMPINFO;
```

43.5 Characteristics of Views

- Views are **dynamic**—they consist of data returned by the SELECT statement used to define them.
- When used in another SQL statement, a view behaves as though its defining SELECT query is executed.
- A view can reference other views and tables.
- The defining SELECT statement can use **WHERE**, **GROUP BY**, and **HAVING** clauses, but **not ORDER BY** or host variables.

43.6 Removing a View

To remove a view completely from the database, use the **DROP VIEW** statement:

```
DROP VIEW view_name;
```

43.7 Summary

- Views provide an alternate way of accessing data stored in tables.
- They can include specific columns from multiple tables or other views.
- Once created, views can be queried and can allow modification of base table data.
- Views are dynamic; only their definitions are stored, not the data itself.
- The **CREATE VIEW** and **DROP VIEW** statements are used to create and remove views, respectively.

44 Hands-on Exercise: Using Views in MySQL using the CommandLine

44.1 Introduction

In this exercise, we explore **Views** in MySQL using the command line. A view is an alternative way of representing data that exists in one or more tables. Just like a real table, it contains rows and columns, but only the **definition** of the view is stored, not the data.

44.2 Objectives

After completing this exercise, we will be able to:

- Create a View and show a selection of data for a given table.
- Update a View to combine two or more tables in meaningful ways.
- Drop a created View.

44.3 Software Used

In this exercise, we will use **MySQL**, a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.

44.4 Database Used

The database used in this practice is a sample **HR database**, consisting of five tables:

- EMPLOYEES
- JOB_HISTORY
- JOBS
- DEPARTMENTS
- LOCATIONS

Each table contains a few rows of sample data.

44.4.1 Setup Instructions

1. Load the provided database file named **“4_Database_and_Sql_with_Python_Module6_HR”** into your MYSQL command line or phpMyAdmin, which consists of these tables.
2. Tables;
 - Departments.csv
 - Employees.csv
 - Jobs.csv
 - Locations.csv
 - JobsHistory.csv
3. Now perform the exercise.

44.5 Task 1: Create a View

We will create a view called **EMPSALARY** to display employee salary information along with basic details.

44.5.1 Step 1: Create the View

```
CREATE VIEW EMPSALARY AS
SELECT EMP_ID, F_NAME, L_NAME, B_DATE, SEX, SALARY
FROM EMPLOYEES;
```

44.5.2 Step 2: Query the View

Retrieve all records from the view:

```
SELECT * FROM EMPSALARY;
```

44.6 Task 2: Update a View

Now, we will update the **EMPSALARY** view to include more salary details such as **minimum salary**, **maximum salary**, and **job title**.

44.6.1 Step 1: Replace the View

```
CREATE OR REPLACE VIEW EMPSALARY AS
SELECT EMP_ID, F_NAME, L_NAME, B_DATE, SEX, JOB_TITLE, MIN_SALARY, MAX_SALARY
FROM EMPLOYEES, JOBS
WHERE EMPLOYEES.JOB_ID = JOBS.JOB_IDENT;
```

44.6.2 Step 2: Query the Updated View

```
SELECT * FROM EMPSALARY;
```

44.6.3 Note

The technique used here is called an **implicit inner join**, where data from two tables (EMPLOYEES and JOBS) are combined using their related columns — **JOB_ID** and **JOB_IDENT**.

44.7 Task 3: Drop a View

We can delete a view completely from the database using the **DROP VIEW** command.

44.7.1 Step 1: Drop the EMPSALARY View

```
DROP VIEW EMPSALARY;
```

44.7.2 Step 2: Verify Deletion

```
SELECT * FROM EMPSALARY;
```

If the view no longer exists, MySQL will return an error indicating that the table or view does not exist.

44.8 Practice Problems

Try solving the following problems to strengthen your understanding:

44.8.1 Problem 1: Create a New View

Create a view **EMP_DEPT** showing the following information: EMP_ID, F_NAME, L_NAME, and DEP_ID from the EMPLOYEES table.

```
CREATE VIEW EMP_DEPT AS  
SELECT EMP_ID, F_NAME, L_NAME, DEP_ID  
FROM EMPLOYEES;
```

44.8.2 Problem 2: Modify the View

Modify **EMP_DEPT** so that it displays department names instead of department IDs.

```
CREATE OR REPLACE VIEW EMP_DEPT AS  
SELECT EMP_ID, F_NAME, L_NAME, DEP_NAME  
FROM EMPLOYEES, DEPARTMENTS  
WHERE EMPLOYEES.DEP_ID = DEPARTMENTS.DEPT_ID_DEP;
```

44.8.3 Problem 3: Drop the View

Drop the **EMP_DEPT** view.

```
DROP VIEW EMP_DEPT;
```

44.9 Conclusion

We have successfully learned how to work with **Views in MySQL**. In this exercise, we practiced how to:

- Create a new view.
- Modify a view to include data from multiple tables.
- Drop a view when it is no longer needed.

45 Introduction to Stored Procedures

45.1 What is a Stored Procedure?

A **stored procedure** is a set of SQL statements that are stored and executed on the database server. Instead of sending multiple SQL statements from the client to the server, they are

encapsulated within a stored procedure on the server. This allows a single statement from the client to execute a series of commands efficiently.

Stored procedures can be written in various languages. For example, in **DB2 on Cloud** and **DB2**, they can be written in SQL, PL, PL/SQL, Java, or C. Stored procedures can:

- Accept input parameters.
- Perform CRUD (Create, Read, Update, Delete) operations.
- Return results to client applications.

45.2 Benefits of Stored Procedures

Stored procedures offer several advantages, including:

45.2.1 1. Reduced Network Traffic

Only one call is needed to execute multiple SQL statements, minimizing data transmission between client and server.

45.2.2 2. Improved Performance

Processing occurs on the server where data resides, with only the final result returned to the client, enhancing execution speed.

45.2.3 3. Code Reusability

A single stored procedure can be reused by multiple applications to perform the same task.

45.2.4 4. Increased Security

- Reduces exposure of table and column details to client-side developers.
- Allows implementation of server-side data validation logic before data is accepted into the system.

Note: SQL is not a fully fledged programming language, so it is not recommended to write all business logic in stored procedures.

45.3 Creating a Stored Procedure in SQL

To create a stored procedure, use the **CREATE PROCEDURE** statement, specifying the procedure name and any parameters it accepts.

45.3.1 Example

Below is an example of a stored procedure named `UPDATE_SAL`, which updates employee salaries based on performance ratings:

```
DELIMITER $$
```

```

CREATE PROCEDURE UPDATE_SAL (IN EMP_ID INT, IN RATING INT)
LANGUAGE SQL
BEGIN
    IF RATING = 1 THEN
        UPDATE EMPLOYEES SET SALARY = SALARY * 1.10 WHERE EMPLOYEE_ID = EMP_ID;
    ELSE
        UPDATE EMPLOYEES SET SALARY = SALARY * 1.05 WHERE EMPLOYEE_ID = EMP_ID;
    END IF;
END $$
DELIMITER ;

```

45.3.2 Explanation

- The DELIMITER is temporarily changed to \$\$ to allow multiple statements inside the procedure.
- The BEGIN and END keywords define the procedural logic.
- The procedure updates salaries based on employee ratings.
- After defining the procedure, the delimiter is reset to a semicolon (;).

45.4 Calling a Stored Procedure

Stored procedures can be called from external applications or within SQL statements using the **CALL** command.

45.4.1 Example

```
CALL UPDATE_SAL(1001, 1);
```

This executes the UPDATE_SAL procedure for the employee with ID 1001, applying the appropriate salary adjustment.

45.5 Summary

- Stored procedures are sets of SQL statements executed on the database server.
- They improve performance, reduce network traffic, enhance security, and allow code reuse.
- You can create, execute, and manage stored procedures efficiently using SQL commands like CREATE PROCEDURE and CALL.

46 Hands-on Practice Exercise: Stored Procedures

46.1 Overview

Stored Procedures in SQL are special routines that group multiple SQL statements into a single executable unit. They are stored in the database's data dictionary and can be executed directly from applications or the database command interface. Stored procedures can:

- Accept input parameters and return output values.
- Include control flow statements like loops and conditionals.
- Improve performance, scalability, and code organization.
- Enforce business logic and data integrity at the database level.

46.2 Objectives

After completing this exercise, we will be able to:

- Create stored procedures.
- Execute stored procedures.

46.3 Software Used

In this exercise, we will use **MySQL**, a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.

46.4 Database Used

We will use the **MySQL_learners** database for this exercise.

46.5 Data Used

- Utilize the given database named **“5_database_and_sql_with_python_module6_pets”**.
- The exercise will utilize internal data from the **PETSALE** table inside the given database.
- Just load the database in your SQL terminal on phpMyAdmin or on your MYSQL command terminal of your computer and perform the “Procedure Exercise”.
- Ensure that the PETSALE table is populated with sample data in phpMyAdmin.

46.5.1 Setting Up the Database

1. Just import the given database file into your phpMyAdmin or on the MYSQL command line of your terminal.
2. The script will create and populate the PETSALE table with sample data.

47 Stored Procedure: Exercise 1 – Retrieving Data

47.1 Goal

Create and execute a stored procedure to read all records from the PETSALE table.

47.1.1 Step 1: Create the Stored Procedure

Use the following SQL script to create the procedure RETRIEVE_ALL:

```
DELIMITER //  
CREATE PROCEDURE RETRIEVE_ALL()  
BEGIN  
    SELECT * FROM PETSALE;  
END //  
DELIMITER ;
```


47.1.2 Step 2: Execute the Stored Procedure

To execute the procedure, open a new SQL tab in phpMyAdmin and run:

```
CALL RETRIEVE_ALL;
```

47.1.3 Step 3: View or Drop the Procedure

- To view the created procedure, expand the **PETS** database and click on **Procedures**.
- To drop the procedure, use:

```
DROP PROCEDURE RETRIEVE_ALL;
```

48 Stored Procedure: Exercise 2 – Updating Data

48.1 Goal

Create and execute a stored procedure to update the sale price of animals based on their health condition.

48.1.1 Step 1: Create the Stored Procedure

Use the following code to create the UPDATE_SALEPRICE routine:

```
DELIMITER @
CREATE PROCEDURE UPDATE_SALEPRICE (IN Animal_ID INTEGER, IN Animal_Health
VARCHAR(5))
BEGIN
    IF Animal_Health = 'BAD' THEN
        UPDATE PETALE
        SET SALEPRICE = SALEPRICE - (SALEPRICE * 0.25)
        WHERE ID = Animal_ID;
    ELSEIF Animal_Health = 'WORSE' THEN
        UPDATE PETALE
        SET SALEPRICE = SALEPRICE - (SALEPRICE * 0.5)
        WHERE ID = Animal_ID;
    ELSE
        UPDATE PETALE
        SET SALEPRICE = SALEPRICE
        WHERE ID = Animal_ID;
    END IF;
END @
DELIMITER ;
```

48.1.2 Step 2: Execute the Stored Procedure

To test the procedure, open a new SQL tab and execute the following commands:

```
CALL RETRIEVE_ALL;
CALL UPDATE_SALEPRICE(1, 'BAD');
CALL RETRIEVE_ALL;
```

Next, execute the procedure again for another record:

```
CALL RETRIEVE_ALL;
CALL UPDATE_SALEPRICE(3, 'WORSE');
```

```
CALL RETRIEVE_ALL;
```

48.1.3 Step 3: View or Drop the Procedure

- To view the UPDATE_SALEPRICE routine, click on **Routines** under the **PETS** database.
- To drop the routine, run:

```
DROP PROCEDURE UPDATE_SALEPRICE;
```

49 Conclusion

Congratulations! We have successfully completed the exercise on creating and executing stored procedures in MySQL.

Now, we can:

- Write stored procedures to perform automated SQL operations.
- Execute and manage stored procedures using SQL commands.
- Drop stored procedures when they are no longer needed.

50 ACID Transactions

50.1 Overview

ACID transactions ensure that database operations are executed reliably, maintaining data integrity even in complex processes. A transaction is an indivisible unit of work that can consist of one or more SQL statements. It is successful only if all SQL statements within it execute successfully—otherwise, none of them should execute.

50.2 Example of an ACID Transaction

Consider a purchase transaction using a bank card. Several operations occur:

- The product is added to the cart.
- The payment is processed.
- The customer's account is debited.
- The store's account is credited.
- The inventory is reduced by the number of items purchased.

For instance, if **Rose buys boots for \$200**, the following SQL statements might occur:

- An **UPDATE** statement decreases Rose's account balance by \$200.
- Another **UPDATE** statement increases the Shoe Shop's balance by \$200.
- A final **UPDATE** statement decreases the Shoe Shop's inventory by 1.

If any of these **UPDATE** statements fail, the entire transaction should fail to maintain data consistency.

50.3 Components of ACID

The term **ACID** stands for four key properties that ensure reliable database transactions:

50.3.1 Atomic

All changes within the transaction must be completed successfully, or none at all.

50.3.2 Consistent

The data must remain in a consistent state before and after the transaction.

50.3.3 Isolated

No other process can modify the data while the transaction is running.

50.3.4 Durable

Changes made by the transaction must persist, even in the event of a system failure.

50.4 Managing ACID Transactions

To start an ACID transaction, use the **BEGIN** command. In **DB2 on Cloud**, this command is implicit. The commands that follow are part of the transaction until either a **COMMIT** or **ROLLBACK** command is issued.

50.4.1 COMMIT

If all commands complete successfully, issue a **COMMIT** command to save the transaction and bring the database to a stable, consistent state.

50.4.2 ROLLBACK

If any command fails (for example, Rose's account lacks sufficient funds), issue a **ROLLBACK** command to undo all changes, restoring the database to its previous consistent state.

50.5 Using SQL Commands in Programming Languages

SQL statements can be executed from programming languages such as **Java**, **C**, **R**, and **Python** using specific database APIs:

- **Java** uses **Java Database Connectivity (JDBC)**.
- **Python** can use connectors such as **ibm_db**.

Most languages utilize the **EXEC SQL** command to run SQL statements, including **COMMIT** and **ROLLBACK**. The **BEGIN** command is typically implicit.

By incorporating SQL commands into application code, developers can create **error-checking routines** that determine whether a transaction should be committed or rolled back.

50.6 Summary

- A transaction represents a complete unit of work that may include multiple SQL statements.
- An **ACID transaction** ensures that either all statements execute successfully or none do.
- **ACID** stands for **A**tomic, **C**onsistent, **I**solated, and **D**urable.
- **BEGIN**, **COMMIT**, and **ROLLBACK** commands manage transaction flow.
- SQL transactions can be integrated into programming languages such as **C**, **R**, and **Python** for reliable application development.

51 Hands-on Lab: Committing and Rolling Back a Transaction

51.1 Overview

A **transaction** is a sequence of operations performed using one or more SQL statements as a single logical unit of work. A database transaction must follow the **ACID** properties — **A**tomic, **C**onsistent, **I**solated, and **D**urable.

All SQL statements in a transaction can either be **applied** to the database using the **COMMIT** command or **undone** using the **ROLLBACK** command.

In this exercise, we will explore **Transaction Control Language (TCL)** commands in SQL through the creation of stored procedures. We will learn how **COMMIT** permanently saves the changes done in a transaction and how **ROLLBACK** undoes the changes that have not yet been saved.

51.2 Objectives

After completing this exercise, we will be able to:

- Permanently save the changes done in a transaction.
- Undo a transaction that has not been saved.

51.3 Software Used

MySQL is a **Relational Database Management System (RDBMS)** designed to efficiently store, manipulate, and retrieve data.

51.4 Database Used

The **MySQL_learners** database will be used in this exercise.

51.5 Data Used

We will work on two tables: **BankAccounts** and **ShoeShop** present inside this given database named **“6_Databases_and_Sql_with_Python_Module6_ACCOUNTS”**.

- Utilize the given database named **“6_Databases_and_Sql_with_Python_Module6_ACCOUNTS”**.
- The exercise will utilize internal data from the **BankAccounts** and **ShoeShop** tables inside the given database.
- Just load the database in your SQL terminal on phpMyAdmin or on your MYSQL command terminal of your computer and perform the “Procedure Exercise”.
- Ensure that the **Accounts** table is populated with sample data in phpMyAdmin.

51.5.1 Setting Up the Database

1. Just import the given database file into your phpMyAdmin or on the MYSQL command line of your terminal.

51.6 Example: Committing and Rolling Back a Transaction

51.6.1 Scenario

Rose is buying a pair of boots from **ShoeShop**. We must:

- Update Rose’s balance and ShoeShop’s balance in the **BankAccounts** table.
- Update the **Boots** stock in the **ShoeShop** table.
- Then attempt to buy Rose a pair of **Trainers**.

51.6.2 Stored Procedure: TRANSACTION_ROSE

Once the tables are ready, create a stored procedure named **TRANSACTION_ROSE** that includes **COMMIT** and **ROLLBACK**.

```
DELIMITER //
CREATE PROCEDURE TRANSACTION_ROSE()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
    UPDATE BankAccounts
    SET Balance = Balance-200
    WHERE AccountName = 'Rose';

    UPDATE BankAccounts
    SET Balance = Balance+200
    WHERE AccountName = 'Shoe Shop';

    UPDATE ShoeShop
    SET Stock = Stock-1
```

```

WHERE Product = 'Boots';

UPDATE BankAccounts
SET Balance = Balance-300
WHERE AccountName = 'Rose';
COMMIT;
END //
DELIMITER ;

```

51.6.3 Executing the Procedure

To test the transaction, run the following commands:

```

CALL TRANSACTION_ROSE;
SELECT * FROM BankAccounts;
SELECT * FROM ShoeShop;

```

51.6.4 Observation

When we observe the tables, no changes are permanently saved through **COMMIT**. The transaction fails, so all changes are undone through **ROLLBACK**. This happens because the last **UPDATE** statement fails (Rose's balance is insufficient for buying Trainers). Since one SQL statement failed, the entire transaction fails and rolls back.

51.6.5 Expected Outcome

- The first three **UPDATE** statements run successfully:
 - Rose's balance decreases by \$200.
 - ShoeShop's balance increases by \$200.
 - The stock of Boots decreases by 1.
- The last **UPDATE** statement fails due to insufficient funds, triggering a **ROLLBACK**.

51.7 Practice Exercise

Create a stored procedure **TRANSACTION_JAMES** based on the following scenario:

Scenario: James buys 4 pairs of Trainers from **ShoeShop**. Update his balance, the ShoeShop balance, and the Trainers stock. Then attempt to buy him a pair of **Brogues**. If any **UPDATE** statement fails, roll back the entire transaction. Commit only if all statements are successful.

51.7.1 Hint

Use **COMMIT** and **ROLLBACK** as demonstrated in the example.

51.7.2 Solution

```

DELIMITER //
CREATE PROCEDURE TRANSACTION_JAMES()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
  START TRANSACTION;

```

```
UPDATE BankAccounts  
SET Balance = Balance-1200  
WHERE AccountName = 'James';
```

```
UPDATE BankAccounts  
SET Balance = Balance+1200  
WHERE AccountName = 'Shoe Shop';
```

```
UPDATE ShoeShop  
SET Stock = Stock-4  
WHERE Product = 'Trainers';
```

```
UPDATE BankAccounts  
SET Balance = Balance-150  
WHERE AccountName = 'James';  
COMMIT;  
END //  
DELIMITER ;
```

51.8 Conclusion

We have successfully learned how to manage transactions using **COMMIT** and **ROLLBACK** in MySQL.

51.8.1 Key Takeaways

- **COMMIT** permanently saves a transaction.
- **ROLLBACK** undoes changes from unsuccessful transactions.
- Transactions ensure data integrity by following **ACID** principles.
- Stored procedures make it easier to manage complex transactional logic across multiple tables.

52 Summary: Views, Stored Procedures, and Transactions

52.1 Overview

Congratulations! We have completed this lesson. At this stage, we have a clear understanding of how **views**, **stored procedures**, and **transactions** operate in SQL.

52.2 Views

- **Views** are dynamic mechanisms for presenting data from one or more tables.
- They simplify data access by allowing users to retrieve specific subsets of information without directly querying complex table structures.

52.3 Transactions

- A **transaction** represents a complete unit of work, which can consist of one or more SQL statements.
- An **ACID transaction** ensures that all SQL statements within the transaction complete successfully or none of them do.

- The **COMMIT** command permanently saves all changes made during the transaction, while the **ROLLBACK** command undoes any uncommitted changes.

52.4 Stored Procedures

- A **stored procedure** is a set of SQL statements that are stored and executed on the database server.
- Stored procedures allow the execution of multiple SQL statements using a single command, improving efficiency and security.
- They can be written in various languages, including **SQL PL**, **PL/SQL**, **Java**, and **C**.

52.5 Key Takeaways

- **Views** provide a simplified way to access complex data.
- **Transactions** ensure consistency and reliability by adhering to the ACID properties.
- **Stored Procedures** streamline database operations by grouping multiple SQL statements into one executable unit.

53 Introduction to JOIN Overview

53.1 Understanding the JOIN Operator

The **JOIN operator** in SQL is used to combine rows from two or more tables based on a related column between them. It allows retrieval of complex data from multiple tables by establishing relationships through shared keys.

A simple **SELECT** statement retrieves data from one or more columns in a single table. However, to extract information spanning multiple tables, the **JOIN** operation is required.

53.2 Role of Primary Keys and Foreign Keys

In relational databases, **Primary Keys** and **Foreign Keys** play an essential role in connecting tables:

- **Primary Key (PK):** A unique identifier for each record in a table.
- **Foreign Key (FK):** A field (or set of fields) in one table that refers to the Primary Key of another table.

53.2.1 Example of Relationship

In a library database:

- **Author** and **Book** are entities represented in separate tables.
- The **Book** table may contain an `author_id` that acts as a **Foreign Key**, referring to the **Primary Key** in the **Author** table.

If we need to know *which borrower has which book copy on loan*, data must be gathered from **three tables**: Borrower, Loan, and Copy. This requires **JOIN operations** to link related data.

53.3 Establishing Relationships Between Tables

To join tables correctly, you must first identify the **common columns** that link them. For instance:

- Borrower ID connects the **Borrower** and **Loan** tables.
- Copy ID connects the **Loan** and **Copy** tables.

The **Borrower ID** serves as a **Foreign Key** in the **Loan** table, referring to the **Primary Key** in the **Borrower** table.

53.4 Combining Multiple Tables

To combine data from multiple tables:

1. Start by joining two tables (e.g., Borrower and Loan) using the matching **Borrower ID**.
2. Then, add another table (e.g., Copy) by joining it with the **Loan** table using the **Copy ID**.

You can continue to extend joins in this way to include additional tables.

53.5 Types of JOIN Operators

SQL provides several types of **JOIN** operations that determine how records are matched and returned:

53.5.1 Inner Join

- Returns only the rows where there is a **match** between both tables.
- Typically, it matches a **Primary Key** in one table with a **Foreign Key** in another.

53.5.2 Outer Join

- Returns **matching rows** and also includes rows from one or both tables that do **not have matches**.
- There are multiple kinds of outer joins, such as **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, and **FULL OUTER JOIN**.

53.6 Summary

- The **JOIN operator** combines rows from two or more tables based on common columns.
- The relationship between tables is usually defined by a **Primary Key** in one table and a **Foreign Key** in another.
- The main types of joins are **Inner Joins** and **Outer Joins**.
- **Inner Joins** return only matched records, while **Outer Joins** can also return unmatched records.

54 Introduction to Inner Join

54.1 Understanding Inner Joins

An **Inner Join** is one of the most commonly used SQL joins. It combines rows from two or more tables based on a relationship between certain columns, typically a **Primary Key** in one table and a **Foreign Key** in another.

When using an Inner Join, only the rows with **matching values** in the common column of both tables are displayed in the result set. Rows without matches in either table are excluded.

54.2 Syntax of the INNER JOIN Statement

The basic syntax for an **INNER JOIN** is as follows:

```
SELECT column_names  
FROM table1 INNER JOIN table2  
ON table1.column_name = table2.column_name;
```

54.2.1 Example Scenario

Imagine you want to retrieve a list of all people who are borrowing books along with the date of the loan. You need data from both the **Borrower** and **Loan** tables.

In the **FROM** clause, specify the join as follows:

```
FROM BORROWER AS B INNER JOIN LOAN AS L  
ON B.BORROWER_ID = L.BORROWER_ID;
```

54.2.2 Explanation of Clauses

- The table listed on the **left side** of the JOIN (in this case, BORROWER) is known as the **left table**.
- The table listed on the **right side** of the JOIN (LOAN) is the **right table**.
- The **ON clause** defines the condition for joining both tables — here, the **Borrower_ID** in the Borrower table must match the **Borrower_ID** in the Loan table.

54.2.3 Use of Aliases

In the above query, the tables are given **aliases** (B for Borrower and L for Loan). Prefixing column names with these aliases makes the query easier to read and write, for example:

```
SELECT B.BORROWER_ID, B.LASTNAME, B.COUNTRY, L.LOAN_DATE  
FROM BORROWER AS B INNER JOIN LOAN AS L  
ON B.BORROWER_ID = L.BORROWER_ID;
```

54.2.4 Result Set Explanation

- The result displays only rows where the **Borrower_ID** in both tables matches.
- Columns such as **Borrower_ID**, **LastName**, and **Country** are taken from the **Borrower** table.
- Columns such as **Borrower_ID** and **Loan_Date** are taken from the **Loan** table.

- Rows with **non-matching Borrower_IDs** do not appear in the result.

54.3 Key Points to Remember

- **Inner Joins** return only the rows with matching values in a common column.
- Typically, this involves a **Primary Key** in one table and a **Foreign Key** in another.
- **Unmatched rows** are not displayed in the result set.
- Aliases can make SQL queries shorter and more readable.

54.4 Summary

In this section, we learned:

- The concept of **Inner Joins** and when to use them.
- The **syntax** of the INNER JOIN statement.
- How **aliases** simplify query writing.
- That **Inner Joins** return only the rows that have matching values in both tables.

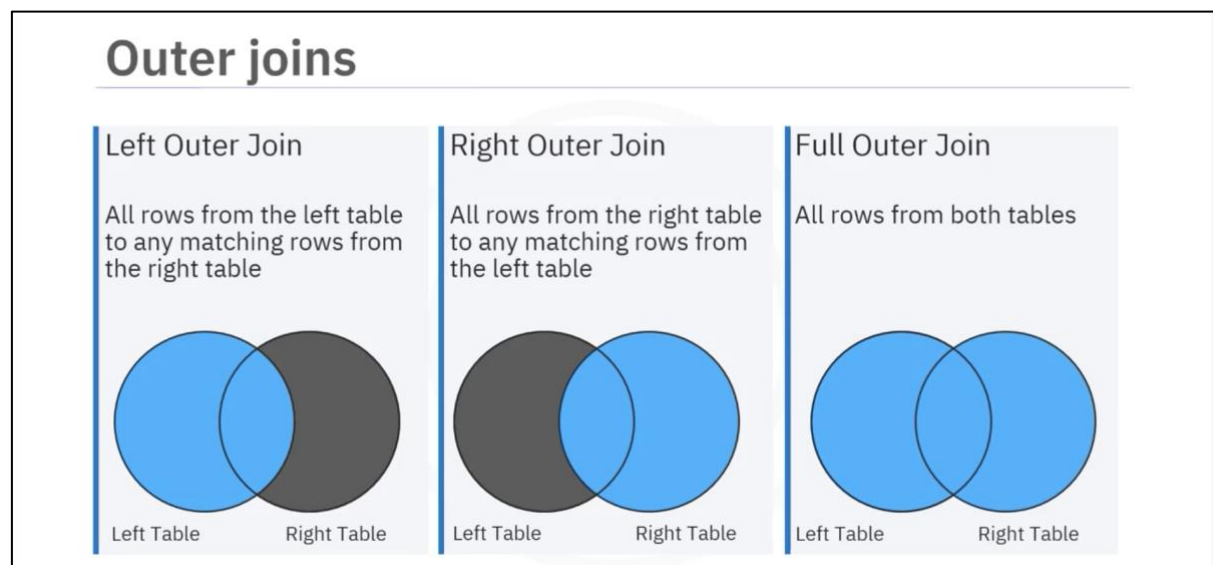
55 Introduction to Outer Join

55.1 Understanding Outer Joins

Outer joins, like inner joins, return rows from each table that have matching values in the join columns. However, **unlike inner joins**, outer joins also return rows that do not have a match between the tables.

SQL provides three types of outer joins:

- **Left Outer Join**
- **Right Outer Join**
- **Full Outer Join**



55.2 Left Outer Join

In a **Left Outer Join**, all rows from the first table (on the left side of the join predicate) are included, along with matching rows from the second table (on the right side of the join predicate). Rows from the left table that do not have matches in the right table will still appear in the result set, with **NULL values** for columns from the right table.

55.2.1 Syntax of LEFT OUTER JOIN

```
SELECT column_names  
FROM table1 LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

55.2.2 Example Scenario

In this example, the **Borrower** table is the left table, and the **Loan** table is the right table:

```
SELECT B.BorrowerID, B.LastName, B.Country, L.LoanDate  
FROM Borrower AS B LEFT JOIN Loan AS L  
ON B.BorrowerID = L.BorrowerID;
```

55.2.3 Explanation of the Result Set

- The result displays each **BorrowerID** from the Borrower table and the corresponding **LoanDate** from the Loan table.
- Borrowers without any loan records will show **NULL values** in the LoanDate column.

55.3 Right Outer Join

A **Right Outer Join** returns all rows from the right table and the matching rows from the left table. Rows from the right table that do not have matches in the left table are included in the result set with **NULL values** for columns from the left table.

55.3.1 Syntax of RIGHT OUTER JOIN

```
SELECT column_names  
FROM table1 RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

55.3.2 Example Scenario

Here, the **Loan** table is the right table and the **Borrower** table is the left table:

```
SELECT L.Borrower_ID, L.LoanDate, B.LastName, B.Country  
FROM Borrower AS B RIGHT JOIN Loan AS L  
ON B.Borrower_ID = L.Borrower_ID;
```

55.3.3 Explanation of the Result Set

- The result displays each **Borrower_ID** from the Loan table and the corresponding borrower information from the Borrower table.
- For rows in the Loan table that do not have a matching Borrower_ID, **NULL values** appear for the Borrower details.
- This may indicate data issues such as a loan linked to an unknown borrower.

55.4 Full Outer Join

A **Full Outer Join** combines the results of both left and right joins. It returns all rows from both tables, with **NULL values** where there is no match.

55.4.1 Syntax of FULL OUTER JOIN

```
SELECT column_names  
FROM table1 FULL JOIN table2  
ON table1.column_name = table2.column_name;
```

55.4.2 Example Scenario

For a full outer join between the **Borrower** and **Loan** tables:

```
SELECT B.Borrower_ID, B.LastName, B.Country, L.LoanDate  
FROM Borrower AS B FULL JOIN Loan AS L  
ON B.Borrower_ID = L.Borrower_ID;
```

55.4.3 Explanation of the Result Set

- The result includes **all records** from both tables.
- Borrowers without loan records display **NULL** in LoanDate.
- Loans with no matching borrower show **NULL** in Borrower details.

55.5 Key Points to Remember

- **Left Outer Join:** Returns all rows from the left table and matching rows from the right table.
- **Right Outer Join:** Returns all rows from the right table and matching rows from the left table.
- **Full Outer Join:** Returns all matching and non-matching rows from both tables.

55.6 Summary

In this section, we learned:

- The concept and types of **Outer Joins** (Left, Right, and Full).
- The **syntax** for each type of outer join.
- How outer joins help retrieve both matching and non-matching data from multiple tables.

56 Hands-on Practice: Working with Joins in MySQL using phpMyAdmin

56.1 Introduction

SQL **JOIN** is a clause that combines rows from two or more tables based on a related column between them. The relationship is established by comparing the values in specific columns. Using JOINS helps us retrieve data from multiple tables in a single query.

There are four main types of JOINS in SQL:

- **INNER JOIN:** Returns only the rows with matching values in both tables.
- **LEFT JOIN:** Returns all rows from the left table and matching rows from the right table.
- **RIGHT JOIN:** Returns all rows from the right table and matching rows from the left table.
- **FULL OUTER JOIN:** Returns all rows from both tables, including non-matching ones.

56.2 Objectives

By the end of this exercise, we will be able to:

- Write SQL queries on multiple tables using **INNER JOINS**.
- Write SQL queries on multiple tables using **OUTER JOINS**.

56.3 Software Used

In this exercise, we will use **MySQL**, a Relational Database Management System (RDBMS) that efficiently stores, manipulates, and retrieves data.

56.4 Database Overview

We will work on a given sample **EMPLOYEES database** named **“7_Databases_and_Sql_with_Python_EMPLOYEES”** consisting of five tables:

- **EMPLOYEES**
- **JOB_HISTORY**
- **JOBS**
- **DEPARTMENTS**
- **LOCATIONS**

Each table contains a few rows of sample data. Ensure all five tables are populated before running the queries.

56.5 Loading the Database

Utilize the given database named **“7_Databases_and_Sql_with_Python_EMPLOYEES”**.

- Just load the database in your SQL terminal on phpMyAdmin or on your MYSQL command terminal of your computer and perform the “Joins Exercise”.
- Ensure that the **Accounts** table is populated with sample data in phpMyAdmin.

56.5.1 Setting Up the Database

2. Just import the given database file into your phpMyAdmin or on the MYSQL command line of your terminal, and start practicing by running the SQL join queries.

56.6 Working with JOINS

Let's practice some examples of JOIN operations using the HR database.

56.6.1 Example 1: Retrieve Names and Job Start Dates of Employees in Department 5

We will use an **INNER JOIN** between EMPLOYEES and JOB_HISTORY tables based on EMP_ID, and filter for Department ID = 5.

```
SELECT E.F_NAME, E.L_NAME, JH.START_DATE
FROM EMPLOYEES AS E
INNER JOIN JOB_HISTORY AS JH
ON E.EMP_ID = JH.EMPL_ID
WHERE E.DEP_ID = '5';
```

56.6.2 Example 2: Retrieve Employee ID, Last Name, Department ID, and Department Name

We will use a **LEFT OUTER JOIN** between EMPLOYEES (left table) and DEPARTMENTS (right table) based on Department ID.

```
SELECT E.EMP_ID, E.L_NAME, E.DEP_ID, D.DEP_NAME
FROM EMPLOYEES AS E
LEFT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP;
```

This query retrieves all employees, including those without a department. Missing department details will appear as NULL.

56.6.3 Example 3: Retrieve First Name, Last Name, and Department Name of All Employees

We will simulate a **FULL OUTER JOIN** (not natively supported in MySQL) using a UNION of LEFT and RIGHT OUTER JOINS.

```
SELECT E.F_NAME, E.L_NAME, D.DEP_NAME
FROM EMPLOYEES AS E
LEFT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP
UNION
SELECT E.F_NAME, E.L_NAME, D.DEP_NAME
FROM EMPLOYEES AS E
RIGHT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP;
```

56.7 Practice Problems

56.7.1 Problem 1

Retrieve the names, job start dates, and job titles of all employees who work for department number 5.

```
SELECT E.F_NAME, E.L_NAME, JH.START_DATE, J.JOB_TITLE
FROM EMPLOYEES AS E
```

```
INNER JOIN JOB_HISTORY AS JH ON E.EMP_ID = JH.EMPL_ID
INNER JOIN JOBS AS J ON E.JOB_ID = J.JOB_IDENT
WHERE E.DEP_ID = '5';
```

56.7.2 Problem 2

Retrieve employee ID, last name, and department ID for all employees, but department names only for those born before 1980.

```
SELECT E.EMP_ID, E.L_NAME, E.DEP_ID, D.DEP_NAME
FROM EMPLOYEES AS E
LEFT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP
AND YEAR(E.B_DATE) < 1980;
```

56.7.3 Problem 3

Retrieve the first name and last name of all employees, but department ID and department names only for male employees.

```
SELECT E.F_NAME, E.L_NAME, D.DEPT_ID_DEP, D.DEP_NAME
FROM EMPLOYEES AS E
LEFT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP AND E.SEX = 'M'
UNION
SELECT E.F_NAME, E.L_NAME, D.DEPT_ID_DEP, D.DEP_NAME
FROM EMPLOYEES AS E
RIGHT OUTER JOIN DEPARTMENTS AS D
ON E.DEP_ID = D.DEPT_ID_DEP AND E.SEX = 'M';
```

56.8 Conclusion

Congratulations! We have completed this exercise and are ready for the next topic.

We can now:

- Query multiple tables using **INNER JOINS**.
- Query multiple tables using **LEFT/RIGHT OUTER JOINS**.
- Query multiple tables using **FULL OUTER JOINS**.

57 Summary: JOIN Statements

At this point in the course, we know:

- A **JOIN** combines the rows from two or more tables based on a relationship between certain columns in these tables.
- To combine data from three or more different tables, we simply add new **JOINS** to the SQL statement.
- There are two main types of **JOINS**:
 - **Inner Join**
 - **Outer Join**

And three subtypes of **Outer Joins**:

- **Left Outer Join**
- **Right Outer Join**
- **Full Outer Join**